Termpaper

ELL305

Unified Adaptive Memory Manager

for Key–Value Cache Management in Large Language Model Inference on Edge Processors

Ishanvi Prasad (2024EE10406)

Sweety (2024MT10535)

Source code: `https://github.com/IshanviPrasad29/uamm2-kv-cache-manager`

Department of Electrical Engineering, IIT Delhi

Copyright Assignment

All intellectual property rights (IPR) arising from this work belong to the Indian Institute of Technology Delhi (IIT Delhi). In the case of cited material, the intellectual property rights belong to the respective authors of the original sources.

Ishanvi Prasad
2024EE10406

Sweety
2024MT10535

Acknowledgements

We thank the course instructor for the guidance he gave us while this term paper took shape. The discussions at the start, when we were still deciding what the problem was, saved us a lot of time.

We also thank the Department of Electrical Engineering, IIT Delhi, for the academic environment and the computing facilities, and the Institute library for access to the papers we needed.

The open source tools we used (Verilator, Yosys, GTKWave, NumPy, matplotlib and others) made it possible for us to build and check our hardware design on ordinary machines. We are grateful to the people who maintain them.

Ishanvi Prasad
Sweety

Abstract

A language model that runs on a small processor has plenty of arithmetic available. What it lacks is memory. Every generated token adds a key and a value vector to the KV cache for each layer and head, the cache has to be read again at the next token, and it shares a few gigabytes with the model weights. When the cache outgrows the fast memory, the processor spends its time moving data.

This paper describes how we approached that problem. We first built UAMM-1, a memory manager that splits the cache into blocks, scores each block with an H2O-style importance value and places it in one of three memory tiers using fixed thresholds. Reviewing it, we found that it had no way of making data smaller, so when memory ran out it could only delete tokens, hence accuracy was less. We then built UAMM-2, in which a block is stored in a (tier, codec) pair. Cold blocks are compressed to 8, 4 or 2 bits instead of being deleted, a utility planner chooses the pair for each block, precision can only go down, and attention reads only the blocks that can matter for the current query. We also wrote three Verilog blocks and a RISC-V vector kernel for the inner loops.

On a synthetic trace with assumed memory costs, a 168 KB budget holds 1,536 tokens that FP16 would need 393 KB for. UAMM-1 had to delete 63 % of the tokens to fit; UAMM-2 keeps all of them with a mean attention-output error between 0.12 and 0.17 (all these came as results of simulations). A simple baseline that stores every block in 4-bit also fits and is faster in our model, so our planner adds lower error at the same size, a smaller size, and the ability to cope with tighter budgets. The Verilog blocks match their software models bit for bit and every injected fault was caught. The vector kernel was compiled but never run, no real language model was used, and the latency figures are assumptions. The report states these limits where each result appears, lists the future scopes, and gives the repository in which all of the work can be reproduced.

Contents

Abstract **3**

1 Problem Description **1**

1.1 Why LLMs on edge devices at all 1

1.2 How an LLM generates text: prefill and decode 2

1.3 What the KV cache is 2

1.4 Why the KV cache is the bottleneck on the edge 3

1.5 Why the obvious fixes fall short 5

1.6 Problem statement and design requirements 5

2 Literature Review **7**

2.1 What we took from the architecture textbook 7

2.2 A taxonomy of approaches 8

2.3 Changing the attention architecture 8

2.4 Memory management: PagedAttention and vLLM 9

2.5 Token eviction 9

2.6 Query-aware selection and retrieval 12

2.7 Quantisation of the KV cache 12

2.8 Low-rank compression 14

2.9 Offloading and flash 14

2.10 Joint systems 14

2.11 Hardware: engines and vector processors 15

2.12 Deployment runtime: compiler-based runtimes 16

2.13 Coverage matrix 17

2.14 Summary of the gaps 17

2.15 What we learned from each paper and how we used it 18

3 Mind Map of the Project **20**

4 UAMM-1: Our First Prototype **21**

4.1 How we arrived at the problem 21

4.2 What we saw in the existing work 21

4.3 First Approach: UAMM-1 22

4.4 Gaps in UAMM-1 23

4.5 Implementing UAMM-1: the code, how to run it and where the data comes from . . . 24

5 UAMM-2: Problems We Found and How We Corrected Them 28

5.1 The idea behind UAMM-2 . . . 28
5.2 How UAMM-2 is built . . . 28
5.3 The System Design of UAMM-2 . . . 28
5.4 UAMM-2 code: structure and how to run it . . . 37
5.5 Project Structure and Code Walkthrough . . . 37
5.6 What we tried, what we saw and what we decided . . . 40
5.7 Problems in the design . . . 42

6 Simulation Results and What We Obtained 45

6.1 What we assumed . . . 45
6.2 Experimental Methodology . . . 45
6.3 UAMM-1 and UAMM-2 compared . . . 48
6.4 Simulation Results . . . 50
6.5 Hardware Results: RTL Simulation, Waveforms and Verification . . . 62
6.6 The Ara Vector Kernel: Results . . . 68
6.7 Scaling the Ratios to Larger Models: Arithmetic Only . . . 70
6.8 Conclusions we can draw . . . 72
6.9 Did we do anything new? . . . 72

7 Future Scope 74

7.1 Scopes on precision . . . 74
7.2 Scopes on placement and policy . . . 77
7.3 Scopes on systems and hardware . . . 79
7.4 Order of work . . . 79

8 Project Timeline 81

8.1 From 10 to 24 August: finding the problem . . . 81
8.2 From 24 August to November: building and checking . . . 81

9 Repository, Git Setup and Summary 83

9.1 What is in the repository . . . 83
9.2 Setting it up with Git . . . 83
9.3 Summary . . . 83

References **85**

A Plagiarism Report **87**

B Readability Indices **93**

C Raw Experiment Tables **94**

D Core Source Listings **96**

E UAMM-1 Source Code (Original Prototype) **98**

List of Tables **100**

List of Figures **103**

Glossary **106**

Index **107**

1. Problem Description

Large language models (LLMs) have moved from data centre curiosities to components that product teams now want inside phones, laptops, vehicles, wearables, industrial controllers and embedded boards. The motivations are practical and well understood: **latency** (no network round trip), **privacy** (prompts and documents never leave the device), **availability** (the feature works offline) and **cost** (no per-token cloud bill). This report is about one specific obstacle that stands between the idea and a working product: the memory that a running LLM needs for its *attention history*, the **key value (KV) cache**, and how that memory can be managed on the small, power constrained processors that live at the edge.

The report is organised as follows. Chapter 1 (this chapter) describes the problem. Chapter 2 is the literature review: the textbook chapters and the research papers we read, what we learned from each and how we used it. Chapter 3 is a step-wise mind map of the whole project. Chapter 4 covers UAMM-1: how we reached the problem, what the prototype is, the gaps we found in it, and its code, how to run it and where its data comes from. Chapter 5 covers UAMM-2: the design, the code, the decisions we made by experiment or by reasoning, and the problems we found and corrected. Chapter 6 gives the simulation and hardware (verilog) results, what we assumed and what we think is new. Chapter 7 lists the future scope. Chapter 8 is the project timeline and Chapter 9 explains how to set up the repository with Git and summarises the work. The references come next. The appendices hold the readability indices, raw tables and source listings. The lists of tables and figures, the glossary and the index are at the end.

1.1 Why LLMs on edge devices at all

An *edge processor* in this report means a processor that runs the model locally rather than in a data centre: a mobile application processor, an embedded RISC-V or Arm SoC, a single-board computer, a small NPU-equipped module. Typical constraints, compared with a data-centre accelerator, are shown in the table below.

Table 1.1: Order-of-magnitude comparison between a data-centre accelerator and an edge SoC. Ranges are representative, not measurements of one device.

Resource	Data-centre accelerator	Edge SoC (typical range)	Consequence
Memory capacity	80–192 GB HBM per accelerator	2–16 GB shared LPDDR, often shared with the OS and other apps	Weights and KV cache compete for the same few gigabytes
Memory bandwidth	2–5 TB/s (HBM)	10–60 GB/s (LPDDR4/5)	Two orders of magnitude less; decode is bandwidth-bound
Compute	Hundreds of TFLOP/s	0.1–40 TOPS (NPU/GPU/vector unit)	Compute is rarely the first limit during decoding
Power / thermal	300–700 W	2–15 W sustained	Every DRAM access costs energy that the battery must pay
Storage tier	NVMe, network storage	eMMC / UFS / SD flash	Flash exists but is 10–100× slower than DRAM
Batching	Dozens of concurrent requests	Usually one user, one sequence	Batching cannot amortise weight reads

The right-hand column is the crux. Almost every technique that makes LLM serving efficient in the cloud relies on batching many requests together so that each read of the weights is shared by many sequences. An edge device typically serves one sequence at a time, so the cost of reading the model is paid by a single user. That makes the work of decoding a single token almost entirely a question of how many bytes must be moved from memory, which is why memory, not arithmetic, is the central resource on the edge.

1.2 How an LLM generates text: prefill and decode

A decoder-only transformer produces text one token at a time. Generation has two phases with very different hardware behaviour.

- **Prefill.** The prompt (say 1,000 tokens) is processed in parallel. All prompt tokens are pushed through every layer at once. The work is dominated by matrix–matrix multiplications and is compute-bound; hardware with strong vector or matrix units handles it well.
- **Decode.** Each new token is produced by one pass through the network. For that single token the processor must read essentially every weight of the model, and in every attention layer it must also read the stored keys and values of every previous token. The work is matrix–vector multiplication, which performs very little arithmetic per byte moved, and is therefore memory-bandwidth-bound.

Because users experience decode as the streaming speed of the answer (tokens per second), and because the decode loop repeats for every token, the number of bytes read per token is the primary determinant of perceived speed on an edge device.

1.3 What the KV cache is

Inside every attention layer, each token produces three vectors from its hidden state: a **query** (q), a **key** (k) and a **value** (v). Attention for the newest token computes a similarity score between its query and the key of every earlier token, converts the scores into weights with a softmax, and returns the weighted sum of the earlier tokens' values:

$$o_t = \Sigma_\{i \leq t\}\ softmax_i(\ q_t \cdot k_i \,/\, \sqrt{d}\ ) \cdot v_i$$

The keys and values of earlier tokens never change once computed (the model is causal). Recomputing them for every new token would multiply the work enormously, so inference engines store them. This store is the **KV cache**. It is the model's working memory of the conversation: every layer, every KV head, every token contributes one key vector and one value vector that must be kept for as long as the token can still be attended to.

The size of that memory follows a simple formula:

$$\textit{bytes per token} = 2\ (K \text{ and } V) \times \textit{layers} \times \textit{KV-heads} \times \textit{head-dim} \times \textit{bytes per element}$$

and the cache for one sequence is that number multiplied by the current context length. Unlike the weights, which are a fixed size, the KV cache **grows with every generated**

token and, in a multi-turn session or a document-question task, may become the largest single object in memory.

Table 1.2: KV-cache size for four illustrative model geometries (FP16, one sequence). Geometries are representative of model families, not of any specific checkpoint.

Illustrative geometry	layers / KV heads / head-dim	bytes per token (FP16)	4K context	32K context	128K context
TinyLlama-like	22 / 4 / 64	22,528	88 MiB	0.69 GiB	2.8 GiB
Llama-3-8B-like	32 / 8 / 128	131,072	512 MiB	4.00 GiB	16.0 GiB
7B MHA-style	32 / 32 / 128	524,288	2,048 MiB	16.00 GiB	64.0 GiB
Llama-3-70B-like	80 / 8 / 128	327,680	1,280 MiB	10.00 GiB	40.0 GiB

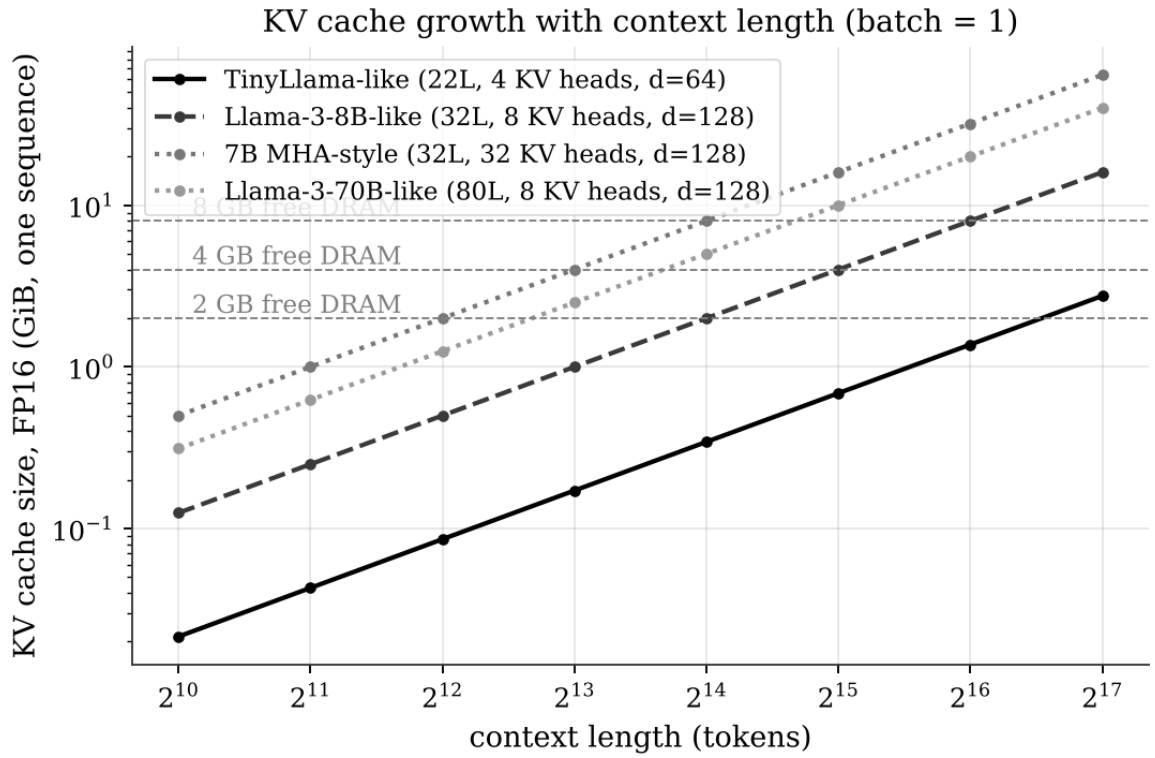


Figure 1.1: KV-cache size grows linearly with context. Dashed lines show plausible free-DRAM budgets on edge devices; every geometry except the smallest crosses one of them within the context lengths that applications now request.

1.4 Why the KV cache is the bottleneck on the edge

1.4.1 Capacity

On a phone with 8 GB of RAM, the operating system and foreground applications may leave 3–5 GB available to an on-device model. A 7–8-billion-parameter model quantised to 4 bits needs roughly 4–5 GB for its weights alone. The weights are fixed, so what remains for the KV cache shrinks to a few hundred megabytes. In the Llama-3-8B-like geometry of Table 2 that budget is exhausted at about 2–3 thousand tokens of FP16 cache, far short of the 32K–128K contexts that long-document and agentic applications need.

When memory is exhausted the platform kills the process, or the engine must truncate the context, or the application must refuse the request.

1.4.2 Bandwidth

Capacity is only half of the problem. Standard attention reads **the entire KV cache for every generated token**. Even if the cache fits, its size determines how many bytes are moved per token, and therefore the maximum speed. The next figure shows the upper bound on decode speed imposed by KV reads alone for a 12.8 GB/s memory system, ignoring weight reads and compute. For a 7B-class model with full multi-head attention the bound drops below ten tokens per second at roughly 3K tokens of context, before a single weight has been read.

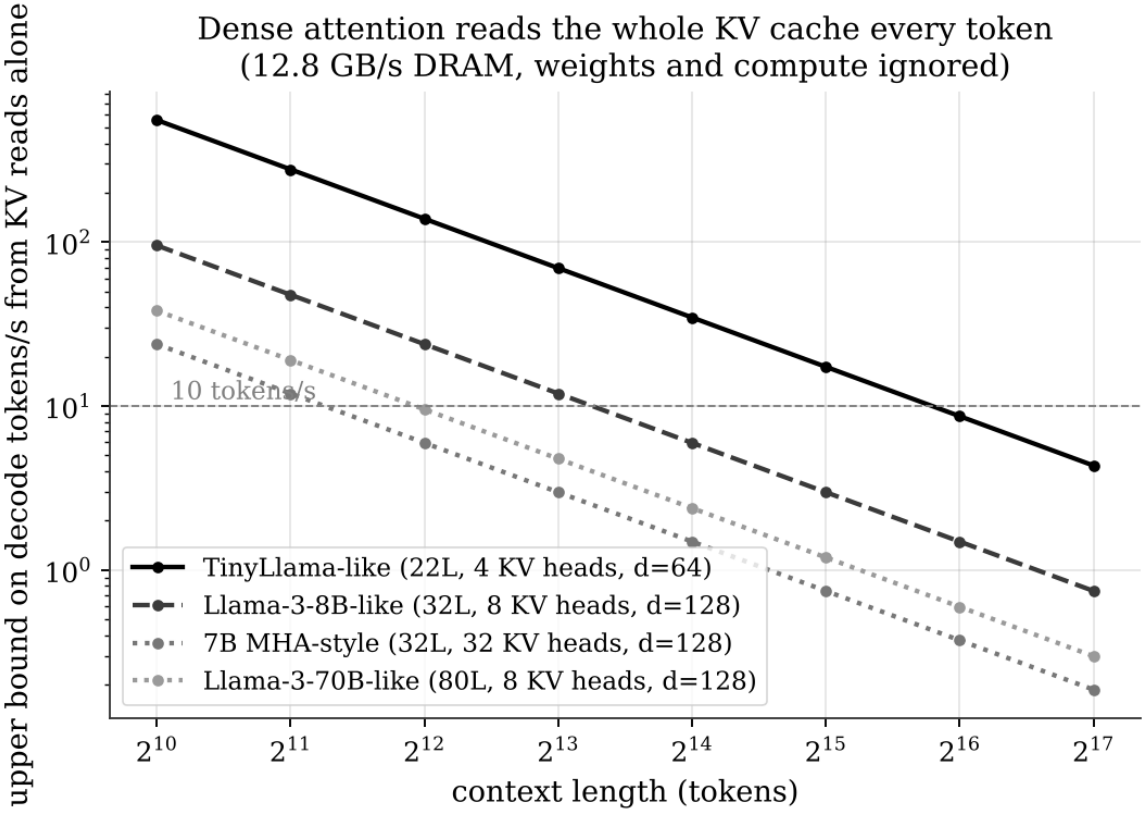


Figure 1.2: Upper bound on decode speed from KV-cache reads alone (12.8 GB/s DRAM). The curves fall as 1/context: this is the bandwidth wall that appears even when the cache fits in memory.

1.4.3 Energy

Each byte read from DRAM costs on the order of tens to a hundred times more energy than an arithmetic operation on that byte, and flash costs more again. A design that reads the whole cache for every token spends the battery on data movement. Reducing the bytes touched per token is therefore both a speed and a battery-life objective.

1.4.4 Heterogeneous memory

Edge SoCs expose a *hierarchy*: a few tens of kilobytes of tightly coupled SRAM or vector register file next to the compute unit, gigabytes of DRAM behind a shared bus, and gigabytes of flash behind a controller. The hierarchy is an opportunity (cold data can be parked cheaply) and a hazard (a wrong placement decision puts hot data behind a slow link). Data-centre inference engines were not designed around this hierarchy, and most KV-cache research assumes a GPU with high-bandwidth memory and a host CPU as the only second tier.

1.4.5 Why "just buy more memory" is not an answer

For consumer and embedded products the memory is part of the bill of materials, shares a thermal envelope with the processor and is soldered. The memory in a shipped device cannot be upgraded after the fact, whereas the context lengths that applications demand grow every year. Any solution must therefore make the same hardware serve longer contexts and do so within a fixed energy budget.

1.5 Why the obvious fixes fall short

Table 1.3: Single-technique fixes and why each leaves part of the problem open.

Obvious fix	What it gets you	Why it is not sufficient alone
Truncate the context to a fixed window	Bounded memory and bandwidth	Discards information the model may need; quality collapses on tasks that refer back to early content
Store the cache in FP16 but page it to flash	Capacity	A flash read per token is far too slow; without knowing which blocks matter the system stalls
Quantise the entire cache uniformly to 4 bits	$\approx 3\times$ capacity and bandwidth	Applies the same precision to important and unimportant data; still reads everything each token; 2-bit quantisation hurts quality
Evict low-importance tokens permanently	Capacity and bandwidth	Irreversible: a token that becomes relevant later is gone; no precision control for what remains
Use a bigger or smaller model	Moves the problem	Smaller models lose quality; the KV cache of any model still grows with context

Each fix addresses one dimension (capacity, bandwidth, precision, placement) and ignores the others. The central observation behind this work is that these dimensions interact: the best precision for a block depends on where it is stored, how likely it is to be read, and how much room is left. A system that decides them separately will make poor joint decisions.

1.6 Problem statement and design requirements

We therefore pose the problem as follows.

Problem statement

Given an autoregressive LLM decoding one sequence on an edge processor with a small fast memory, a larger DRAM and a slow flash tier, keep the KV cache within a fixed byte budget and reduce the bytes read per token, while keeping the attention output as

close as possible to what an unconstrained FP16 dense implementation would produce, without irreversibly discarding information unless explicitly allowed.

One word needs explaining before the list. We use **codec** for the way a block of keys and values is stored in memory: as plain FP16, or squeezed to 8, 4 or 2 bits, optionally with a lossless zlib layer on top. It is our own word for this storage format, not a standard term. From the problem statement we derived the requirements that shaped the design:

1. **Unit of management is a block, not a token.** Per-token bookkeeping is too costly and defeats memory locality; paging into fixed-size blocks makes allocation, migration and hardware tables simple.
2. **Placement and precision are decided together.** A block is placed in a (tier, codec) pair; the two choices are never made independently.
3. **Importance must be measured, not assumed.** Block priority must come from what attention actually does, not only from age.
4. **Reads must be selective.** The system should be able to skip fetching and decoding blocks that provably cannot matter for the current query.
5. **Loss must be controlled and visible.** Precision may only decrease along an explicit ratchet; permanent drop is opt-in; capacity overflow is reported rather than hidden.
6. **Decisions must be cheap enough for hardware.** The inner scoring loop must be implementable without a divider and verified bit-exactly against a software model.
7. **The design must connect to real runtimes and real vector hardware.** The codecs must be swappable, the compute kernel must target a RISC-V vector unit, and a runtime adapter must isolate the engine from a specific inference framework.

The rest of the report shows how each requirement was met, how it was tested, and where it is still unmet.

2. Literature Review

Before we designed anything we read through existing research done about the KV cache. This chapter goes through that reading. We first list the parts of the computer-architecture textbook by Sarangi [27] that we used, and then take the research papers one family at a time. For every paper we write what it does, what it solves, what it leaves open for a small edge device, and what we learned from it and used. The reference numbers in square brackets point to the list at the end of the report.

Words used in this report

UAMM-1 and **UAMM-2** are the two systems we built. They are described in the next chapter, so the boxes below only say how each paper shaped them.
Codec is our own word, not a standard one. It means the storage format of a block of keys and values: FP16 (no compression), INT8, 4-bit or 2-bit, optionally with a lossless zlib layer on top. UAMM-2 chooses one for every block. When we describe a paper below we usually say "quantisation scheme" for the same idea.
Tier is a level of memory: the small fast memory next to the processor, DRAM, and flash.

2.1 What we took from the architecture textbook

Smruti R. Sarangi's *Basic Computer Architecture* [27] is the book for our course. We went back to it at several points, mostly to check that what we were designing made sense as hardware. The table lists the chapters we used and what for. It is a map of where each idea came from, not a summary of the book.

Table 2.1: Chapters of Sarangi's Basic Computer Architecture [27] and how we used them.

Chapter of the book	What we used it for in this work
The Language of Bits	Number formats: why FP16, INT8 and 4-bit values behave differently, and how a fixed-point number such as our Q4.12 reciprocal is stored.
RISC-V Assembly Language	Background for the RISC-V vector extension that our kernel targets, and for reading the vector instructions the compiler produced.
Logic Gates, Registers, and Memories	Registers, SRAM and flip-flops, which we needed to build the block table and to understand why it appears as flip-flops in synthesis.
Computer Arithmetic	Multiplication and division in hardware. This is why the scorer uses weights that sum to 256 and shifts instead of dividing by 100, and why the quantiser multiplies by a stored reciprocal.
Processor Design	How a small controller and datapath are organised, which guided how we wrote the sweep scorer as a simple state machine.
Principles of Pipelining	Splitting the codec into stages so that it can handle 16 values per beat.
The Memory System	The memory hierarchy, locality, caches and page tables. A block table that maps a logical block to a tier and a slot is the same idea as a page table, and our tiers follow the same cost-versus-size logic.
I/O and Storage Devices	Flash storage: large units, slow access and limited writes, which shaped the rule that only cold, compressed blocks go to the flash tier.

2.2 A taxonomy of approaches

KV-cache research can be organised by *which lever it pulls*. The table below lists the levers; the rest of the chapter takes them in turn.

Table 2.2: Levers used by the existing work.

Lever	Idea	Representative work	Primary saving
Change the model	Fewer KV heads, or a compressed latent KV	MQA, GQA, multi-head latent attention (DeepSeek-V2)	Bytes per token (requires training)
Manage memory better	Page the cache into blocks to remove fragmentation	PagedAttention / vLLM	Wasted capacity
Evict tokens	Keep only tokens judged important	H2O, Scissorhands, StreamingLLM, SnapKV, FastGen, PyramidKV	Capacity and bandwidth (lossy, irreversible)
Select at read time	Keep everything but fetch only relevant blocks	Quest, InfiniGen	Bandwidth
Quantise	Store K and V in fewer bits	KIVI, KVQuant	Capacity and bandwidth
Low-rank / projection	Store low-rank projections of K/V	Palu	Capacity and bandwidth
Offload	Spill to CPU memory or flash and prefetch	FlexGen, InfiniGen, LLM in a flash	Capacity
Joint systems	Combine compression, eviction and tiering	EvicPress, KV-Compress, TensorRT-LLM KV-compression framework	Several at once
Hardware engines	Dedicated logic for compress/decompress or cache control	hardware KV-compression engines, CXL-SpecKV, Ara vector unit	Energy and throughput
Deployment runtimes	Compile and run models portably on many devices	MLC LLM (TVM-based)	Portability

2.3 Changing the attention architecture

2.3.1 Multi-query attention (MQA) [15] and grouped-query attention (GQA) [16]

Shazeer's multi-query attention (2019) shares one key head and one value head across all query heads; Ainslie et al.'s grouped-query attention (2023) interpolates, sharing each KV head across a group of query heads. Both reduce the KV cache by the ratio of query heads to KV heads without changing the sequence length dimension. They are now standard in many open models (the 8-KV-head geometry used in Chapter 1 is an example).

Limit. They must be built into the model during training or fine-tuning. For a model already on the device they offer nothing, and the cache still grows linearly with context.

What we learned and how we used it

Architecture-level savings are complementary to ours and multiply with them. Our engine treats the per-token K and V as vectors of whatever size the model produces, so a GQA or latent-attention model simply gives it smaller vectors. We did not attempt to change the model.

2.3.2 Multi-head latent attention (DeepSeek-V2) [17]

DeepSeek-V2 compresses keys and values into a shared low-dimensional latent vector per token and re-expands them inside the attention computation, cutting the cache by an order of magnitude for that model family. As with GQA, the saving requires the model to be trained with this structure.

2.4 Memory management: PagedAttention and vLLM

2.4.1 PagedAttention / vLLM [1]

Kwon et al., SOSP 2023. Before vLLM, serving systems reserved one contiguous slab of memory per request sized for the *maximum* sequence length, because the final length is unknown. This wasted most of the allocated memory through internal and external fragmentation.

How it works. PagedAttention borrows the idea of virtual-memory paging. The KV cache of each sequence is split into fixed-size **blocks**; a per-sequence block table maps logical blocks to physical blocks scattered anywhere in memory. Blocks are allocated on demand, so waste is limited to the unfilled tail of the last block. The attention kernel gathers keys and values through the block table. Blocks can be shared between sequences (copy-on-write) for common prefixes.

What it solves. Near-zero fragmentation (the authors report under 4 % waste), simple allocation and the ability to share prefixes. It made memory management of the KV cache a solved engineering problem for GPU serving.

What it leaves open for an edge device. PagedAttention solves *how memory is laid out*, not *how much memory is needed* or *which blocks matter*. It has no notion of importance, no compression, no tiers below the GPU/CPU pair, and it assumes every block of a sequence is read for every token.

What we learned and how we used it
Blocks are the right unit. We adopted fixed-size blocks and a block table, and made the block the unit that carries metadata (importance, selection frequency, codec, tier, key min/max summary). A block table is also a natural hardware structure; it became one of our three RTL modules.

2.5 Token eviction

A family of methods observes that attention is highly concentrated, so most cached tokens contribute almost nothing to any given step, and drops them. Section 6.4 measures the same concentration on our synthetic workload.

2.5.1 H2O: Heavy-Hitter Oracle [2]

Zhang et al., NeurIPS 2023. H2O notes that attention scores follow a power law: a small set of tokens ("heavy hitters") receives most of the accumulated attention across

steps.

How it works. The method keeps a fixed budget of tokens: a window of recent tokens plus the tokens with the highest accumulated attention score. When the budget is full, the lowest-scoring non-recent token is evicted. Scores are the running sum of attention weights that each token has received. Reported experiments retain quality with about 20 % of the cache on the models tested.

What it solves. A principled, cheap importance signal that is computed from attention weights the model already produces, and a simple eviction rule.

What it leaves open for an edge device. H2O decides **only whether a token lives or dies**. There is no quantisation, so what is kept stays at full precision; there is no lower tier, so evicted tokens are gone forever; and a token that is unimportant now but important to a later question cannot be recovered. It also does not reduce bytes per kept token.

What we learned and how we used it
We kept H2O's *signal* (decayed attention mass per token, aggregated to blocks) but changed what it controls. In UAMM-2 the signal drives **placement and precision**, which are reversible or bounded, while permanent drop is an explicit opt-in. This directly addresses the gap noted above: H2O does eviction but no quantisation.

2.5.2 Scissorhands [3]

Liu et al., NeurIPS 2023. Scissorhands builds on the "persistence of importance" hypothesis: tokens that mattered in the past tend to matter in the future.

How it works. It keeps a budget of tokens, using the history of attention scores to decide which to discard, with compression performed when the budget is exceeded.

What it solves. It reaches large compression with a light-weight rule and no training.

What it leaves open for an edge device. Like H2O it is eviction-only. The persistence assumption is also challenged by workloads whose focus shifts across topics.

What we learned and how we used it
Persistence is a reasonable prior, but it can fail when the query returns to an old topic. Our synthetic workload therefore includes topic phases that revisit older topics, to test whether cold blocks can be promoted back.

2.5.3 StreamingLLM and attention sinks [4]

Xiao et al., ICLR 2024. StreamingLLM observes that the first few tokens of a sequence receive disproportionate attention regardless of content ("attention sinks"), and that removing them breaks the model.

How it works. The method keeps the first few tokens plus a sliding window of recent tokens and evicts the rest, enabling streaming over very long inputs with a bounded cache.

What it solves. A trivial, fast policy that is stable for unbounded streams.

What it leaves open for an edge device. The evicted middle is lost; the method does not improve the model's ability to use information from the evicted span.

What we learned and how we used it

We protect **sink tokens** (the first four) and a **recent window** (32 tokens) unconditionally, hold them in the fastest tier at full precision, and never move or compress them. The protection is also implemented in the RTL scorer (its `protected` input).

2.5.4 SnapKV [5]

Li et al., 2024. SnapKV targets long prompts. It uses an observation window at the end of the prompt to vote on which earlier positions are relevant and compresses the prompt cache accordingly.

How it works. Attention weights from the window to earlier positions are pooled, the highest-voted positions are retained per head, and the rest are dropped during prefill.

What it solves. Substantial prompt-cache reduction with little quality loss on long-context benchmarks.

What it leaves open for an edge device. It is a one-shot decision at the end of prefill and does not adapt as generation proceeds; dropped positions cannot be recovered.

What we learned and how we used it

Importance can be estimated from a small window rather than from all history. We retained the continuously updated H2O-style score (cheap, vectorised) rather than a one-shot vote, but noted that a prefill-time seed for the score is a natural extension (Chapter 7).

2.5.5 FastGen and PyramidKV [6], [7]

Ge et al., 2023; Cai et al., 2024. FastGen profiles each attention head and applies a different eviction policy per head; PyramidKV allocates different budgets to different layers, giving more to lower layers where attention is broad.

How it works. Both use structure in how attention behaves across heads or layers to allocate a fixed budget unevenly.

What it solves. Better quality for the same total budget than uniform schemes.

What it leaves open for an edge device. They again decide only eviction and presume per-layer or per-head access that a flattened single-vector cache does not model.

What we learned and how we used it

Budgets should be non-uniform across layers and heads. Our reference engine flattens K and V per token into a single vector per layer-head group; per-layer and per-head budgets are listed in Chapter 7 as unfinished work, and the block metadata structure was chosen so that a (layer, head) index can be added.

2.6 Query-aware selection and retrieval

2.6.1 Quest [8]

Tang et al., ICML 2024. Quest observes that which tokens matter depends on the **current query**, so a fixed eviction rule is wasteful.

How it works. For each page of the KV cache it stores the element-wise minimum and maximum of the keys. At decode time it computes, from the query and these two vectors, an upper bound on the attention score any key in the page can have, selects the pages with the highest bounds, and runs attention only on them. Nothing is evicted; everything remains stored.

What it solves. Large reductions in memory reads (and therefore decode latency) with little accuracy loss, without losing information.

What it leaves open for an edge device. Quest reduces **bandwidth** but not **capacity**; the whole cache must still be resident, which is exactly what an edge device cannot afford.

What we learned and how we used it

This is the technique that makes tiering practical. If the system can bound which blocks are needed before fetching them, cold blocks can live in slow memory at low precision, because most steps never touch them. We adopted min/max key summaries per block and the bound Σ`_c max(q_c·kmin_c, q_c·kmax_c)`, and our tests verify that it is a true upper bound. Quest's capacity gap is closed by our tiers and codecs.

2.6.2 InfiniGen [9]

Lee et al., OSDI 2024. InfiniGen targets the case where the KV cache does not fit in GPU memory and lives in CPU memory.

How it works. It uses a lightweight speculation, computed from a skewed view of the weights, to predict at layer i which tokens layer *i+1* will need, and prefetches only those from CPU memory.

What it solves. Large speedups over naive offloading by reading only the needed entries across the slow link.

What it leaves open for an edge device. It handles one slow tier, assumes a GPU/CPU split and does not change precision.

What we learned and how we used it

A slow tier is usable if the fetch set is predicted ahead of time. Our reference engine uses bound-ranking at read time (not prefetch) and lists **prefetching** as a missing piece (Chapter 7). The tier model (SRAM, DRAM hot, DRAM warm, flash) generalises InfiniGen's two-level view.

2.7 Quantisation of the KV cache

Weights are routinely quantised to 4 bits; the KV cache is harder because its statistics differ between keys and values and, in particular, a few key channels carry very large

magnitudes.

2.7.1 KIVI [10]

Liu et al., ICML 2024. KIVI studies the element distributions of the KV cache in popular models and reports that keys have a few channels with large magnitudes (outlier channels) while values do not show this pattern.

How it works. It quantises **keys per channel** (one scale and zero-point per channel across a group of tokens) and **values per token** (one scale and zero-point per token across channels), using asymmetric 2-bit integers, and keeps a small recent window in full precision. It needs no fine-tuning.

What it solves. About 2.6× lower peak memory (including weights) at 2 bits with small quality loss on the models tested, enabling larger batches or longer contexts.

What it leaves open for an edge device. KIVI is applied uniformly. Every block gets the same precision whether it is attended a thousand times or never. It does not decide placement, and it still reads the entire quantised cache each step.

What we learned and how we used it

The axis of quantisation matters. We implemented per-channel keys and per-token values and measured the effect directly (Section 6.4): per-channel key quantisation reduced error by 1.9× at 2 and 4 bits on outlier-free data and 5.4× at 2 bits and 5.2× at 4 bits when outlier channels are present, which supports the paper's central design choice. We then used the codec **selectively** per block, which uniform quantisation cannot do.

2.7.2 KVQuant [11]

Hooper et al., NeurIPS 2024. KVQuant pushes KV quantisation toward sub-4-bit precision for very long contexts.

How it works. It combines per-channel key quantisation, quantising keys before the rotary positional embedding, non-uniform datatypes fit per layer, dense-and-sparse handling of outliers, and normalisation of the quantisation centroids.

What it solves. Context lengths of millions of tokens on large models with modest quality loss.

What it leaves open for an edge device. The techniques require calibration data and custom kernels; they are applied uniformly and aimed at GPUs.

What we learned and how we used it

Outliers must be handled explicitly. Our codec supports separate treatment of outlier channels (the key-axis experiment in Section 6.4), and our hardware quantiser follows the simple per-group form because outlier handling is cheaper in the surrounding logic. The more elaborate KVQuant ingredients (pre-RoPE keys, non-uniform datatypes) are listed as future work.

2.8 Low-rank compression

2.8.1 Palu [12]

Chang et al., 2024. Palu observes that the projection matrices that produce K and V can be factorised so that a low-rank latent is cached instead of the full vectors.

How it works. It decomposes the projection weights into low-rank factors, stores the latent, and reconstructs keys and values on the fly with a fused kernel; it adds rank-search and quantisation-friendly transformations.

What it solves. Compression that composes with quantisation, on a pre-trained model, without retraining from scratch.

What it leaves open for an edge device. It adds reconstruction compute and requires modifying how the model is run. The latent still has to be stored for every token.

What we learned and how we used it

Low-rank storage is orthogonal to the codecs and could be added as another codec family. It was not included because it needs access to the model's projection weights, which our reference engine does not have.

2.9 Offloading and flash

FlexGen [13] (Sheng et al., ICML 2023) offloads weights and KV cache across GPU, CPU memory and disk for throughput-oriented batch inference, using a linear-programming search over placement and compression (it also compresses weights and cache to 4 bits). **LLM in a flash** [14] (Alizadeh et al., 2023) shows how to run models larger than DRAM from flash by minimising the number of transferred bytes and the number of reads, with windowing and row–column bundling.

What we learned and how we used it

Flash is usable only if accesses are rare, large and sequential. That is why our flash tier stores only entropy-coded, low-precision blocks and why the planner charges a large fixed cost per flash access. Our experiments (Section 6.4) show that the planner keeps flash reads low as long as replanning is frequent and the blocks that live in flash are genuinely cold.

2.10 Joint systems

A newer line of work recognises that compression, eviction and tiering interact and tries to decide them together. We reviewed the following; they influenced the design most.

Table 2.3: Joint and system-level work reviewed, and how UAMM-2 relates to each.

Work	What it does	What we took	What we did differently
EvicPress	Jointly chooses compression level and eviction across storage tiers using a utility function	The core idea of a joint utility, with an explicit quality term	Added hysteresis, a precision ratchet, query-aware reads and a verified hardware scorer

Work	What it does	What we took	What we did differently
KV-Compress / Zipage (paged eviction)	Eviction at block granularity inside a paged cache, with variable budgets per head	Eviction and paging should share the same granularity	Default is lossless tiering; drop is opt-in
TensorRT-LLM KV-cache compression framework	Applies a codec when pages move between tiers (cold-page compression)	Compress at tier transitions, not on the hot path	Added codec ratcheting and per-block measured error tables

None of these is, to our knowledge, a complete edge-oriented stack that includes a verified decision engine in hardware and a vector kernel for a RISC-V vector unit. That gap is where UAMM-2 sits.

2.11 Hardware: engines and vector processors

2.11.1 Hardware KV-compression engines [24]

Open RTL implementations and accelerator papers. Several designs implement the compress/decompress datapath for KV data in hardware.

How it works. A typical engine quantises keys and values in groups with separate handling of outlier values, may use different bit widths for keys and for values, and may evict a fraction of tokens. Whatever ratio such a design advertises, the stored size includes scales, zero points and any outlier list, so the achieved ratio has to be measured on the stored format.

What it solves. A concrete, synthesisable datapath for compression, useful as an architectural reference.

What it leaves open for an edge device. They contain no policy: nothing decides which blocks to compress, how far, or where to store them. Built-in eviction conflicts with a design that avoids irreversible loss by default.

What we learned and how we used it

Compression ratios must be checked against the stored size including metadata; we do not quote any engine's ratios here. Our own codec table (Section 6.4) reports measured ratios with scales and zero points counted. The codec interface (`encode`/`decode`/`Payload`) is the slot where such an engine's format can be plugged in.

2.11.2 Ara and Ara2 [20], [21]

Cavalcante et al., IEEE TVLSI 2020; Perotti et al., 2023. Ara is an open-source RISC-V Vector (RVV 1.0) coprocessor designed to be attached to the CVA6 application core; Ara2 extends it for multi-lane scaling.

How it works. Ara executes vector instructions on 2–16 parallel lanes, each with its own slice of the vector register file, an FPU and integer units, plus a vector load/store unit that fetches data over AXI. Vector length is configurable at run time; the programming model is standard RVV.

What it solves. Open, synthesisable vector compute with a clean memory interface,

suited to a custom SoC that includes a KV manager.

What it leaves open for an edge device. Ara is a compute engine and has no concept of KV blocks, tiers or codecs. An LLM kernel for Ara must be written and benchmarked on Ara itself.

What we learned and how we used it

We placed our **fused QK kernel** on Ara: it loads packed 4-bit key bytes with vector byte loads, widens and converts them, and accumulates with fused multiply–add, without first writing an expanded floating-point key block to memory. This saves exactly the bandwidth that a dequantise-then-dot approach would waste. The kernel compiles for `rv64gcv` and lowers to the expected vector instructions; execution on Ara or a simulator is future work (Chapter 7).

2.11.3 CXL-SpecKV and other cache-engine proposals [24]

arXiv preprints and project pages. CXL-SpecKV describes an FPGA-based cache engine that places KV data on CXL-attached memory with speculative fetch.

How it works. A hardware controller predicts and prefetches KV blocks from a larger, slower memory pool.

What it solves. Shows that a dedicated controller can hide slow-memory latency for KV data.

What it leaves open for an edge device. Targets data-centre CXL fabrics rather than an edge SoC.

What we learned and how we used it

A hardware controller for the cache is a legitimate architecture. In UAMM-2 the per-block scoring and the block table are implemented as RTL so that the planner's inner loop does not occupy the CPU.

2.12 Deployment runtime: compiler-based runtimes

2.12.1 MLC LLM and TVM-based compiler runtimes [22], [23]

MLC team (MLC LLM, a deployment engine built on TVM); TVM: Chen et al., OSDI 2018. MLC LLM is a machine-learning compilation framework for deploying LLMs on many targets (phones, browsers, GPUs). It builds on the TVM compiler stack and its newer Unity design with dynamic-shape support.

How it works. Models are compiled into target-specific kernels; quantised weights are supported; the runtime exposes an OpenAI-style chat API and manages a KV cache internally.

What it solves. Real portability: the same model can be compiled for several hardware back ends.

What it leaves open for an edge device. Its KV cache is internal to the runtime. We have not verified the details of its KV-cache interface, nor that TVM can target RISC-V vector back ends such as Ara end to end.

What we learned and how we used it

A runtime is an adapter target, not a design dependency. We therefore defined a narrow `RuntimeAdapter` interface (append token, get attended blocks, report) with a working reference adapter, and an `CompilerRuntimeAdapter` skeleton with the integration checklist. This keeps the research engine testable on its own while leaving a clear seam for deployment.

2.13 Coverage matrix

The matrix summarises which capabilities each family of related work covers, as judged from the descriptions above; it is our reading of the papers, not a measurement.

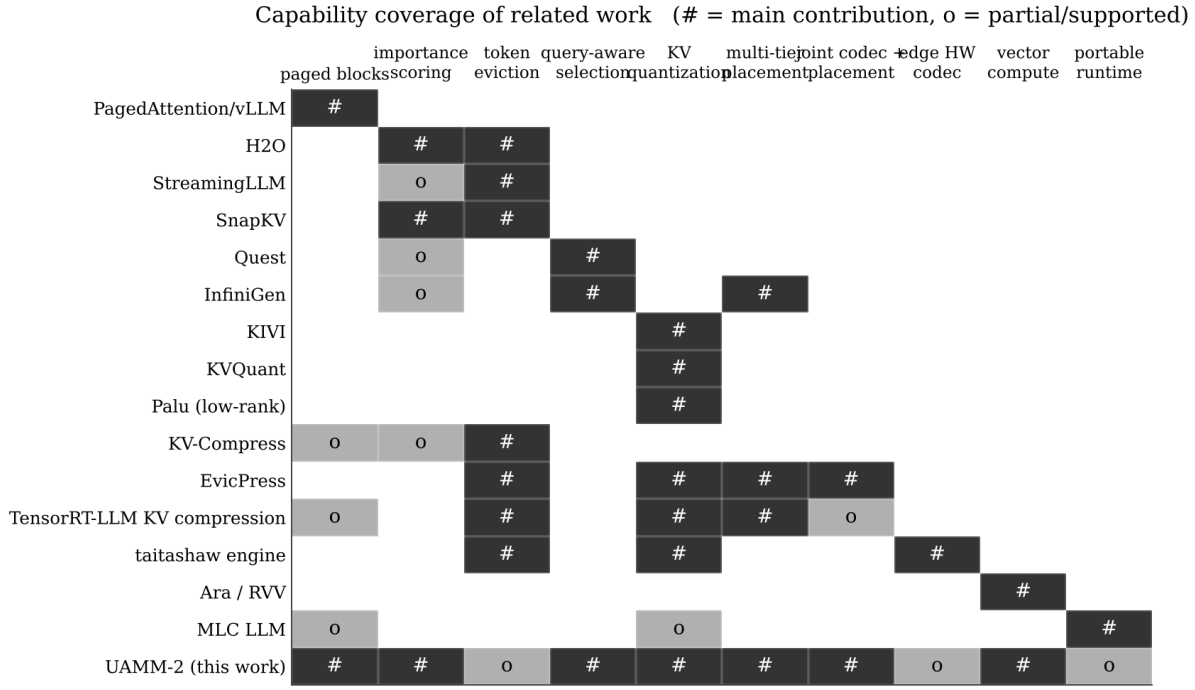


Figure 2.1: Capability coverage of the related work. No single prior system covers placement, precision, query-aware selection, importance scoring, a hardware decision engine and a vector kernel together.

2.14 Summary of the gaps

1. **Eviction without precision control** (H2O, Scissorhands, StreamingLLM, SnapKV): loss is irreversible and kept tokens stay at full size.
2. **Quantisation without selection or placement** (KIVI, KVQuant): uniform precision; the whole cache is still read every token.
3. **Selection without capacity savings** (Quest): reads fewer bytes but needs everything resident.
4. **Offloading without compression** (FlexGen, InfiniGen): one slow tier; no joint precision decision.

5. **Joint ideas without an edge hardware realisation** (EvicPress, TensorRT-LLM framework): no verified decision hardware, no RISC-V vector path.
6. **Hardware engines without a policy layer** (compress engines, Ara): datapaths exist, but nothing decides what to compress and where to put it.

UAMM-2 is an attempt to fill the space between these gaps in one consistent design. The next chapters describe the starting point and the result.

2.15 What we learned from each paper and how we used it

This chapter collects, source by source, what we concluded and how it changed the design. Each paper has its own subsection above. Here the lessons are put in one table so that the source of each design decision can be traced. The numbers in square brackets are the entries in the reference list.

Table 2.4: Sources, inferences and effects on the design.

Source	What we learned	How we used it	Not used, and why
PagedAttention / vLLM (SOSP 2023) [1]	Blocks are the right unit; block tables are cheap	Fixed 16-token blocks, block table RTL	Prefix sharing / copy-on-write (open item)
H2O (NeurIPS 2023) [2]	Attention is power-law; accumulated attention is a cheap importance signal	Decayed token score → block importance (normalised by p95)	Irreversible eviction as default
Scissorhands (NeurIPS 2023) [3]	Importance persists, but can shift	Workload with topic revisits; ability to promote cold blocks	Eviction-only budgeting
StreamingLLM (ICLR 2024) [4]	Sink tokens + recent window must be kept	Protected blocks (RTL and Python)	Dropping the middle
SnapKV (2024) [5]	A small observation window can seed relevance	Noted as a prefill-time seed (future)	One-shot prompt compression
FastGen / PyramidKV [6], [7]	Budgets should differ by head and layer	Structure for future (layer, head) index	Per-head policies (open item)
Quest (ICML 2024) [8]	Min/max page summaries give a query-aware bound	Selection by Σ `max(q·kmin, q·kmax)`; tested as a true bound	Page-level only; no token-level refinement
InfiniGen (OSDI 2024) [9]	A slow tier works if the fetch set is predicted	Multi-tier model; planner prices tier latency; prefetch listed as next step	Layer-ahead speculation
KIVI (ICML 2024) [10]	Quantise keys per channel and values per token	Codec layer; measured 1.9–5.4× lower key error	Residual full-precision window (we use protected T0 blocks instead)
KVQuant (NeurIPS 2024) [11]	Outliers must be handled explicitly; sub-4-bit needs care	Outlier-aware test data; hardware quantiser kept simple	Non-uniform datatypes, pre-RoPE keys (open item)
Palu (2024) [12]	Low-rank storage composes with quantisation	Left as a future codec family	Needs model weights
FlexGen (ICML 2023) [13]	Placement across tiers can be searched; offload needs compression	Tier + codec search, greedy rather than LP	LP-based optimiser (open item)
LLM in a flash [14]	Flash needs large, rare, sequential reads	Flash holds only cold, entropy-capable blocks; large fixed cost per access	Row–column bundling
MQA / GQA / MLA [15]–[17]	Architecture changes shrink K/V at training time	Engine accepts any vector size	No model changes
EvicPress [24]	Compression and eviction should be decided jointly with a utility	Utility planner (λ-weighted error + latency + congestion)	Our additions: hysteresis, ratchet, query-aware reads, RTL
KV-Compress / Zipage [24]	Evict at the paging granularity	Block as unit for every decision	Variable per-head budgets

Source	What we learned	How we used it	Not used, and why
TensorRT-LLM KV compression [24]	Compress when pages change tier	Codec change is part of the same plan step as the move	Vendor-specific kernels
CXL-SpecKV [24]	A hardware cache controller can hide slow-memory latency	RTL block table + sweep scorer as controller pieces	CXL fabric
Ara / Ara2 [20], [21]	Open RVV vector unit with AXI memory path	Fused QK kernel; codec RTL placed on the load path	Anything run on Ara itself (open item)
MLC LLM / TVM-based runtimes [22], [23]	A runtime is an adapter target, not a design dependency	`RuntimeAdapter` seam; skeleton with checklist	Actual integration (open item)
SmoothQuant [19]	Targets weights and activations, not the KV cache	Removed from UAMM-1's list of KV solutions	—
Our UAMM-1 (Section 4.3)	Placement without precision cannot fit a tight budget	Ablation baseline; 10 defects fixed	Threshold policy as default

2.15.1 Lessons that cut across the papers

1. **Combine axes, not just methods.** The ablation shows that placement alone, codecs alone and eviction alone each leave the budget problem unsolved or solved by loss.
2. **Measure, do not assume.** Per-block calibrated error is more reliable than a formula; advertised compression ratios are not measured ones.
3. **Make irreversibility explicit.** The ratchet and opt-in drop make every loss visible and bounded.
4. **Churn is a first-class cost.** Hysteresis and replanning cadence changed migrations by up to 4× and flash reads by up to 5×.
5. **Hardware should accelerate the inner loop and be verified like software.** Golden models, boundary vectors and mutation testing found real defects (an ignored `protected` input, an untested clamp).
6. **Keep seams.** Codec, runtime and kernel boundaries let each piece be replaced as real measurements arrive.

3. Mind Map of the Project

The figure on the next page shows the whole project as a sequence of eight steps. Read it left to right and then along the second row. Each step leads to the next one: the problem sent us to the literature, the literature to UAMM-1, the gaps in UAMM-1 to UAMM-2, and the results to the open questions in the future scope. The section numbers under each step say where to find the details.

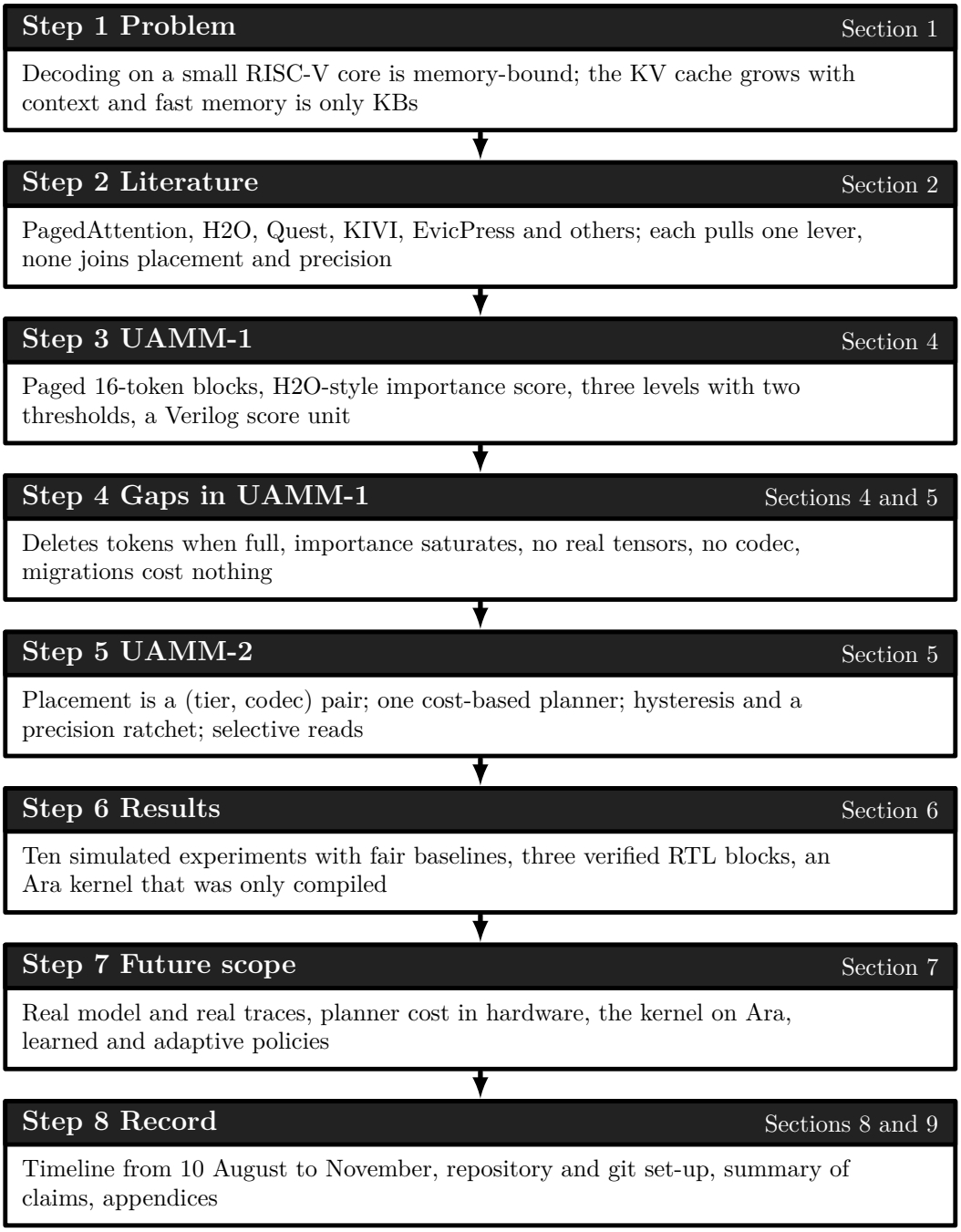


Figure 3.1: Mind map of the project, step by step.

4. UAMM-1: Our First Prototype

This section covers UAMM-1 from the beginning: how we reached the problem, what we noticed in existing work, what UAMM-1 is, what we found missing in it, and the code we wrote.

4.1 How we arrived at the problem

We did not start with the KV cache. Our first idea was about the processor itself: edge RISC-V cores have vector extensions and address-generation units, and we wanted to see whether better address generation could speed up language-model inference. To judge that, we had to find out where the time actually goes when a language model decodes one token on a small core.

The answer was not what we expected. The multiply-accumulate units are rarely the limit. The core spends most of its time computing addresses, walking tables, unpacking compressed data and waiting for data to arrive from slow memory. Each new token needs the keys and values of every earlier token, and those do not fit in the fast memory next to the processor. The arithmetic is cheap; the memory traffic is not. That moved our attention from the instruction set to the KV cache and to the question of where each part of it should live.

The first draft of this paper, submitted on 24 August 2026, was therefore a study of the problem: why generative inference on constrained RISC-V platforms is limited by memory and what a memory manager for it would have to do. This report keeps that analysis as its first chapter and adds what we built afterwards.

4.2 What we saw in the existing work

After the problem was clear, we read the papers (Chapter 2 has the details). What we noticed first was that every method pulls one lever. Paging removes wasted capacity, eviction keeps fewer tokens, selection reads fewer blocks, quantisation stores fewer bits and offloading uses slower memory. Each one helps, and each one leaves the other dimensions alone. We could not find a system that treats where a block lives and how many bits it uses as a single decision.

We also noticed that most of this work is written for GPU servers. On an edge processor the fast memory is measured in kilobytes, flash is a real tier and decoding the data competes with the model for the same few cores. So we thought a design built around those limits was worth trying.

Our plan was simple. First build something small that joins the ideas we understood best, see where it breaks, and then fix what breaks. That small thing became UAMM-1.

4.3 First Approach: UAMM-1

Having surveyed the algorithms of Chapter 2, we observed that each solves one part of the problem in isolation: paging manages layout, H2O scores token importance, quantisation shrinks tensors, and selection skips reads. The first system we built, **UAMM-1** (Unified Adaptive Memory Manager, version 1), was an attempt to join the first two into one memory manager for an edge processor. This chapter describes everything UAMM-1 contained.

4.3.1 Components

4.3.1.1 Paged block store

The KV cache was divided into fixed-size blocks of consecutive tokens, as in PagedAttention. A block table recorded, for each logical block, the memory level that held it. A block was the unit of allocation, movement and scoring; per-token bookkeeping was avoided.

4.3.1.2 H2O-style importance

Following H2O, each token accumulated the attention weight it received at every decode step. The importance of a block was a function of the accumulated weights of its tokens. The idea is that a small set of heavy-hitter tokens receives most of the attention, so blocks containing them should be kept close to the processor.

4.3.1.3 Three-level placement

Three memory levels were modelled: a fast level near the processor, DRAM and a slow non-volatile level (flash). Each block received a combined score from three signals:

$$\textit{score} = 0.60 \cdot \textit{importance} + 0.25 \cdot \textit{recency} + 0.15 \cdot \textit{access frequency}$$

and was assigned a level by two fixed thresholds: a block with score ≥ 0.68 went to the fast level, a block with $0.35 \leq$ score < 0.68 to DRAM, and a block below 0.35 to flash. When a level was full the block was pushed down to the next level (cascade demotion); when no level had room, the block was discarded.

Worked example

A block has importance 0.90, recency 0.50 and access frequency 0.40. Score $= 0.60 \times 0.90 + 0.25 \times 0.50 + 0.15 \times 0.40 = 0.54 + 0.125 + 0.06 = 0.725$. Since $0.725 \geq 0.68$ it is placed in the fast level.
A second block has importance 0.20, recency 0.30 and frequency 0.10: score $= 0.12 + 0.075 + 0.015 = 0.21 < 0.35$, so it goes to flash. If flash is also full, the block is dropped.

4.3.1.4 Hardware decision engine

A small Verilog module computed the weighted score and the level decision for one block at a time, intended to offload the arithmetic from the processor. It was checked by a testbench with three input vectors.

4.3.1.5 Software model

A Python simulator reproduced the policy, advanced the H2O scores token by token, and estimated latency from the level each block resided in.

4.3.2 What UAMM-1 achieved

UAMM-1 showed that importance-driven tiering is workable: hot blocks stay near the processor, cold ones move away, and the decision can be made cheaply in hardware. It also supplied the structure that UAMM-2 keeps: blocks, a block table, a hot/warm/cold classifier and its RTL scorer. For this reason the original threshold policy was retained in the code as a baseline (`ThresholdPlanner`), so that UAMM-1 and UAMM-2 can be compared on identical inputs.

4.4 Gaps in UAMM-1

Combining paging and importance scoring was a first step, but UAMM-1 inherited the limits of the algorithms it combined and added defects of its own. This chapter identifies each gap, first at the level of ideas and then at the level of implementation, and gives the evidence for its consequence.

4.4.1 Gaps at the level of ideas

Table 4.1: Gaps at the level of ideas.

Gap	Explanation	Consequence
No quantisation	H2O-style importance only decides where a block lives or whether it is deleted. Kept data stays at full 16-bit precision. Quantisation methods such as KIVI, which would shrink kept data, were not part of the system.	When the budget is smaller than the FP16 data, the only remedy is deletion
Placement without precision	Importance and placement were linked, but placement and precision were not. A block could not be stored smaller in a slower level.	Slow levels hold full-size blocks and are quickly exhausted
No selective reading	All retained blocks were assumed to be read at every step. There was no query-aware rule, as in Quest, for skipping blocks that cannot matter.	Bandwidth stays proportional to the retained cache
No measure of codec loss	Because no codec existed, nothing estimated how much error a smaller format would cause for a given block.	No principled trade-off between size and accuracy
Irreversible, silent loss	Deletion happened whenever the lowest level was full, without a flag or an opt-in.	Output quality degraded with no visible signal
Isolated hardware	The decision engine scored one block at a time and had no memory of block metadata or a translation table.	The software could not hand a whole table to the hardware

4.4.2 Defects in the implementation

Table 4.2: Implementation defects found on review and the response to each.

Defect	Why it matters	Response in UAMM-2
The eviction decision was computed but never acted on, and data was deleted silently when the lowest level was full	Output changes without notice	Drop is explicit and opt-in; otherwise overflow is reported as a number
Importance was clamped at 1.0 although H2O scores are sums that grow without bound	Heavy hitters and sinks saturate; the signal stops discriminating	Scores decay each step; block importance is normalised by the 95th percentile
Recency was constant or unused	A quarter of the score carried no information	Recency enters through an exponential average of how often attention fetched the block
No capacity-aware cascade and no hysteresis	Blocks oscillate between levels; each move costs energy	Greedy capacity-aware assignment, minimum residency and a relative-improvement margin
Latency charged per token, migrations free	Cost of moving data was invisible	Cost per block per level and codec; migrations accounted separately
O(N) Python loop per token for H2O	Does not scale with context	One vectorised decay per step
Division by 100 in Verilog	Dividers are large and slow	Integer weights 154/64/38 (sum 256) and a shift
Verilog and Python disagreed on protected blocks	Hardware and software make different decisions	RTL honours the protected flag; bit-exact golden model
Three test vectors, no boundary cases	Little confidence in the RTL	17 sweeps with threshold edges, 400 codec vectors, 6,000 random table operations, mutation testing
SmoothQuant listed as a KV-cache solution	It targets weights and activations	Replaced by KV-specific KIVI-style codecs

4.4.3 Evidence: what the gaps cost

The experiments of Section 6.4 measure the consequence. With a total budget of 168 KB for 1,536 tokens (FP16 would need 393 KB), the UAMM-1 policy keeps 0.37 of the FP16 footprint only by permanently deleting 973 of the 1,536 tokens on average. The remaining blocks then retain only 0.76 of the attention mass, the mean relative error of the attention output is 0.48, and about 12,613 block reads are served from flash. A system with codecs, by contrast, keeps every token at 0.31 of the FP16 footprint with 0.94 recall. The gap is therefore not in the placement rule but in the absence of a way to make data smaller and of a way to read less.

Summary of what was left unresolved

1. No quantisation, so a tight budget forces deletion.
2. Placement and precision decided separately (placement only).
3. No query-aware selection of what to read.
4. No measured error per block, hence no principled quality control.
5. Silent, irreversible loss and several implementation defects.
6. Hardware that was neither connected to a block table nor adequately verified.

4.5 Implementing UAMM-1: the code, how to run it and where the data comes from

4.5.1 What we wrote

UAMM-1 is a small Python simulator with one Verilog module. It does not run a language model. It models only the *bookkeeping* of a KV cache: which block is where, how much

attention each token received and what a read from each memory level would cost. The files are listed in the table below.

Table 4.3: The files of UAMM-1.

File	What it contains
`config.py`	Block size (16 tokens), capacities of the three levels in blocks (8, 16, 64), the weights 0.60/0.25/0.15, the thresholds 0.68 and 0.35, the recent window (32 tokens) and three assumed read times
`paged_cache.py`	`KVBlock` (importance, recency, access count, protected flag) and `PagedKVCache`, which appends tokens and opens a new block every 16 tokens
`h2o.py`	`H2OImportanceTracker`: adds the attention a token received to the token's running score and sums it up per block
`controller.py`	`UnifiedMemoryController`: computes the weighted score of a block and returns a target level and an evict flag
`memory_tiers.py`	`MemoryTiers`: counts blocks in each level, enforces capacity, counts transfers and evictions
`simulator.py`	`UAMMSimulator`: the per-token loop that ties the above together, and a `Metrics` summary
`cli.py`	Command line: `-tokens`, `-block-size`, `-gpu-blocks`, `-cpu-blocks`, `-nvme-blocks`
Verilog	`uamm_controller`: computes the same score for one block and outputs a level and an evict flag; with a testbench of three vectors

4.5.2 What it does at every token

1. Append the token to the cache. If the last block is full, open a new one and place it in the fastest level.
2. Create a simulated attention vector over the recent tokens (see below) and let the H2O tracker add it to the token scores.
3. Mark every block that holds one of the last 32 tokens as protected, and update each block's recency.
4. Charge a read time for every block that the attention touched, depending on the level the block is in.
5. For every block compute `0.60·importance + 0.25·recency + 0.15·frequency` and compare it with the two thresholds. Move the block to the level the score selects, pushing the least valuable block down when that level is full.
6. At the end report the simulated first-token time, inter-token time, throughput, peak use of the fast level, number of transfers and number of evictions.

4.5.3 How to run it

From the folder that holds the UAMM-1 package, run the command-line module with the number of tokens and, if you want to change them, the capacities of the three levels:

Running UAMM-1

```
python3 -m <package>.cli --tokens 512 --block-size 16 --gpu-blocks 8 --cpu-blocks 16 --nvme-blocks 64
```

The program prints the number of blocks in each level and the metrics dictionary. The Verilog testbench is run with any Verilog simulator (we used Verilator). The complete source is in the appendix on the original prototype, and a copy of the listing is in `uamm1_original/` in the repository.

4.5.4 Where its data points come from

UAMM-1 has no input data in the usual sense. At each token the simulator *invents* an attention vector: a random number between 0 and 1 for each of the last eight tokens, a random number scaled by 0.15 for the older tokens in the recent window, and a constant boost of 0.8 for token 0, which plays the part of an attention sink. The random generator is seeded with 7, so a run is repeatable. The read times (0.02 ms, 0.20 ms and 4.0 ms for the three levels) are numbers we assumed, not measured. The metrics are therefore outputs of a model with made-up inputs, and we only used them to compare policies with each other. We did not obtain any value from a real language model.

What this tells us about UAMM-1

UAMM-1 let us check that an importance-driven placement runs end to end and that the scoring logic can sit in hardware. It could not tell us how much accuracy a policy loses, because it never held real key and value vectors. This was the first thing we changed in UAMM-2: the cache now stores real tensors and attention is computed on them.

4.5.5 The scoring code, line by line

The decision for one block is made in `UnifiedMemoryController` (`controller.py`). The listing below is the core of it, unchanged from the prototype. `score` turns three numbers into one: the block's importance from the H2O tracker, how recently it was used, and how often it was used. Each is scaled to the range 0 to 1 before the weights `alpha`, `beta` and `gamma` (0.60, 0.25 and 0.15) are applied. `decide` then compares the score with two thresholds and picks the level. A block is also marked for eviction when the lowest level is nearly full (pressure at least 0.95), its score is below the lower threshold and it is not protected.

UAMM-1 controller: score and decide (controller.py)

```
class UnifiedMemoryController:
    def __init__(self, config: Config):
        self.cfg = config

    @staticmethod
    def _norm(value: float, maximum: float) -> float:
        return max(0.0, min(1.0, value / maximum if maximum > 0 else 0.0))

    def score(self, block: KVBlock, now: int, max_age: int, max_frequency: int) -> float:
        importance = self._norm(block.importance, 1.0)
        age = max(0, now - block.last_access)
        recency = 1.0 - self._norm(age, max(1, max_age))
        frequency = self._norm(block.access_count, max(1, max_frequency))
        return (self.cfg.alpha * importance +
                self.cfg.beta * recency +
                self.cfg.gamma * frequency)

    def decide(self, block: KVBlock, now: int, max_age: int, max_frequency: int,
               pressure: float) -> Decision:
        score = self.score(block, now, max_age, max_frequency)
        if score >= self.cfg.gpu_threshold:
            tier = "gpu"
        elif score >= self.cfg.cpu_threshold:
            tier = "cpu"
        else:
            tier = "nvme"
        evict = pressure >= 0.95 and score < self.cfg.cpu_threshold and not block.protected
        return Decision(score, tier, evict)

from collections import defaultdict
```

```
from typing import Dict, Iterable
from .paged_cache import PagedKVCache
```

Two things in this code explain the gaps we found later. The importance is clamped to 1.0 by `_norm`, so heavy-hitter blocks all reach the same value. And the only outcomes are a level or an eviction, so there is no way to keep a block in a smaller form.

4.5.6 Making room when a level is full

`_ensure_capacity` in `simulator.py` runs whenever a block has to move into a level that has no free slot. It picks the resident block with the lowest importance that is not protected and pushes it one level down. If the bottom level is also full, the victim is removed from the cache altogether, which is the deletion that cost UAMM-1 973 of 1,536 tokens in our comparison.

UAMM-1: demoting or deleting a victim block (simulator.py)

```
    def _ensure_capacity(self, target: str, incoming_id: int):
        if self.tiers.has_space(target):
            return True
        # Demote the least valuable, unprotected resident block to a slower tier.
        candidates = []
        for block_id in self.tiers.blocks[target]:
            b = self.cache.get_block(block_id)
            if not b.protected and block_id != incoming_id:
                candidates.append((b.importance + b.recency * 0.01, block_id))
        if not candidates:
            return False
        _, victim_id = min(candidates)
        fallback = "cpu" if target == "gpu" else "nvme"
        if fallback == "nvme" and not self.tiers.has_space("nvme"):
            self.tiers.remove(victim_id)
            self.cache.remove_block(victim_id)
```

5. UAMM-2: Problems We Found and How We Corrected Them

This section follows UAMM-2 the way we lived it. For each part of the design we describe what we tried, what we saw when we ran it, what we decided and what we set aside. Some decisions came from an experiment and some from reasoning about the edge processor; we say which. After the decisions we list the problems in the design and in our own results, and then describe the system as it stands and how the code is organised.

5.1 The idea behind UAMM-2

When we went through the gaps in UAMM-1, most of them could be fixed locally. Deleting data silently, an importance value that saturated and migrations that cost nothing were all bugs of the kind you patch. One gap was different: UAMM-1 could decide *where* a block lived but never *how big* it was. That needed a new structure, and it is what UAMM-2 is built around.

So in UAMM-2 every block has a placement that is a pair, (tier, codec), where the codec is the storage format of the block (we defined the word in Chapter 1). A cold block does not have to be deleted; it can move to a slower tier and be stored in 4 or 2 bits. A hot block can stay in FP16 in the fastest tier. A planner makes the choice for all blocks together by adding up three costs: the time to read and decode the block, a penalty for the error the codec introduces (weighted by how hot the block is), and a price for using scarce capacity. Three rules keep it stable: precision can only go down, blocks must stay put for a minimum time and a move must be a clear improvement, and deletion is switched off unless explicitly requested. Attention then reads only the blocks that can matter for the current query. Chapter 4 describes the design in full.

5.2 How UAMM-2 is built

This part describes the system we ended with: its memory tiers, the block and its life cycle, the codecs, the importance signal, the planner, selective attention, the hardware blocks and the vector kernel. The decisions behind each choice, and the problems we met, follow in the next two parts.

5.3 The System Design of UAMM-2

The knowledge that went into UAMM-2 comes from the papers in the literature review: blocks from PagedAttention, the importance signal from H2O, protected sink and recent tokens from StreamingLLM, the query-aware bound from Quest, the asymmetric quantisation of keys and values from KIVI, and the idea of deciding compression and eviction together from EvicPress. The rest of this part shows how these were turned into code, hardware and a kernel.

5.3.1 The core idea: a placement is a (tier, codec) pair

Every block of the KV cache is described by two coordinates. The **tier** says *where* it is stored (and therefore what a read costs). The **codec** says *how many bits* it occupies (and therefore how big it is and how much error it carries). The planner picks both together. A block that is almost never attended is best stored in flash as a 2-bit entropy-coded blob. A block that is hot but large can stay in DRAM at 4 bits. Protected blocks (sinks, recent window) sit in the fastest tier at full precision. Choosing the two jointly lets the system exchange precision for capacity instead of exchanging data for nothing.

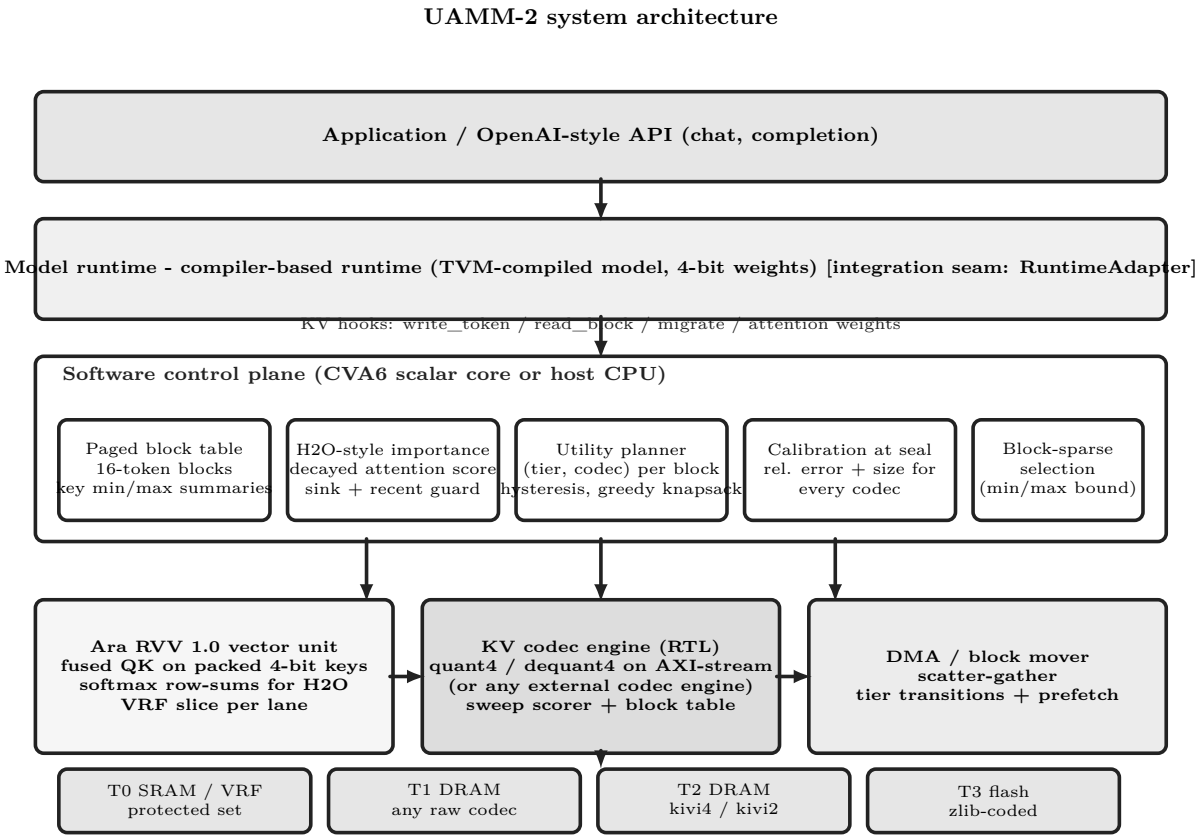


Figure 5.1: UAMM-2 system architecture: the decode loop, the paged cache with its tiers and codecs, the importance and planning loop, and the hardware and vector-kernel attachments.

5.3.2 Memory hierarchy and tiers

Table 5.1: Tiers used in the simulations. Capacities are deliberately small so that the budget is tight at a few thousand tokens; latencies and bandwidths are assumed values (Section 6.2).

Tier	Where it models	Capacity in experi-ments	Base latency / bandwidth (assumed)	Allowed codecs	Intended contents
T0	Near-compute SRAM / vector register file	24 KB	0.5 µs / 100 GB/s	any raw codec	Sink tokens, recent window, unsealed block
T1	DRAM, hot	48 KB	2 µs / 12.8 GB/s	any raw (non-entropy) codec	Frequently fetched blocks; also where 2-bit blocks sit if the planner prices them there
T2	DRAM, warm	32 KB	2 µs / 12.8 GB/s	kivi4, kivi2	Occasionally fetched blocks

Tier	Where it models	Capacity in experiments	Base latency / bandwidth (assumed)	Allowed codecs	Intended contents
T3	Flash	64 KB	100 µs / 1 GB/s	int8z, kivi4z, kivi2z (entropy-coded)	Cold blocks

The tier restrictions are policy, not physics. They encode two beliefs: that holding full-precision data in a tier labelled "warm" wastes the capacity that tier exists to provide, and that entropy coding (zlib) is only worth its decode time where the access cost is already high.

5.3.3 The block and its lifecycle

A block holds 16 consecutive tokens (configurable). While it is filling it is *open*: staged as raw vectors in the fastest tier and always read. When the 16th token arrives it is **sealed**: the K and V vectors are rounded to FP16 (this rounded copy is the ground truth from then on), encoded, and the following summaries are computed and kept resident because they are tiny:

- `kmin`, `kmax`: element-wise minimum and maximum of the block's keys (64 values each in the experiments);
- `err_table[codec]`: the measured relative squared error obtained by actually encoding and decoding this block with every codec;
- `size_table[codec]`: the measured byte size of the block under every codec;
- `importance`, `hit_ema`, `access_count`, `last_move_step`, `protected`: scheduling state.

Calibration by real encoding, rather than by a formula, is deliberate. The error of a codec depends strongly on the block's content (for example on outlier channels), and the planner should know what *this* block will lose, not an average. The cost is one extra encode per codec at seal time, which is a background cost in a real system.

5.3.4 Codecs: KIVI-style asymmetric quantisation

The codec layer turns a block of T tokens × d channels of keys and values into a byte payload and back. Four precision families exist: **fp16** (reference), **int8**, **kivi4** and **kivi2**; each non-FP16 family also has an entropy-coded variant with suffix `z` (zlib over the serialised blob), used on flash.

5.3.4.1 Asymmetric affine quantisation

For a set of values with minimum m and maximum M, and b bits (so L = 2^b − 1 levels), the scale and zero are s = (M − m)/L and z = m. A value x is stored as q = clip(round((x − z)/s), 0, L) and reconstructed as \ensuremath{\hat{x}} = q · s + z. Scales and zeros are stored as FP16 (scale floored at 2^−10 to avoid division by zero). "Asymmetric" means that the zero point is the true minimum rather than zero, which uses all levels even when the data are not centred on zero.

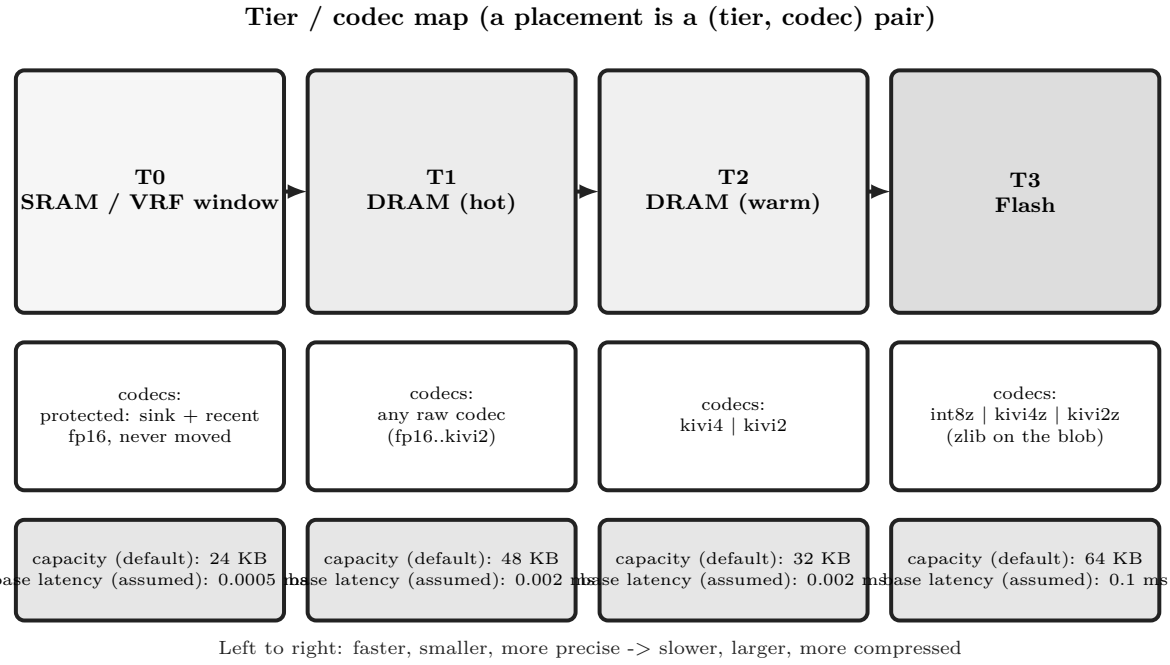


Figure 5.2: Which codecs are permitted in which tier. Flash only accepts entropy-coded variants; the warm DRAM tier only accepts 4- and 2-bit codecs.

5.3.4.2 Keys per channel, values per token

Following KIVI, **keys are quantised per channel**: for each of the d channels one scale and one zero are computed across the block's tokens. **Values are quantised per token**: each token's value vector is split into groups of 32 channels and each group has its own scale and zero. Per-channel quantisation of keys isolates the few channels with large magnitude so they do not stretch the range of the others; values have no such outlier structure and are naturally dequantised token by token as attention weights multiply them.

5.3.4.3 Packing

Quantised integers are packed little-endian into bytes: two 4-bit values per byte (element 0 in the low nibble) or four 2-bit values per byte. The same convention is used by the RTL nibble packer and by the RVV kernel, so a block produced by the Python codec can be consumed unchanged by the hardware and the kernel.

5.3.4.4 Payload layout and measured sizes

The serialised payload is `[packed K][packed V][K scales][K zeros][V scales][V zeros]`. For one block of 16 tokens × 64 channels (1,024 K elements + 1,024 V elements) the measured sizes are given in the table below: metadata is not free, which is why a 4-bit block is 0.344 of FP16 rather than 0.25.

Table 5.2: Measured size and reconstruction error per codec for a 16 × 64 block of K and V (mean over test blocks; synthetic data, see Section 6.2). The z variants reduce size slightly on flash-like data; for 4-bit and 2-bit they add a few bytes of overhead on random-like blocks.

Codec	Bytes / block (avg)	Ratio to FP16	Rel. sq. error (no outlier channels)	Rel. sq. error (with outlier channels)
fp16	4096	1.000	4.30e-08	4.24e-08
int8	2432	0.594	1.80e-05	1.62e-05
kivi4	1408	0.344	5.18e-03	4.68e-03
kivi2	896	0.219	1.30e-01	1.18e-01
int8z	2407	0.588	1.80e-05	1.63e-05
kivi4z	1419	0.346	5.17e-03	4.66e-03
kivi2z	905	0.221	1.31e-01	1.20e-01

5.3.4.5 Precision ratchet

Re-encoding a lossy block decodes it and encodes it again, so errors compound. The cache therefore enforces a **one-way ratchet**: a block can be moved to a codec of equal or lower precision, never higher. Promoting a 4-bit block to a "fast tier" changes where it lives, not how accurate it is. The only same-precision transformations allowed are toggling the entropy wrapper, which is a lossless blob operation. This rule makes the quality effect of any sequence of planner decisions bounded and easy to reason about; it also means the planner must be cautious about early demotion (see Chapter 7).

5.3.5 The importance signal: H2O reinterpreted

Each decode step produces attention weights over the tokens that were actually read. The engine updates a score per token: it multiplies all scores by a decay (0.95 by default) and then adds the new attention weight to the tokens that received it. A block's importance is the aggregate (mean by default) of its tokens' scores, divided by the 95th percentile of block scores and clipped to [0, 1]. Normalising by the 95th percentile prevents the enormous score of the attention-sink block from forcing every other block's importance toward zero. A second signal, `hit_ema`, is an exponential moving average of whether the block was selected for fetching at each step.

The **heat** of a block combines them: heat = ½ · hit_ema + ½ · importance. Importance reflects long-run attention mass; hit_ema reflects recent fetch behaviour. Using both gives the planner a stable and a responsive signal.

How this differs from H2O

In H2O the score decides which tokens are *deleted*. Here it decides where a block is *stored* and at what precision. The signal is the same; the consequence of acting on it is reversible-or-bounded instead of irreversible. Permanent drop remains available (`allow_drop`, importance below 0.02) but is off in all headline experiments.

5.3.6 The utility planner

Every `plan_interval` steps (default 8) the planner re-evaluates every sealed, unprotected block. It considers every (tier, codec) candidate that the tier rules allow and whose precision is not higher than the block's current precision, and computes an expected cost

in milliseconds-equivalent:

$$cost(b,\ t,\ c) = p_access(b)\ \cdot\ [read_ms(t,\ c,\ bytes) + decode_ms(c,\ bytes)] + \lambda\ \cdot\ heat(b)\ \cdot\ err_b(c) + \kappa\ \cdot\ (bytes\ /\ cap_t)\ \cdot\ (1 + 4\ \cdot\ fill_t)$$

- **Latency term.** p_access is the block's `hit_ema` (floored at 0.01); the read cost is the tier's fixed latency plus bytes over bandwidth plus decode time. A block that is rarely read pays little for living in flash.
- **Quality term.** λ (default 2.0) converts measured codec error, weighted by the block's heat, into the same unit as latency. A hot block with a high-error codec is penalised; a cold block is almost free to compress.
- **Congestion term.** κ (default 0.05) charges for the share of the tier that the block would occupy, scaled up as the tier fills, so that the planner does not exhaust the fast tiers early.

Blocks are visited in descending heat so that the hottest blocks get first choice of fast tiers. Each takes the cheapest candidate that fits the remaining capacity (with 3 % headroom). If nothing fits, the block stays where it is and the overflow is counted. This greedy procedure is a heuristic, not an optimal knapsack solution.

5.3.6.1 Hysteresis

Without damping, a block whose cost estimate fluctuates would bounce between tiers, paying migration cost each time. Two rules stop that: a block may not move again within `min_residency_steps` (default 16) of its last move, and it only moves if the best alternative is cheaper than staying by more than `hysteresis_margin` (default 25 %) of the best cost. Section 6.4 measures the effect.

5.3.7 Selective attention: reading less

At each decode step, protected blocks (sinks, recent window and the open block) are always read. Of the remaining sealed blocks, the engine reads only the `attn_block_budget` (default 24) blocks with the largest upper bound on the query–key score, computed from the resident kmin/kmax summaries:

$$bound(b) = \Sigma_c\ max(\ q_c\ \cdot\ kmin_c\ ,\ q_c\ \cdot\ kmax_c\ ) \geq q\ \cdot\ k\ for\ every\ key\ k\ in\ b$$

The bound holds because each product q_c · k_c, for k_c between kmin_c and kmax_c, is at most the larger of the two endpoint products. A block with a low bound cannot contain a key that scores highly against this query, so it is neither fetched nor decoded. The test suite checks the bound property on random data. Selected blocks are decoded and attention is computed over the union of protected and selected tokens.

This is where capacity savings and bandwidth savings combine. Because skipped blocks are never touched, they can be stored in the cheapest (slowest, smallest) tier and codec without most steps ever paying for that choice; and because tier and codec are chosen on measured heat, the blocks that are touched are the ones that the system placed where they are cheap to read.

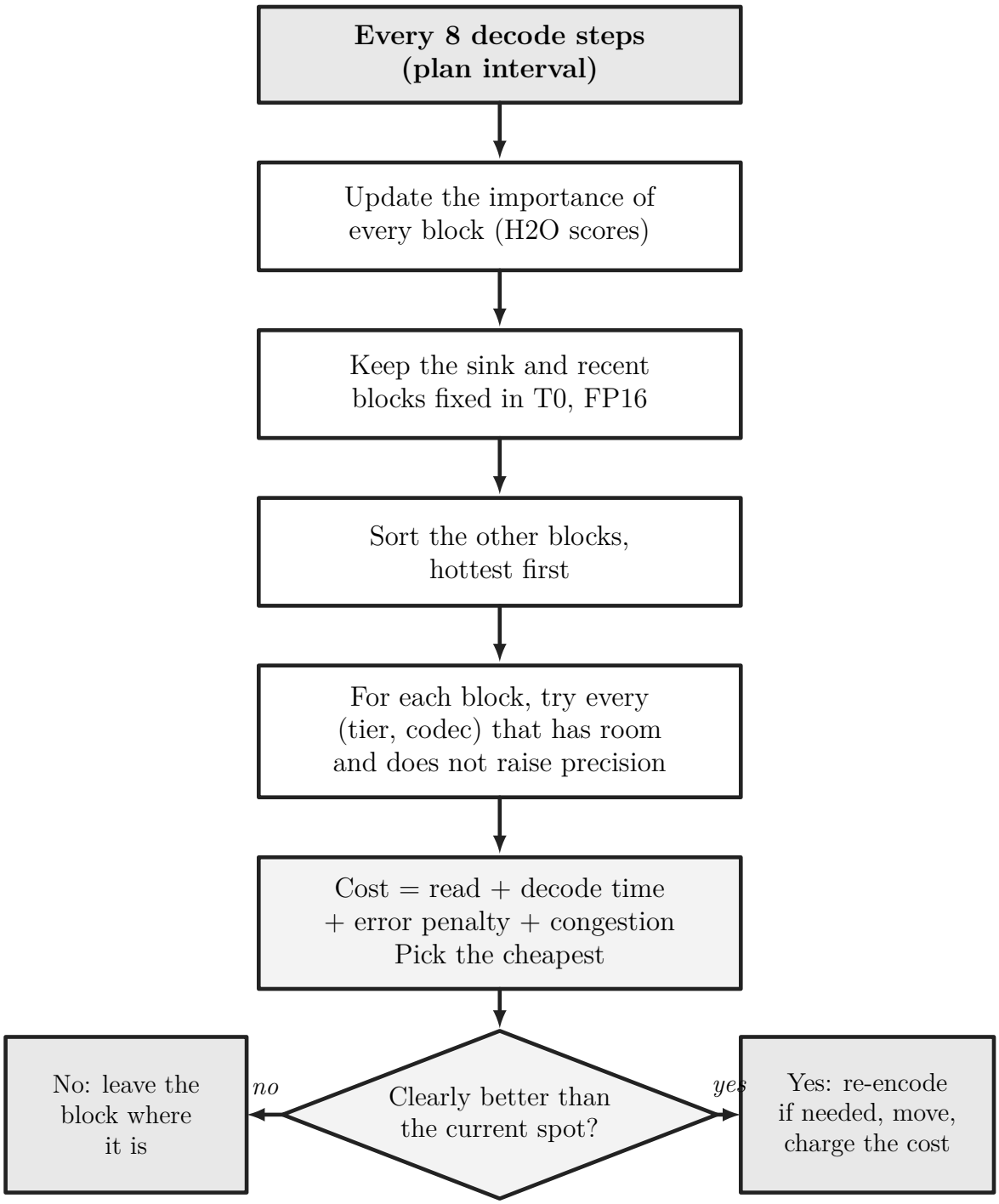


Figure 5.3: Planner flow: candidate enumeration, cost evaluation, greedy capacity-aware selection and the hysteresis check.

5.3.8 Hardware: three verified RTL blocks

Three modules were written in synthesisable Verilog-2001. Each has a bit-exact Python golden model (`uamm2/hwmodel.py`), generated test vectors and a self-checking testbench run in Verilator. They accelerate the *inner loops* of the system; the utility planner itself remains software.

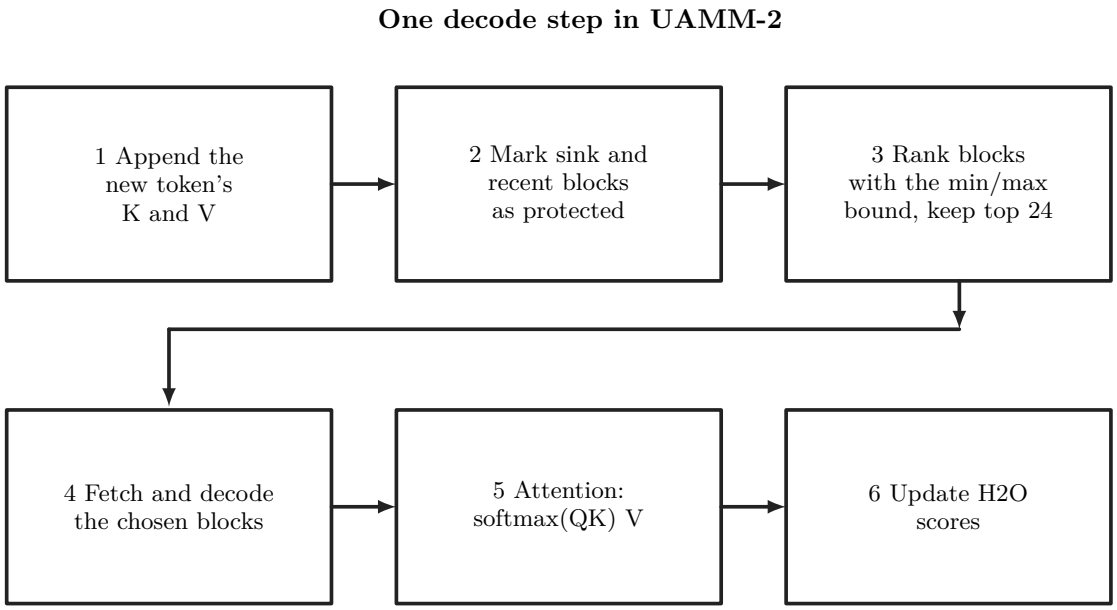


Figure 5.4: One decode step: bound ranking, fetch of selected blocks, attention, H2O update and (every few steps) a planning pass.

5.3.8.1 Sweep scorer (`uamm_sweep_scorer`)

The scorer is the hardware heat classifier derived from UAMM-1. Block metadata is held in a small SRAM as a 25-bit word `{protected, frequency[7:0], recency[7:0], importance[7:0]}`. A `start` pulse latches an 8-bit memory-pressure value and launches a **sweep**: every block is read (one per cycle), passes through a 2-stage pipeline, and emerges as a result record `{idx, score, tier, evict}`. The score is computed without a divider:

$$score = (154 \cdot imp + 64 \cdot rec + 38 \cdot frq) \gg 8 \ (154 + 64 + 38 = 256)$$

The tier is hot if the block is protected or score ≥ 174, warm if score ≥ 89, otherwise cold. A block is flagged for eviction only if it is **not** protected, the latched pressure is at least 243, and its score is below 89. A sweep over 64 blocks has latency NBLK + 3 cycles and completes with a `done` pulse; a `start` received while busy is ignored.

5.3.8.2 KV codec pipeline (`uamm_kv_quant4`, `uamm_kv_dequant4`)

The quantiser processes 16 byte-wide lanes per beat. Software or firmware supplies per-group parameters derived from the group's min and max: the zero point `zp`, a Q4.12 reciprocal `inv_q12 = round(15·4096/range)` and a Q8.8 step `step_q8 = round(range·256/15)`. Quantisation is q = clamp(((x − zp) · inv + 2048) » 12, 0, 15) with x < zp clamped to zero; dequantisation is x' = min(255, zp + ((q · step + 128) » 8)). Each module has one pipeline register. The output is packed with lane i in nibble i, matching the software packing. The intended location is on the AXI stream between memory and Ara's vector load/store unit, so that (de)quantisation happens in the load path rather than in vector instructions.

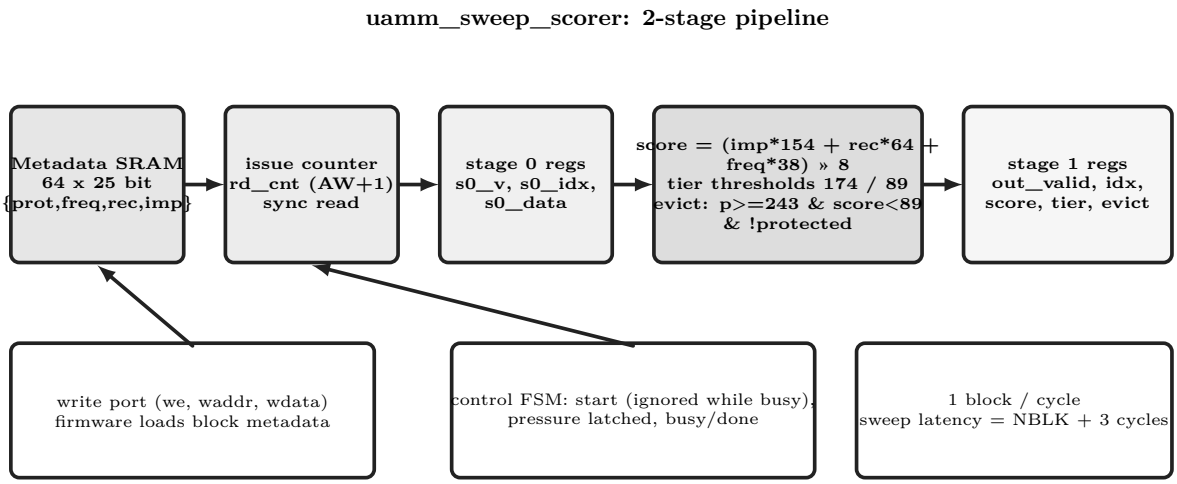


Figure 5.5: Sweep scorer micro-architecture: metadata SRAM, weighted sum, threshold comparators, protected override and eviction logic.

5.3.8.3 Block table (`uamm_block_table`)

A direct-mapped translation table maps a logical block index (8 bits, 256 entries) to `{tier[1:0], codec[1:0], phys[11:0]}` with a separate valid bit that is cleared by reset. A lookup is a one-cycle synchronous read that returns the entry and a `hit` flag. A read in the same cycle as a write to the same address returns the **old** entry, a property fixed by design and verified by test.

5.3.9 The vector kernel for the Ara vector processor

Ara is a RISC-V Vector coprocessor. Attention scores for a block of 4-bit keys can be computed without first dequantising the keys into floating point. Because the key for token t at channel c is x = q · s_c + z_c, the dot product with a query splits into:

$$q \cdot k_t = \Sigma_c \; (\; q_c \cdot s_c \;) \cdot Q[t,c] + \Sigma_c \; q_c \cdot z_c$$

The first term uses the integer codes Q and a query vector pre-scaled by the per-channel scales (computed once per block); the second term is a constant for the block (the **bias**). The kernel therefore folds scales and zeros into the query once, then, for each token, loads packed bytes with a vector byte load, extracts the low and high nibbles (even and odd channels), zero-extends and converts them to float, and accumulates with fused multiply–add into two pre-scaled query halves, finishing with a vector reduction. No expanded floating-point key block is ever written to memory.

A scalar version of the kernel with identical data layout runs on the host and is checked against NumPy (Section 6.6). The RVV version uses the standard intrinsics and compiles for `rv64gcv`. It has not been executed (Chapter 7).

5.3.10 The runtime seam and the codec slot

A narrow `RuntimeAdapter` interface (write token, read block, migrate block, report attention importance) separates the engine from any inference framework. `ReferenceAdapter`

implements it on the NumPy engine and is exercised in the tests. `RuntimeAdapter skeleton for a compiler-based runtime` is a skeleton whose docstring lists the integration steps: map to the runtime's KV pages, obtain attention weights (fused attention kernels do not normally expose them, so a row-sum output or a side kernel is needed), perform synchronised tensor movement, and confirm that the compiler can target RVV.

The codec layer exposes `encode(k, v, codec, v_group)` → `Payload` and `decode(Payload)` → `(k, v)`. The group quantiser and outlier handling of a hardware compression engine, or any other codec, can be wired in by implementing the same interface and registering a codec name.

5.4 UAMM-2 code: structure and how to run it

5.5 Project Structure and Code Walkthrough

This chapter explains how the project is organised and what each part of the code does, so that a reader can find any behaviour described in this report.

5.5.1 Directory layout

```
uamm2_project/
  README.md  requirements.txt
  uamm2/                 Python reference engine
    config.py              all parameters and the assumed cost constants
    codecs.py              quantisation, packing, entropy coding, calibration
    paged_cache.py         blocks, sealing, re-encoding, dropping
    tiers.py               tier usage and the latency model
    h2o.py                 attention-based importance
    attention.py           block selection, sparse attention, dense oracle
    planner.py             utility planner and the original threshold planner
    simulator.py           decode loop, metrics, per-step trace
    workload.py            synthetic query/key/value generator
    ablation.py cli.py __main__.py     variants and command line
    hwmodel.py             bit-exact models of the RTL + test-vector writer
    ara.py                 host build of the C kernel; RISC-V compile check
    runtime.py             runtime adapter interface, reference adapter, skeleton
  hw/
    rtl/                   uamm_sweep_scorer.v  uamm_kv_codec.v  uamm_block_table.v
    tb/                    self-checking testbenches (+trace writes VCD files)
    vectors/               generated stimulus and expected outputs (hex)
    Makefile               test, lint, vectors, clean
  ara/                     kv_kernels.c / .h : scalar and RISC-V vector kernel
  tests/                   59 pytest tests
  experiments/             run_experiments.py, mutation_test.py, figure scripts, results/
  docs/                    waveform dumps, synthesis reports, captured tool output
  report/                  scripts that build this document
```

5.5.2 Python engine, module by module

Table 5.3: Python modules.

File	Main items	What it does
`config.py`	`Config`, `TierSpec`	Holds geometry (64 channels, 16-token blocks), protection (4 sinks, 32 recent), planner settings (interval 8, hysteresis 0.25, residency 16, $\lambda = 2$), codec set and the assumed latency, bandwidth and decode constants. Validates inputs on creation.

File	Main items	What it does
codecs.py	encode, decode, calibrate, pack_bits, parse_codec	Turns a block of keys and values into a byte payload: asymmetric quantisation (keys per channel, values per token in groups of 32), bit packing, optional zlib. calibrate encodes a block with every codec and returns its error and size tables. Codec names: fp16, int8, kivi4, kivi2 and their z forms.
paged_cache.py	KVBlock, PagedKVCache	append_token fills the open block; at 16 tokens _seal rounds to FP16, stores the payload, computes key min/max and calibrates. reencode changes codec (lossless for the entropy wrapper, lossy otherwise); drop removes a block.
tiers.py	usage, read_cost_ms, move_cost_ms, decode_cost_ms	Accounts bytes per tier and computes the modelled read, decode and migration costs. No data actually moves.
h2o.py	H2OImportance	update decays all token scores and adds the attention weight each token received; refresh aggregates to blocks (mean, max or sum) and normalises by the 95th percentile.
attention.py	select_blocks, sparse_attention, dense_oracle	Ranks candidate blocks by the key-bound formula and returns the top budget; computes attention over the read blocks; the oracle computes full FP32 attention for the error metrics.
planner.py	UtilityPlanner, ThresholdPlanner, codec_allowed	The utility planner enumerates allowed (tier, codec) pairs, evaluates the cost, visits blocks by heat, applies capacity and hysteresis and returns a plan. The threshold planner reproduces the original policy for comparison.
simulator.py	Simulator.run	For every token: generate q/k/v, append, update protection, select blocks, run sparse and dense attention, record metrics, update hit rates and importance, and every 8 steps plan and apply. Writes a per-step trace.
workload.py	SyntheticWorkload	Generates sinks, outlier channels, topic phases and local recency (Section 6.2).
ablation.py, cli.py	variants, main	Defines the ablation variants; commands simulate, ablate, hwvectors.
hwmodel.py	scorer_model, quant4_model, dequant4_model, write_vectors	Integer-exact copies of the RTL arithmetic and the generator of boundary and random vectors that the testbenches read.
ara.py	AraKernels, rvv_compile_check	Builds the C kernel for the host and calls it through ctypes to score packed 4-bit keys; compiles the vector path for a RISC-V target.
runtime.py	RuntimeAdapter, ReferenceAdapter	Interface to an inference runtime (write token, read block, migrate block, report attention); a working reference implementation and a skeleton for a compiler-based runtime.

5.5.3 The planner in code

The planner is the part of UAMM-2 that replaces UAMM-1's thresholds, so it is worth reading in full. `_cost` is the formula from the design section: the read and decode time weighted by how likely the block is to be read, plus the quality penalty weighted by the block's heat, plus a price for using a scarce tier. `plan` sorts the movable blocks from hottest to coldest and, for each one, tries every allowed (tier, codec) pair that has room and does not raise precision. It keeps the cheapest pair, but stays where it is when the block moved recently or when the gain is smaller than the hysteresis margin. If nothing fits, the block stays and the overflow is counted.

UAMM-2 planner: cost and plan (planner.py)

```
class UtilityPlanner:
    def __init__(self, cfg: Config):
        self.cfg = cfg
        self.cands = [(t, c) for t in range(len(cfg.tiers)) for c in all_codec_names(cfg)
                      if codec_allowed(t, c)]

    @staticmethod
    def heat(b: KVBlock) -> float:
        return 0.5 * b.hit_ema + 0.5 * b.importance

    def _cost(self, b, t, c, nbytes, used, cap) -> float:
        cfg = self.cfg
        p = max(b.hit_ema, 0.01)
```

```
        fill = used[t] / max(cap[t], 1.0)
        return (p * read_cost_ms(cfg, t, c, nbytes)
                + cfg.lambda_quality * self.heat(b) * b.err_table[c]
                + cfg.capacity_price * (nbytes / cfg.tiers[t].capacity_bytes) * (1.0 + 4.0 * fill))

    def plan(self, blocks: List[KVBlock], step: int) -> Plan:
        cfg = self.cfg
        cap = [s.capacity_bytes * (1.0 - cfg.capacity_headroom) for s in cfg.tiers]
        used = _fixed_usage(cfg, blocks)
        movable = [b for b in blocks if b.sealed and not b.protected and not b.dropped]
        movable.sort(key=lambda b: -self.heat(b))
        plan: Plan = {}
        for b in movable:
            best, best_cost, cur, cur_cost = None, float("inf"), None, float("inf")
            for t, c in self.cands:
                if codec_rank(c) < b.rank:
                    continue
                nbytes = b.payload.nbytes if c == b.codec else b.size_table[c]
                if used[t] + nbytes > cap[t]:
                    continue
                cost = self._cost(b, t, c, nbytes, used, cap)
                if cost < best_cost:
                    best, best_cost = (t, c, nbytes), cost
                if (t, c) == (b.tier, b.codec):
                    cur, cur_cost = (t, c, nbytes), cost
            choice = best
            if cur is not None and best is not None:
                too_soon = (step - b.last_move_step) < cfg.min_residency_steps
                close = cur_cost <= best_cost + cfg.hysteresis_margin * max(best_cost, 1e-4)
                if too_soon or close:
                    choice = cur
            if choice is None:
                if cfg.allow_drop and b.importance < cfg.drop_importance:
                    plan[b.block_id] = (DROP, b.codec)
                else:      # nothing fits: stay put (counted as overflow by the simulator)
                    used[b.tier] += b.nbytes(cfg.head_dim)
                continue
            t, c, nbytes = choice
            used[t] += nbytes
            plan[b.block_id] = (t, c)
        return plan


class ThresholdPlanner:
```

5.5.4 Hardware

Table 5.4: Hardware files.

File	What it does
`uamm_sweep_scorer.v`	Metadata SRAM (25-bit words) and a 2-stage pipeline. A `start` pulse latches the pressure value and streams all blocks through `score = (154·imp + 64·rec + 38·frq) » 8`, the tier decision (protected override, thresholds 174 and 89) and the eviction flag (pressure ≥ 243, score < 89, not protected).
`uamm_kv_codec.v`	`uamm_kv_quant4`: 16 lanes of subtract, multiply by the Q4.12 reciprocal, round, clamp to 15, pack nibbles. `uamm_kv_dequant4`: multiply by the Q8.8 step, round, add zero point, saturate at 255.
`uamm_block_table.v`	256-entry table of `{tier, codec, phys}` with valid bits and a one-cycle lookup returning `hit`.
`tb_*.v`	Read the generated vectors, drive the module, compare every output and print PASS or FAIL; with `+trace` they write a VCD.
`Makefile`	`make test` regenerates vectors and runs the three benches in Verilator; `make lint` runs `-Wall` lint.

5.5.5 Vector kernel

`kv_qk_prepare` folds the per-channel scale into the query (split into even and odd channels) and returns the bias Σ q $\cdot$ zero. `kv_qk_dot_kivi4_scalar` loops over tokens and packed bytes with two multiply–adds per byte. `kv_qk_dot_kivi4_rvv` does the same with vector instructions (byte load, nibble extraction, widen, convert, fused multiply–add with tail-undisturbed policy, reduction). `kv_qk_dot_kivi4` selects the vector path when compiled for a vector target.

5.5.6 Tests

Table 5.5: Test files.

File	Tests	What is checked
`test_codecs.py`	10	Pack/unpack round trip and nibble order; FP16 exactness; error and size ordering across codecs; entropy variants lossless; per-channel keys beat per-token on outlier channels; constant input; bad inputs
`test_cache_h2o_attention.py`	8	Block sealing and partial tail; ratchet and lossless entropy switch; the selection bound is a true upper bound; sparse equals dense when everything is read; heavy hitters rank higher; decay forgets
`test_system.py`	12	Combined system fits without data loss; threshold policy must drop; combined system beats it on quality here; capacities respected; protected blocks stay hot and lossless; precision never increases; hysteresis lowers churn; drop only when enabled; adapter round trip; skeleton honesty
`test_ara_hw.py`	6	C kernel against NumPy; rejects other codecs; vector path compiles; golden scorer properties; codec error bounded; full RTL regression in Verilator

5.5.7 How to run everything

```
pip install numpy pytest verilator        # g++ with C++20 needed for the RTL tests
python -m uamm2 simulate --tokens 1024     # one configuration
python -m uamm2 ablate --tokens 1536       # UAMM-1 vs UAMM-2 variants
python -m pytest -q tests                  # 56 tests, ~40 s
make -C hw test ; make -C hw lint          # RTL regression and lint
python experiments/run_experiments.py      # all studies -> experiments/results/*.json
python experiments/mutation_test.py        # RTL mutation testing
bash docs/gtkwave/view_waves.sh 3          # open a waveform with signals preloaded (1-6)
```

5.6 What we tried, what we saw and what we decided

The table at the end of this part collects the decisions. The paragraphs explain the thinking behind the ones that mattered most.

5.6.1 Thresholds or a cost?

UAMM-1 placed a block by comparing a weighted score with two thresholds. It was easy to write and easy to put in Verilog, but a threshold knows nothing about how full a level is or what a codec would cost. On the synthetic trace it deleted 973 of 1,536 tokens once memory ran out. We then made a capacity-aware version of UAMM-1 so that the comparison would be fair, and it still had no way of making data smaller. We moved to a cost: the expected read and decode time of the block, plus a penalty for quantisation error

weighted by how hot the block is, plus a price for scarce capacity. With a cost, adding a new codec or tier only adds a new option to choose from; with thresholds it adds more thresholds to tune. This was a decision by reasoning, backed by the E1 results.

5.6.2 Block size

We started with 16-token blocks because UAMM-1 and PagedAttention use that size. We then tried 8 and 64 in the sensitivity experiment. Blocks of 8 reduced the error to 0.110 but needed 1,652 migrations, against 630 for 16 and 0.159 error. Blocks of 64 were coarse: the error rose to 0.341 and the budget overflowed by about 14 KB. Smaller blocks also mean more table entries and more key summaries, which an edge core has to store. We kept 16. This was decided by experiment.

5.6.3 An importance score that does not saturate

The importance value in UAMM-1 was clamped to 1.0. After a short time the heavy-hitter tokens all sat at the limit and the controller could no longer tell which one mattered more. In UAMM-2 we decay the token scores by 0.95 at every step so that old attention fades, and we normalise the block score by the 95th percentile of all blocks so that one extreme block cannot flatten the others. This was found by looking at the scores of a run and then confirmed in the sensitivity experiment.

5.6.4 Which storage formats to offer

We implemented FP16, INT8, KIVI-style 4-bit and KIVI-style 2-bit, and tried them in groups. FP16 with INT8 alone gave a size of 0.606 of FP16 and overflowed the budget by 104,704 B, so it cannot fit the trace. Using 2-bit for everything fitted and was small, but the error was 0.308 and the modelled fetch time 0.475 ms. INT8 with 4-bit fitted at an error of 0.122. Adding 2-bit as well gave the smallest size, 0.310 of FP16, at an error of 0.166. We left all four on, because the planner only uses 2-bit for blocks that are cold. This was decided by experiment.

5.6.5 Entropy coding on top of quantisation

We also tried zlib on the packed blocks, hoping that cold, low-precision data would compress further. On our synthetic data it made no difference to the measured size. We kept the code in the repository but did not use it in the headline results. We cannot say whether real activations would behave differently, so we treat this as undecided and not as a negative result. Entropy coding would also cost decode time on a core that has little of it to spare.

5.6.6 Moving blocks too often

In the first runs the planner moved blocks back and forth after almost every plan: 1,871 migrations. Each move costs bandwidth that the model does not see unless it is charged. We added three rules: a block must stay for a minimum of 16 tokens, a move must beat

the current placement by a margin of 0.25, and the planner only runs every 8 tokens. Migrations fell to 630. We then varied the planning interval: replanning every 2 tokens gave 599 flash reads and 0.090 ms per token, but costs more planning work; every 64 tokens gave 3,326 flash reads and 0.272 ms because the plan was stale. An interval of 8 was a compromise. This was decided by experiment.

5.6.7 How many blocks to read

Reading every block at every step is what makes memory traffic large. We used a Quest-style upper bound on the attention a block can receive for the current query, and then read only the blocks with the highest bounds. With a budget of 4 blocks the attention mass kept was only 0.568 and the error 1.10. With 24 blocks it was 0.945 and 0.159. With 48 blocks it was 0.986 and 0.07, but the fetch time rose to 0.278 ms. We chose 24. This was decided by experiment.

5.6.8 Ideas we set aside

- **Running the whole planner in hardware.** The cost function needs several multiplies and a division per block. We kept to the integer scorer (weights 154/64/38, shift by 8) and left the planner in software. This was decided by reasoning about area and about what an edge core can afford, not by synthesis.
- **Per-token eviction.** Deleting single tokens would need a table entry per token. At 16 tokens per block the table is small enough for the memory we have. Chosen by reasoning.
- **Large pages.** The 64-token block size above is the experiment that told us this was not a good idea for our budget.
- **Learned policies.** A learned policy needs real traces and training data that we did not have. It is listed under future scope.
- **Running a real model.** We had neither the memory nor the tools on our machines to run a full model on RISC-V, so we kept a synthetic generator and said so wherever a number is quoted.

Table 5.6: Design decisions and how each one was reached.

Decision	What we chose	How it was decided
Placement rule	Cost-based planner	Reasoning, supported by E1
Block size	16 tokens	Experiment (E4)
Importance signal	Decay 0.95, 95th-percentile normalisation	Observation, then E4
Storage formats	FP16, INT8, KIVI-4, KIVI-2	Experiment (E1, E5b)
Entropy coding	Not used	Experiment: no gain on synthetic data
Stability	Residency 16, margin 0.25, interval 8	Experiment (E4)
Blocks read per token	24	Experiment (E3)
Hardware scope	Integer scorer, codec, table	Reasoning

5.7 Problems in the design

Building the system and measuring it raised problems we had not planned for. The table below lists the design problems and what we did about each. The list after it covers what

we changed in the way we check our own results.

5.7.1 Problems in the design

Table 5.7: Design problems and what we did about them.

Problem	How we tackled it
H2O scores are sums that grow without limit; UAMM-1 clamped them at 1.0, so heavy hitters saturated and could not be told apart.	Decay the token scores at every step and normalise block importance by the 95th percentile of all blocks.
We could not say how much error a codec would add to a given block.	Measure it: when a block is sealed, encode and decode it with every codec and store the relative error. The planner uses the measured value.
Keys have a few channels with very large values, so plain per-token quantisation wrecks them.	Follow KIVI: scale keys per channel and values per token. A test checks that per-channel keys give lower error on outlier data.
No real model or real activations were available on our machines.	Write a synthetic generator that has the properties we care about (a few heavy-hitter tokens, topic revisits, outlier channels), say so wherever a number is quoted, and compare variants with each other, not with an absolute bar.
Blocks moved back and forth between tiers after every plan.	Add a minimum residency, a required improvement margin and a plan interval. Migrations fell from 1,871 to 630.
The Verilog scorer divided by 100, which costs area and time.	Choose integer weights 154/64/38 that sum to 256 and shift right by 8. The codec likewise multiplies by a precomputed reciprocal.
The first testbench had three vectors and passed everything.	Write 17 sweeps with the threshold edge cases, 400 codec vectors, 6,000 random table operations and a mutation test. The mutation test found a quantiser clamp that no vector exercised; we added vectors until it was caught.
We had no Ara processor or simulator to run the vector kernel on.	Run the scalar path on the host against NumPy, compile the vector path for rv64gcv and inspect the instructions, and state clearly that it was never executed.

5.7.2 What we do now

- **Make the baseline fair before claiming a win.** The most important error was comparing against a weak baseline. We asked what the simplest competent alternative would be, built it (a single 4-bit codec with no cost model), and let it into every table. The result changed the conclusion: the planner is not the reason the data fits; the codecs are.
- **Report what is charged and what is not.** Any memory that is used but not counted flatters the result. We measured the key summaries, reported them, and ran the experiment with them charged.
- **Separate the measurement from the model.** Latency, bandwidth and decode costs are assumptions, the workload is synthetic, no language model was run and nothing ran on Ara. Each statement now says which of these it depends on.
- **State the budget dependence.** The gain over UAMM-1 comes from a budget 2.3× smaller than the data. At 3× the budget FP16 fits and wins. We now show the sweep next to the headline.
- **Re-derive each number from the result files.** The text was checked against the JSON files in the repository. Every number we changed in the text is one that could be traced to an experiment.
- **Label design ideas as design ideas.** Things that are proposed but not built are labelled as such in the chapter title and in a status note.
- **Cite or remove.** A claim with no source was removed; a named system was cited.

5.7.3 What our results show

On a synthetic workload with assumed memory costs, a joint (tier, codec) choice lets a 168 KB budget hold 1,536 tokens that FP16 cannot, without deleting any token, where the original UAMM-1 threshold policy deleted 63 % of them. A simple all-4-bit baseline does the same and is faster in the model. The planner's measured benefits are lower output error at equal memory, lower memory with the 2-bit codec, and feasibility in tighter budgets, at the price of a higher modelled fetch time than that baseline. We claim nothing about speed on real hardware, about real language models, or about the vector processor, which only compiled the kernel.

6. Simulation Results and What We Obtained

This chapter reports what the simulations and the hardware checks showed and what we can conclude from them. It starts with what we assumed, because every number below depends on those assumptions, and ends with what we think is new in this work.

6.1 What we assumed

Table 6.1: What the results rest on.

Assumption	What we used	Effect on the conclusions
Workload	A synthetic stream of queries, keys and values for one head of 64 channels, with heavy-hitter tokens, topic revisits and outlier channels. No language model, no tokenizer.	Errors are relative to a dense FP32 reference on this stream. Compare variants with each other, not with a fixed bar.
Memory costs	Four tiers of 24, 48, 32 and 64 KB with assumed latencies and bandwidths (SRAM, two DRAM tiers, flash). Decode costs per byte assumed for each codec.	Fetch time is a model output. Changing the constants can change which variant is fastest.
Budget	168 KB in total for 1,536 tokens; FP16 would need 393 KB.	The advantage over UAMM-1 exists because the budget is tight. At three times the budget FP16 simply fits.
Key summaries	The per-block min/max key vectors (24,576 B) are not charged in the main runs.	Charging them changes which variants fit (see the key-summary experiment).
Hardware	Verilator simulation against Python models. No place and route, no timing, no power.	The RTL results show functional correctness, not area or speed.
Vector processor	The kernel was compiled for RISC-V vector but never run on Ara or a simulator.	No speed-up from the kernel is claimed.

6.2 Experimental Methodology

This chapter states precisely what was simulated, what was real, how quality was measured and which constants were assumed. The goal is that a reader can judge exactly what each number in the results chapter means.

6.2.1 What is real and what is modelled

Table 6.2: Status of each element of the evaluation.

Component	Status in the experiments
KV tensors, codecs, packing, entropy coding	**Real computation.** Blocks are actually quantised, bit-packed, optionally zlib-compressed, decoded and used for attention.
Attention output and quality metrics	**Real computation** on decoded tensors, compared with a dense FP32 attention oracle over all tokens at the same step.
Block selection (min/max bound), importance scores, planner decisions	**Real computation** (the same code that would run on a device).
Q, K, V stream	**Synthetic**: produced by a parametrised trace generator, described in 6.2, not by a neural network.
Tier latencies, bandwidths, decode costs	**Modelled constants** (6.4). The reported latency is a model output, not a measurement.
RTL behaviour	**Real simulation** with Verilator, compared bit-exactly with Python golden models.

Component	Status in the experiments
RVV kernel	Scalar variant executed on the host; RVV variant compiled to RISC-V vector instructions but not executed.

6.2.2 The synthetic trace generator

An LLM was not run for the experiments in this report. Instead a trace generator produces, for each decode step, a query vector and a key and value vector with the structural properties that KV-cache managers are designed around. The generator has 64 channels (`head_dim`) and exposes these features:

- **Attention sinks.** The first four tokens, and 3 % of later tokens at random, receive an extra component along a fixed "heavy" direction; the query also has a component along it. These tokens attract disproportionate attention, as heavy hitters and sinks do in real models.
- **Outlier key channels.** One in sixteen channels of the keys is scaled by 6× (4 of 64 channels). This is the structure that motivates per-channel key quantisation.
- **Topic phases.** Four unit topic directions are defined. Keys in each phase of 96 tokens carry that phase's topic. The query's topic follows the phase but periodically (every third phase) returns to an older topic, so that previously cold blocks become hot again and must be promoted.
- **Local recency.** Part of the query is the direction of one of the 12 most recent keys, producing local attention.
- **Sharpness.** The query is scaled by a factor of 4, which controls how peaked the softmax is; value vectors are independent Gaussian noise.

Because the generator is designed rather than learned, its absolute numbers must not be read as what a particular LLM would give. In particular, values that are independent noise average to a small-norm vector, which inflates the *relative* L2 error of the output; the comparison of variants against each other on the same stream is the meaningful quantity. Section 6.4 includes a measurement of how concentrated the attention on this generator is, so that the reader can compare it with concentration reported for real models.

6.2.3 Metrics

Table 6.3: Metrics reported in the experiments.

Metric	Definition	Why
`resident_vs_fp16`	Total bytes held across tiers divided by the FP16 size of all tokens	Memory saving (lower is better)
`dropped_tokens`	Tokens permanently removed	Irreversible information loss
`attn_rel_l2`	$\|o_sys - o_dense\| / \|o_dense\|$ for the attention output at each step after a warm-up of 64 steps (mean and 95th percentile)	Output fidelity vs a dense FP32 oracle over the same (FP16-rounded) data
`attn_mass_recall`	Fraction of the oracle's attention probability that falls on the tokens the system read	How much of what attention wants was available
`sim_fetch_ms_per_token`	Modelled time to fetch and decode all blocks read in a step (mean)	Latency of reading the KV cache under the assumed constants
`flash_block_reads`	Count of block reads served from flash	Proxy for slow-tier traffic
`migrations`, `reencodes`	Cumulative counts of tier moves and codec changes	Churn (energy and bandwidth cost of management)

Metric	Definition	Why
`overflow_total`	Sum over tiers of the maximum bytes by which a tier budget was exceeded	Feasibility: non-zero means the data did not fit

Interpretation guard. The attention-mass recall depends only on which blocks are read and not on their precision, so it is identical for all UAMM-2 variants that use the same selection logic; the codec choice shows up in the relative L2 error. A non-zero error floor remains even with FP16 everywhere, because block-sparse attention omits some tokens; the FP16-only variant (error 0.092 at 1,536 tokens) measures that floor.

Accounting rule. `resident_vs_fp16` and the tier budgets count the bytes of each block's encoded payload, including its scales and zero points. They do **not** count the small per-block summaries that the engine keeps resident for selection (key minimum and maximum, 64 values each) or the scheduling state. Stored in FP16, the key summaries would add about 256 bytes per block, i.e. about 25 KB for the 96 blocks of a 1,536-token run, which is roughly 6 % of the FP16 cache size and 15 % of the 168 KB budget. The results below should be read with that omission in mind; Chapter 7 lists it among the open items.

6.2.4 Assumed cost constants

The following constants are placeholders for an edge SoC and are not measurements. They define the latency model used by the planner and the simulator. Conclusions about *ratios* between variants depend on them; they should be replaced by values measured on the target before any absolute claim is made.

Table 6.4: Default configuration of the experiments.

Constant	Value	Meaning
T0 latency / bandwidth	0.5 µs / 100 GB/s	Near-compute SRAM or vector register file
T1, T2 latency / bandwidth	2 µs / 12.8 GB/s	LPDDR-class DRAM
T3 latency / bandwidth	100 µs / 1 GB/s	Flash
Decode cost, fp16 / int8 / kivi4 / kivi2	0 / 0.05 / 0.12 / 0.20 ns per byte	Software dequantisation cost model
zlib decode cost	1.0 ns per byte	Additional cost for the **z** variants
Tier capacities T0..T3	24, 48, 32, 64 KB (total 168 KB)	FP16 KV for 1,536 tokens is 393 KB
Block size, v_group, sink, recent window	16 tokens, 32 channels, 4 tokens, 32 tokens	Geometry and protection
Attention block budget	24 blocks per step	Non-protected blocks fetched
Planner: plan interval, hysteresis, residency, λ, κ, headroom	8 steps, 0.25, 16 steps, 2.0, 0.05, 3 %	See Section 5.3
H2O decay, aggregation, hit_ema	0.95, mean, 0.9	Importance signal

λ was chosen on seeds 7–9 during development; all reported experiments use other seeds (11–15), so the default configuration is evaluated on data it was not tuned on.

6.2.5 Experimental design

Table 6.5: Experiments performed.

ID	Question	Variable	Seeds	Tokens
E1	Does each ingredient matter? (ablation)	Policy and codec set: UAMM-1 threshold; planner with FP16 only; INT8+KIVI-4; all codecs	5	1,536
E2	How does the system scale with context?	Context 512–3,072 at a fixed budget	3	512–3,072
E3	How many blocks must be read?	Attention block budget 4–48	3	1,536
E4	How sensitive are the planner and importance knobs?	λ, hysteresis margin, residency, plan interval, decay, aggregation, block size	3	1,536
E5	How does memory budget change the picture?	Tier budgets $\times 0.5$ to $\times 3.0$; and codec sets with and without entropy coding	3	1,536
E7	How good are the codecs, and does the key axis matter?	Codec and outlier presence; per-channel vs per-token keys	200 random blocks per cell	—

Values are reported as mean ± standard deviation across seeds. With three seeds the standard deviation is itself a rough estimate; differences smaller than about one standard deviation should not be treated as significant. The experiment code is in `experiments/run_experiments.py`, listed in the appendices, and all raw numbers are tabulated in Appendix C.

6.3 UAMM-1 and UAMM-2 compared

Both systems were run on the same synthetic trace of 1,536 tokens with the same total memory budget of 168 KB. FP16 storage of that cache would need 393 KB, so the budget is 2.3 times too small and something has to give. The table below lists the outcome; we also include the simple baseline of storing every block in 4-bit, because it is the fair question to ask of any planner.

Table 6.6: UAMM-1 against UAMM-2 and a one-codec baseline (1,536 tokens, 168 KB, mean ± std over 5 seeds).

System	Tokens deleted	Resident / FP16	Attention mass kept	Output error	Fetch ms / token (model)
UAMM-1 (threshold policy, FP16)	973 ± 16	0.367 ± 0.010	0.758 ± 0.016	0.479 ± 0.035	0.923 ± 0.026
No planner, every block 4-bit	0 ± 0	0.364 ± 0.000	0.939 ± 0.012	0.152 ± 0.015	0.068 ± 0.003
UAMM-2, INT8 + 4-bit codecs	0 ± 0	0.371 ± 0.003	0.939 ± 0.012	0.122 ± 0.014	0.116 ± 0.002
UAMM-2, INT8 + 4-bit + 2-bit codecs	0 ± 0	0.310 ± 0.001	0.939 ± 0.012	0.166 ± 0.020	0.112 ± 0.002

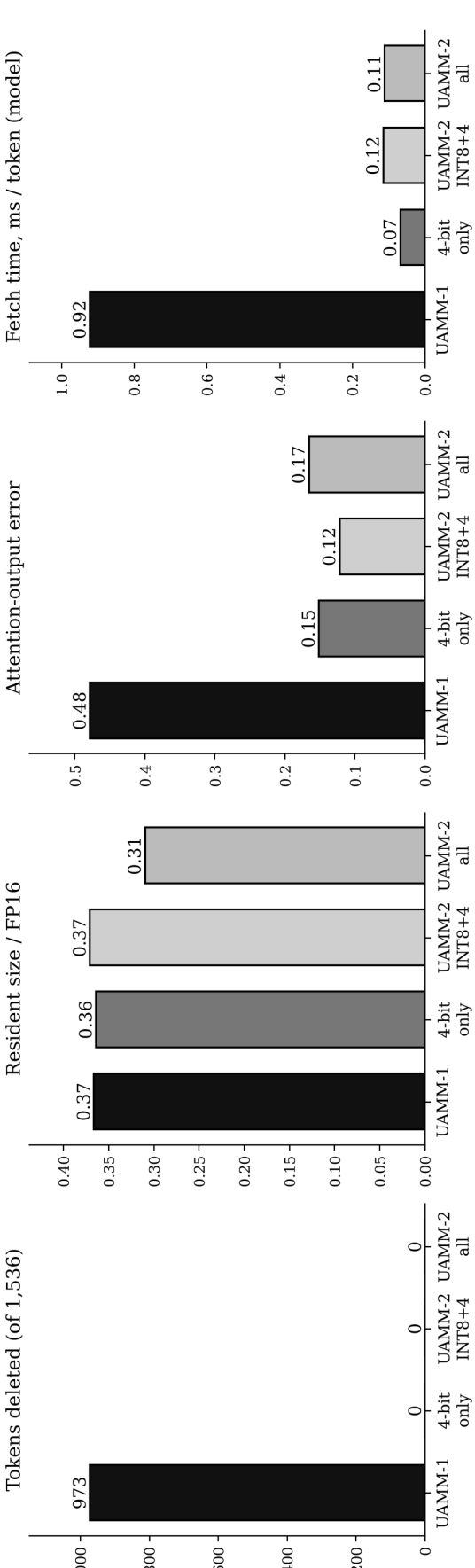


Figure 6.1: UAMM-1 against UAMM-2 on the same trace and budget. Labels: 4-bit only = every block in 4-bit with no planner; INT8+4 = UAMM-2 with INT8 and 4-bit codecs; all = UAMM-2 with every codec.

- **What clearly improved.** UAMM-1 had to delete 973 of 1,536 tokens to fit and kept 76 % of the attention mass. UAMM-2 keeps every token and 94 % of the attention mass, and its output error is about a quarter to a third of UAMM-1's.
- **What did not.** UAMM-2 is not faster than the all-4-bit baseline in our model (0.11 against 0.07 ms per token), and that baseline also keeps every token. The error of the baseline is higher (0.152 against 0.122 for UAMM-2 at the same size).
- **So what does the planner add?** Control. It spends precision where attention actually goes, so at the same size it has lower error; with the 2-bit codec it reaches a smaller size; and in tighter budgets it still fits when the single-codec baseline does not (Chapter 5).

6.4 Simulation Results

All numbers in this chapter come from `experiments/run_experiments.py` (raw data in `experiments/results/*.json`) under the configuration of Section 6.2. Latency figures are model outputs under the assumed constants. Errors are measured against a dense FP32 oracle on the synthetic trace. Where we interpret a result, we say so.

6.4.1 Ablation: what each ingredient contributes (E1)

Table 6.7: Ablation and baselines at 1,536 tokens (mean ± std over 5 seeds). The first column of sizes does not count the key min/max summaries; the second does.

Variant	Resident / FP16	Resident + summaries	Dropped tokens	Rel. L2 error	Mass recall	Fetch ms / token (model)	Flash reads	Overflow (B)
UAMM-1 threshold policy, FP16	0.367 ± 0.0102	0.412 ± 0.0115	973 ± 16	0.479 ± 0.0346	0.758 ± 0.0164	0.923 ± 0.0259	12613 ± 374	0 ± 0
UAMM-1 made capacity-aware, FP16	0.406 ± 0	0.457 ± 0	912 ± 0	0.406 ± 0.0305	0.792 ± 0.0116	0.618 ± 0.00894	8205 ± 129	0 ± 0
No planner: every block 4-bit	0.364 ± 0	0.489 ± 5.55e-17	0 ± 0	0.152 ± 0.015	0.939 ± 0.0117	0.0681 ± 0.00253	270 ± 37	0 ± 0
No planner: every block 2-bit	0.243 ± 0	0.368 ± 0	0 ± 0	0.572 ± 0.0187	0.939 ± 0.0117	0.0493 ± 4e-06	0 ± 0	0 ± 0
Planner, FP16 only	1 ± 0	1.12 ± 0	0 ± 0	0.0921 ± 0.0191	0.939 ± 0.0117	1.04 ± 0.006	14364 ± 86	258048 ± 1832
Planner, INT8 + KIVI-4	0.371 ± 0.00265	0.496 ± 0.00265	0 ± 0	0.122 ± 0.0137	0.939 ± 0.0117	0.116 ± 0.00216	970 ± 32	0 ± 0
Planner, INT8 + KIVI-4 + KIVI-2	0.31 ± 0.00133	0.435 ± 0.00133	0 ± 0	0.166 ± 0.0203	0.939 ± 0.0117	0.112 ± 0.00203	915 ± 30	0 ± 0

Reading the table.

- **UAMM-1 fits by deleting.** With only FP16 blocks and a 168 KB budget the threshold policy must remove on average 973 of 1,536 tokens (63 %). Attention-mass recall falls to 0.758 and the output error reaches 0.479; the reads that remain are frequently from flash (about 12,600 flash block reads).
- **A fairer UAMM-1 still deletes.** The threshold rule sends a block to the tier its score selects or to a slower one, so it can place data in flash while faster tiers have room. Letting the best-scoring blocks take the fastest tier with room (the capacity-aware variant) lowers modelled fetch time from 0.923 to 0.618 ms and error from 0.479 to 0.406, but it still drops 912 tokens, because FP16 alone cannot hold 393 KB in 168 KB.

- **The planner alone cannot create space.** With FP16 only, the planner drops nothing but cannot fit: 258 KB of overflow, i.e. the data exceeds the stated budget by that much. The lower error (0.092) is the floor due to block-sparse selection with exact data. This variant shows that placement alone is insufficient: without a codec, a budget below the data size is infeasible without deletion.
- **Codecs make the budget feasible.** Adding INT8 and 4-bit codecs reduces residency to 0.371 of FP16, with no overflow and no drops. Error rises to 0.122 (vs the 0.092 floor): the price of quantisation.
- **A single codec for everything is a strong baseline.** Storing every unprotected block in 4-bit, with no cost model, also fits the budget (0.364 resident), keeps every token, and reaches error 0.152 and 0.068 ms. Against it the planner with INT8 + KIVI-4 has lower error (0.122 versus 0.152) at almost the same memory (0.371), but a higher modelled fetch time (0.116 versus 0.068 ms), because it spends DRAM on higher-precision blocks and pushes other blocks to flash. Section 6.4.1.1 shows that these are different points on one trade-off curve.
- **The 2-bit codec is enabled by default and used heavily.** Enabling 2-bit lowers residency from 0.371 to 0.310 (a further 16 % saving) at an error of 0.166, i.e. 36 % more than INT8 + KIVI-4. It is not a rarely used option: in the final state about 49 % of all blocks are 2-bit, 45 % are 4-bit and 6 % are FP16 or INT8. Storing everything at 2-bit reaches 0.243 resident but error 0.572.
- **Latency model.** Modelled fetch time per token falls from 0.92 ms (UAMM-1) to 0.116 ms (INT8 + KIVI-4) and 0.112 ms (all codecs). Part of that difference comes from the UAMM-1 placement rule described above, and the 4-bit baseline is faster still (0.068 ms). The FP16-only planner is slower than UAMM-1 (1.04 ms) because overflowing blocks must live in flash with no compression.

What the ablation says

UAMM-1's weakness is the lack of a way to make data smaller: a joint (tier, codec) choice turns a budget that was infeasible without deleting tokens into a feasible one, at a bounded and measured quantisation error. The planner is not what makes the budget feasible, however, since a one-codec baseline does that too. What the planner adds is control over where precision is spent: lower error at the same memory (INT8 + KIVI-4), lower memory (with 2-bit), and feasibility in tighter budgets (Section 6.4.5). In the model it does not make fetching faster than the 4-bit baseline.

6.4.1.1 Baselines without a planner (E10)

Table 6.8: Baselines that place blocks without a cost model (hottest block first into the fastest tier with room) against the planner over its quality weight λ (1,536 tokens, 3 seeds).

Configuration	Resident / FP16	Rel. L2 error	Fetch ms / token	Overflow (B)
No planner, all INT8	0.615 ± 0.003	0.083 ± 0.018	0.706 ± 0.013	110,251 ± 1,247
No planner, all 4-bit	0.364 ± 0.000	0.143 ± 0.013	0.068 ± 0.003	0 ± 0
No planner, all 2-bit	0.243 ± 0.000	0.577 ± 0.020	0.049 ± 0.000	0 ± 0
Planner INT8 + KIVI-4, $\lambda = 0.5$	0.364 ± 0.000	0.131 ± 0.012	0.110 ± 0.002	0 ± 0
Planner INT8 + KIVI-4, $\lambda = 2.0$	0.372 ± 0.002	0.116 ± 0.013	0.115 ± 0.003	0 ± 0
Planner INT8 + KIVI-4, $\lambda = 8.0$	0.383 ± 0.004	0.110 ± 0.013	0.118 ± 0.002	0 ± 0
Planner all codecs, $\lambda = 0.5$	0.284 ± 0.002	0.200 ± 0.028	0.106 ± 0.003	0 ± 0
Planner all codecs, $\lambda = 2.0$	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	0 ± 0

Configuration	Resident / FP16	Rel. L2 error	Fetch ms / token	Overflow (B)
Planner all codecs, $\lambda = 8.0$	0.325 ± 0.003	0.138 ± 0.012	0.245 ± 0.003	0 ± 0

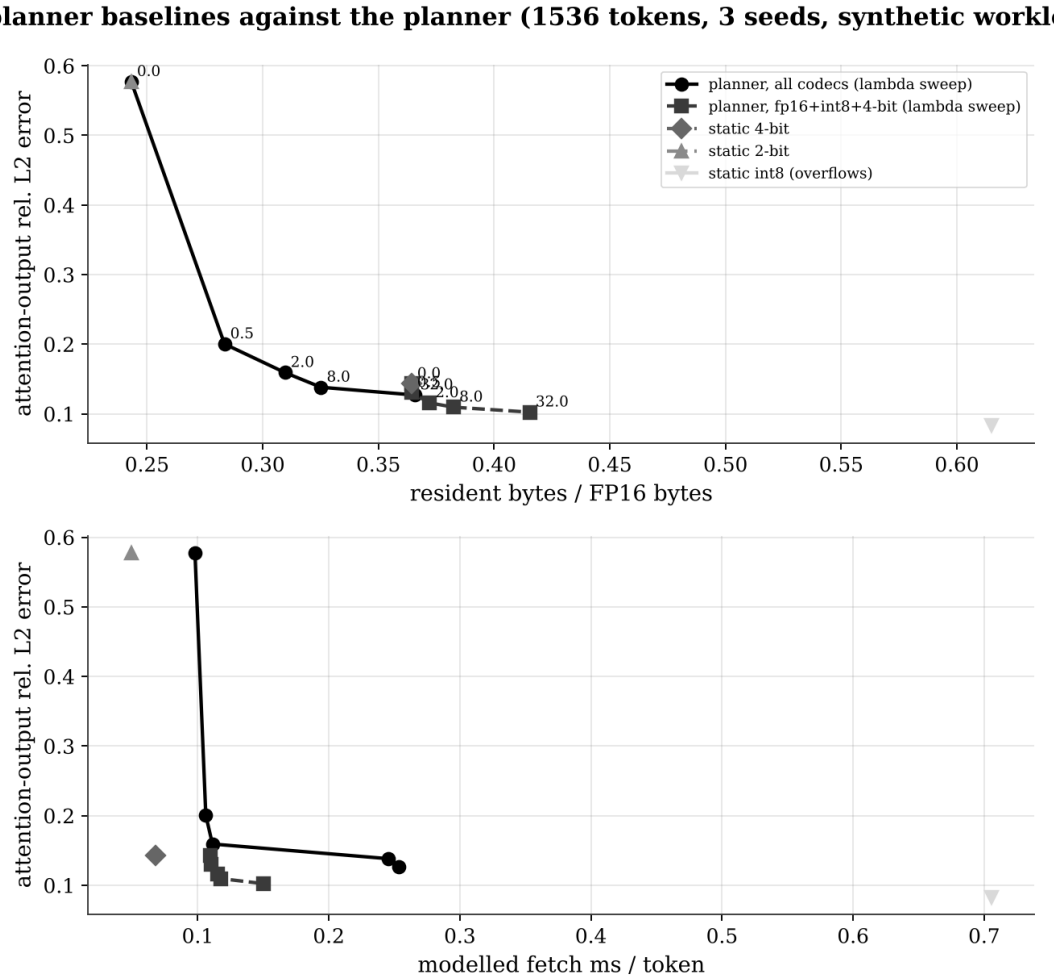


Figure 6.2: Error against resident size (left) and against modelled fetch time (right) for the no-planner baselines and for the planner over its λ sweep.

The static baseline places the hottest blocks in the fastest tier with room and stores every unprotected block in one codec. With 4-bit blocks everything fits mostly in DRAM, which is why its modelled fetch time is the lowest (0.068 ms). The planner with INT8 + KIVI-4 reaches lower error (0.116–0.131 for λ between 0.5 and 2, against 0.143) at the same residency, but fetches in about 0.11 ms, because the higher-precision blocks it keeps are larger and push other blocks to flash. No setting of λ in the sweep gives both the speed of the static baseline and the lower error. With all codecs enabled the planner can go below the static baseline in memory (0.284–0.310 against 0.364) at higher error. The honest reading is that the planner offers a controllable trade-off between memory, error and fetch time, whereas the static policy is one fixed, simple and fast point on it, and that the planner is not faster than that point in this model.

6.4.1.2 Charging the key summaries to the budget (E9)

Table 6.9: Effect of charging the per-block key min/max summaries to the 168 KB budget (1,536 tokens, 5 seeds).

Variant	Resident incl. summaries / FP16	Overflow, summaries free (B)	Overflow, summaries charged (B)	Dropped	Rel. L2 error	Fetch ms
UAMM-1 threshold policy	0.412 ± 0.012	0 ± 0	0 ± 0	1069 ± 16	0.618 ± 0.037	1.049 ± 0.021
No planner: every block 4-bit	0.489 ± 0.000	0 ± 0	7,187 ± 0	0 ± 0	0.152 ± 0.015	0.095 ± 0.005
Planner, INT8 + KIVI-4	0.496 ± 0.003	0 ± 0	3,438 ± 1,007	0 ± 0	0.127 ± 0.013	0.128 ± 0.004
Planner, all codecs	0.435 ± 0.001	0 ± 0	0 ± 0	0 ± 0	0.177 ± 0.018	0.124 ± 0.004

The selection step keeps, for every block, a minimum and a maximum key vector in FP16: $2 \times 64 \times 2 = 256$ B per block, 24,576 B for 96 blocks, which is 14 % of the 168 KB budget. The main experiments do not charge this memory. Experiment E9 charges it by shrinking the tier capacities by the same amount (to 0.857 of their size). Under that accounting the INT8 + KIVI-4 planner overflows by about 3.4 KB and the all-4-bit baseline by about 7.2 KB, so the statement that these variants fit the budget holds only if the summaries are free. The all-codec planner still fits, with error 0.177 and fetch time 0.124 ms. Including the summaries, its resident size is 0.435 of FP16, not 0.310. The summaries are also read at every step to rank blocks, a cost that the fetch model does not include.

6.4.2 Scaling with context length (E2)

Table 6.10: Context scaling with the 168 KB budget fixed (mean of 3 seeds). UAMM-2 columns use all codecs unless stated. (columns 1 to 5)

Tokens	Err UAMM-1	Err UAMM-2 (all)	Recall UAMM-1	Recall UAMM-2	Dropped (UAMM-1)
512	0.001	0.016	0.999	0.999	0
768	0.100	0.042	0.954	0.990	245
1024	0.241	0.071	0.881	0.976	496
1536	0.505	0.159	0.750	0.945	971
2048	0.683	0.270	0.657	0.913	1461
3072	0.985	0.515	0.532	0.830	2469

Table 6.11: Context scaling with the 168 KB budget fixed (mean of 3 seeds). UAMM-2 columns use all codecs unless stated. (columns 6 to 9)

Tokens	Resident/FP16 (UAMM-2)	Fetch ms UAMM-1	Fetch ms UAMM-2	Overflow B: INT8+K4 / all
512	0.538	0.473	0.035	0 / 0
768	0.445	0.728	0.049	0 / 0
1024	0.376	0.846	0.072	0 / 0
1536	0.310	0.919	0.111	0 / 0
2048	0.266	0.948	0.152	25,899 / 0
3072	0.241	0.946	0.178	132,309 / 28,501

- **Short contexts.** At 512 tokens the cache fits at FP16 (UAMM-1 resident ratio 1.0, error 0.0013). UAMM-2 still compresses (resident 0.538) and is 14× faster in the model (0.035 ms vs 0.473 ms) but incurs a small error (0.016). That is a policy consequence of the cost function: when latency is priced and the quality term is

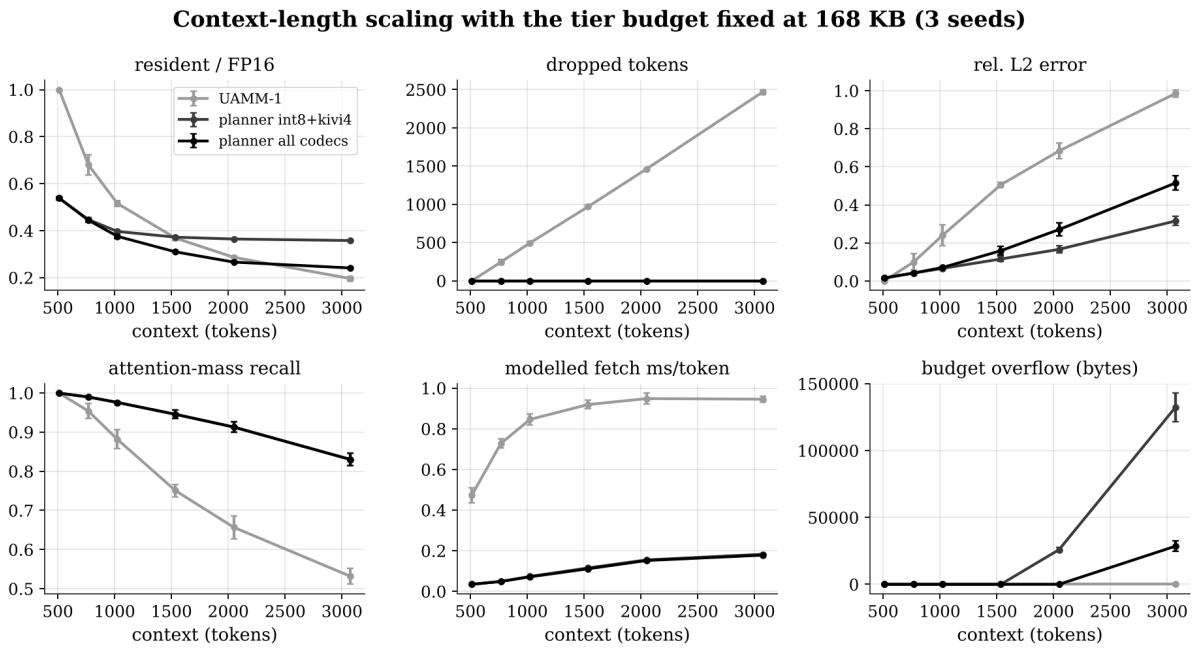


Figure 6.3: Context scaling at a fixed budget. UAMM-1's error and drops grow quickly beyond 768 tokens; UAMM-2 degrades gradually and keeps every token until the budget is exceeded near 3,072 tokens.

small, the planner lowers precision even without capacity pressure. If the application prefers exactness when memory is available, λ should be raised or compression gated on pressure (Chapter 7).

- **Medium contexts.** At 1,024 tokens UAMM-1 has dropped 496 tokens (error 0.241, recall 0.882); UAMM-2 holds all tokens with error 0.071 and recall 0.976.
- **Long contexts.** At 2,048 tokens UAMM-2 (all codecs) still fits (resident 0.266, error 0.270, recall 0.913). At 3,072 tokens it overflows its budget by 28.5 KB (INT8+KIVI-4 only: 132 KB) and error is 0.515 vs 0.985 for UAMM-1, which has dropped 2,469 of 3,072 tokens. Beyond this point the fixed budget is simply too small for the data even at 2-bit; opt-in drop or a larger budget would be needed.
- **Recall.** Recall falls with context in both systems but for different reasons: in UAMM-1 because tokens are missing, in UAMM-2 because the fixed block budget of 24 covers a shrinking fraction of the history (see E3).

6.4.3 How many blocks must be read? (E3)

Table 6.12: Effect of the attention block budget at 1,536 tokens (mean ± std, 3 seeds).

Block budget B	Mass recall	Rel. L2 error	Fetch ms / token	Resident / FP16
4	0.568 ± 0.013	1.101 ± 0.016	0.036 ± 0.003	0.269 ± 0.001
8	0.744 ± 0.017	0.663 ± 0.015	0.062 ± 0.002	0.272 ± 0.003
12	0.841 ± 0.019	0.443 ± 0.027	0.087 ± 0.001	0.277 ± 0.001
16	0.895 ± 0.017	0.308 ± 0.022	0.098 ± 0.003	0.285 ± 0.002
24	0.945 ± 0.011	0.159 ± 0.023	0.111 ± 0.002	0.310 ± 0.001
32	0.966 ± 0.007	0.101 ± 0.008	0.132 ± 0.004	0.337 ± 0.004
48	0.986 ± 0.003	0.070 ± 0.002	0.278 ± 0.024	0.376 ± 0.002

Selecting 24 blocks (about a quarter of the roughly 96 blocks at 1,536 tokens) retains 94.5 % of the attention mass with an error of 0.159; 48 blocks reach 98.6 % at a cost that is 2.5× higher (0.278 ms vs 0.111 ms) because more blocks are fetched from slower tiers. The marginal return of a block falls quickly, as the sorted block mass predicts. Note the interaction with residency: a larger budget reads many cold blocks, which pushes the planner to keep more of them in higher precision (resident 0.376 at B=48 vs 0.310 at B=24).

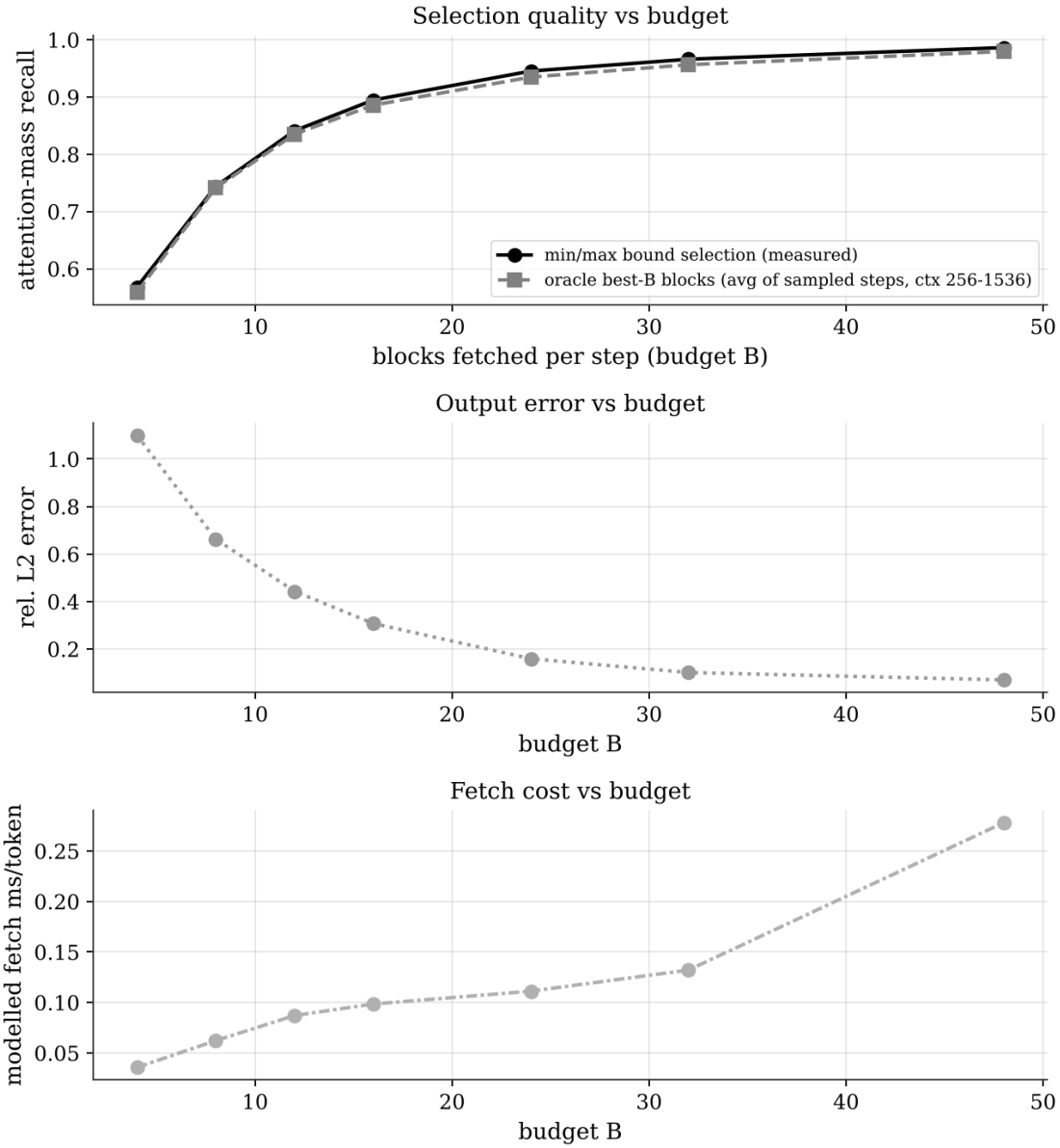


Figure 6.4: Block budget sweep. Left: recall of the min/max-bound selector versus the oracle choice of the B heaviest blocks averaged over sampled steps (indicative; the system also reads the protected blocks). Middle and right: error and fetch cost.

As λ rises the planner protects quality: error falls from 0.577 at $\lambda = 0$ (maximum compression, resident 0.243) to 0.127 at $\lambda = 32$, at the price of larger residency (0.366). Latency is not monotone: up to $\lambda = 2$ fetch time is nearly flat at about 0.10–0.11 ms, then jumps to 0.20–0.25 ms at $\lambda \geq 4$ and flash reads triple. The cause, visible in the planner's cost, is that choosing larger encodings for a block can push other blocks out of DRAM into flash, whose fixed 100 µs access cost dwarfs the transfer savings. The default $\lambda = 2$ sits at the knee: the error is within 25 % of the best seen (0.159 vs 0.127) while fetch time is roughly half that of $\lambda \geq 4$ (0.111 ms against 0.20–0.25 ms).

6.4.4.2 Hysteresis, residency and replanning interval

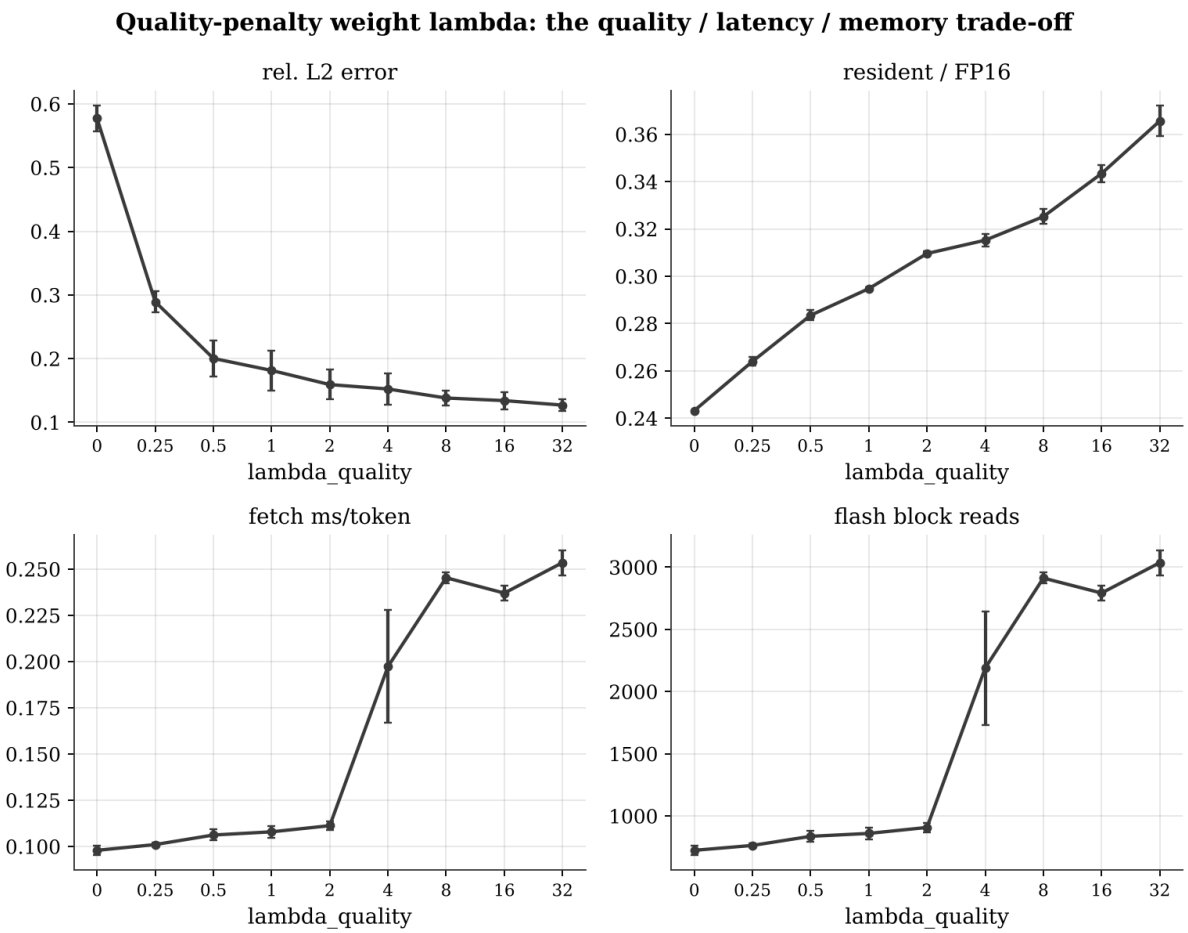
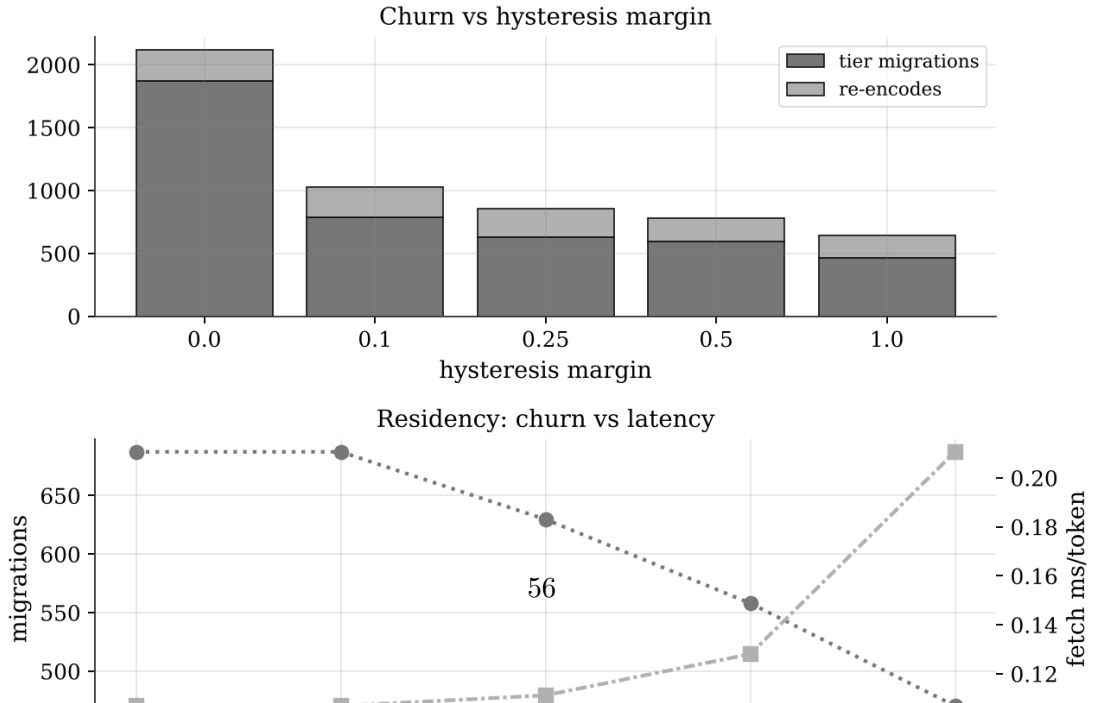


Figure 6.5: Effect of λ on error, memory, fetch latency and flash reads.

Table 6.14: Hysteresis margin (3 seeds).

Hysteresis margin	Migrations	Re-encodes	Resident / FP16	Rel. L2 error	Fetch ms / token
0.0	1871 ± 45	247 ± 3	0.286 ± 0.004	0.208 ± 0.030	0.112 ± 0.002
0.1	784 ± 27	240 ± 3	0.292 ± 0.004	0.181 ± 0.027	0.111 ± 0.004
0.25	630 ± 15	226 ± 2	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002
0.5	593 ± 14	188 ± 4	0.359 ± 0.006	0.111 ± 0.013	0.116 ± 0.003
1.0	463 ± 26	180 ± 3	0.373 ± 0.005	0.110 ± 0.013	0.115 ± 0.003



bias.

The minimum-residency rule has no effect below 16 steps because the planning interval is 8 (identical rows for 0 and 8): residency is only meaningful when it exceeds the interval. At 64 steps migrations fall to 470, but fetch time doubles (0.211 ms) and flash reads reach 2,384, because blocks that have become hot cannot be promoted in time. Likewise the replanning interval: planning every 2 steps gives 599 flash reads and 0.090 ms but 768 migrations, whereas every 64 steps gives 3,326 flash reads and 0.272 ms with only 345 migrations. The planner must keep up with the changes in attention focus; the cost of keeping up is the planning work itself, which is why the hardware sweep scorer exists.

6.4.4.3 H2O decay, aggregation and block size

Table 6.15: H2O decay (3 seeds).

H2O decay	Resident / FP16	Rel. L2 error	Fetch ms / token	Re-encodes
0.5	0.285 ± 0.003	0.202 ± 0.018	0.112 ± 0.002	244 ± 3
0.8	0.285 ± 0.003	0.202 ± 0.018	0.111 ± 0.002	244 ± 2
0.9	0.286 ± 0.003	0.202 ± 0.018	0.112 ± 0.002	244 ± 2
0.95	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	226 ± 2
0.98	0.329 ± 0.005	0.135 ± 0.010	0.108 ± 0.002	210 ± 1
0.995	0.368 ± 0.004	0.114 ± 0.009	0.110 ± 0.002	173 ± 2

A longer memory of attention (decay 0.995) makes the planner more conservative about compressing blocks that were important at some point: error falls from 0.202 (decay $\leq$ 0.9) to 0.114 and residency rises from 0.285 to 0.368, at essentially constant fetch time (0.110–0.112 ms). Decay therefore moves the system along the same memory-versus-quality axis as λ but at no latency cost in this configuration. Mean and sum aggregation are equivalent (sum is normalised by the same percentile; blocks have equal size); max aggregation gives slightly lower residency (0.301) and similar error (0.162).

Table 6.16: Block size (3 seeds; block budget scaled to keep ≈384 non-protected tokens fetched).

Block size (tokens)	Resident / FP16	Rel. L2 error	Fetch ms / token	Migrations	Overflow (B)
8	0.387 ± 0.008	0.110 ± 0.014	0.216 ± 0.009	1652 ± 86	0 ± 0
16	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	630 ± 15	0 ± 0
32	0.264 ± 0.003	0.219 ± 0.036	0.063 ± 0.002	275 ± 19	0 ± 0
64	0.264 ± 0.013	0.341 ± 0.010	0.038 ± 0.002	121 ± 6	14336 ± 0

Block size is a three-way trade-off. Small blocks (8 tokens) isolate hot tokens well, giving the lowest error (0.110), but have more metadata, more migrations (1,652) and much higher fetch time (0.216 ms, since each block pays a fixed latency). Large blocks (64) cut migrations to 121 and fetch time to 0.038 ms but raise error to 0.341 and overflow the budget by 14 KB: a block must be placed and compressed as a unit, so a few hot tokens drag a whole large block into a fast tier. A block of 16 tokens is a reasonable compromise and also fits the hardware tables.

6.4.5 Memory budget (E5)

Table 6.17: Memory-budget sweep at 1,536 tokens (mean of 3 seeds).

Budget	UAMM-1 resident	UAMM-1 dropped	UAMM-1 error	UAMM-1 fetch ms	UAMM-2 resident	UAMM-2 error	UAMM-2 fetch ms	UAMM-2 overflow B
×0.5 (84 KB)	0.219	1200	0.855	0.757	0.288	0.220	0.301	33,109
×0.75 (126 KB)	0.299	1077	0.661	1.001	0.293	0.180	0.141	0
×1.0 (168 KB)	0.368	971	0.505	0.919	0.310	0.159	0.111	0
×1.5 (252 KB)	0.521	736	0.280	0.733	0.323	0.131	0.099	0
×2.0 (336 KB)	0.625	576	0.171	0.612	0.330	0.115	0.088	0
×3.0 (504 KB)	0.830	261	0.086	0.502	0.324	0.152	0.049	0

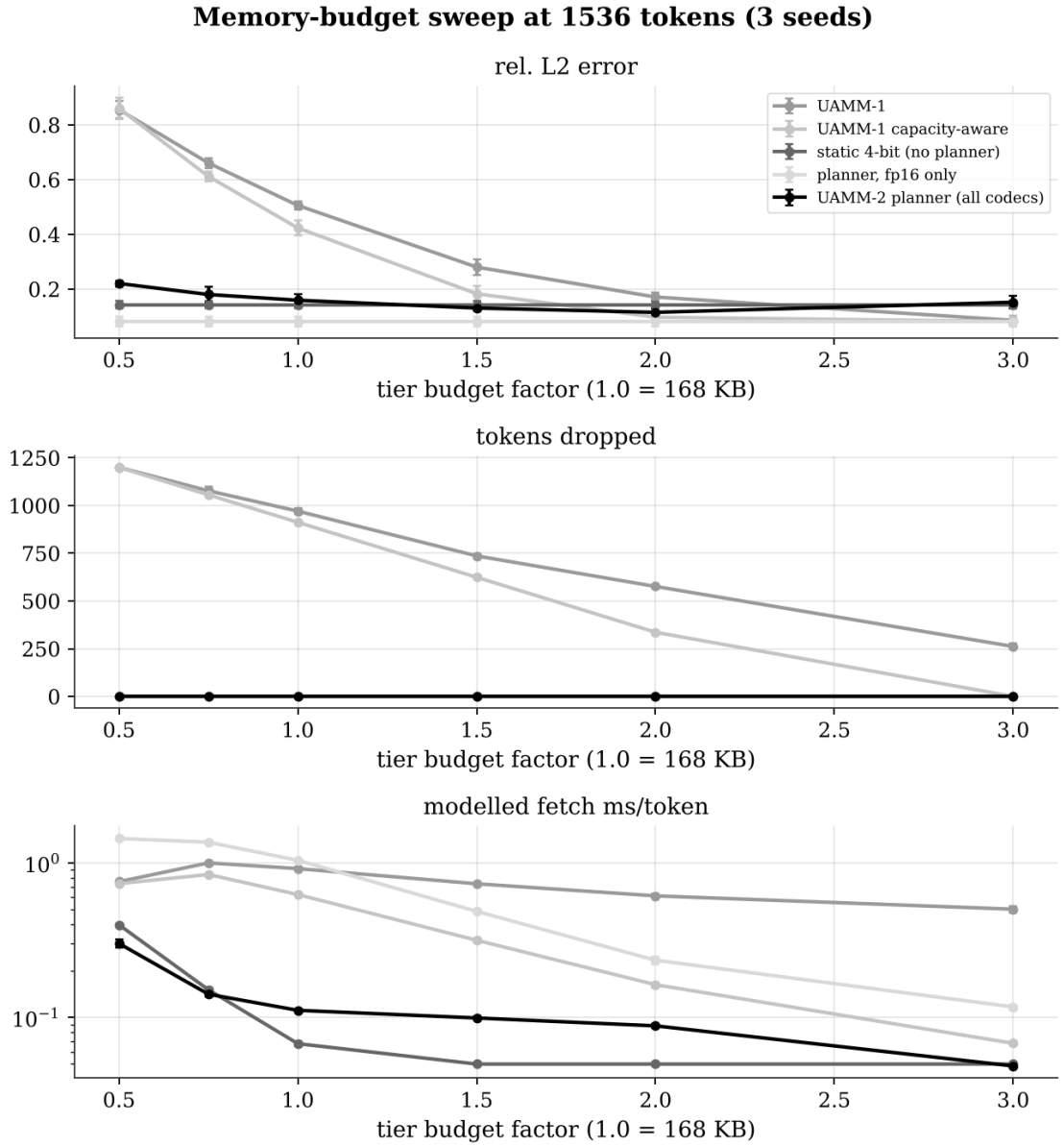


Figure 6.7: Budget sweep. UAMM-1 needs roughly three times the baseline budget to bring error below 0.09 and still drops 261 tokens; UAMM-2 reaches comparable error without drops at much smaller budgets.

- **Equal-quality comparison with UAMM-1.** UAMM-2 at ×0.75 of the budget (126 KB) has error 0.180 and drops nothing; the original UAMM-1 needs ×2.0 (336 KB) to reach a similar error (0.171) and still drops 576 tokens. This comparison is against the original threshold rule only; the fairer baselines below narrow it.

- **Infeasibility.** At ×0.5 (84 KB) UAMM-2 overflows by 33 KB (the data do not fit even at its most compressed); UAMM-1 does not report overflow but drops 1,200 tokens and has error 0.855.
- **Non-monotonic quality.** UAMM-2's error is 0.115 at ×2.0 but 0.152 at ×3.0, even though more memory is available. At ×3.0 flash is almost unused (3 reads) and the planner spends the headroom on speed (fetch 0.049 ms vs 0.088 ms) by choosing smaller encodings. This is the cost function working as designed, but it is a reminder that the optimiser, not the budget, decides the trade-off, and that a quality floor (Chapter 7) is needed to guarantee monotonic behaviour.

Fairer baselines across budgets. The same sweep was repeated for three baselines: the capacity-aware UAMM-1, the FP16-only planner and the planner-free 4-bit policy.

Table 6.18: Baselines across memory budgets at 1,536 tokens (3 seeds).

Budget	UAMM-1 cap.-aware: dropped	error	All 4-bit: overflow B	error	FP16 planner: overflow B	error	UAMM-2: overflow B	error
×0.5	1200 ± 0	0.860 ± 0.040	67,925 ± 2,465	0.143 ± 0.013	331,776 ± 0	0.081 ± 0.018	33,109 ± 3,205	0.220 ± 0.010
×0.75	1056 ± 0	0.611 ± 0.019	24,064 ± 0	0.143 ± 0.013	298,325 ± 1,931	0.081 ± 0.018	0 ± 0	0.180 ± 0.028
×1.0	912 ± 0	0.423 ± 0.028	0 ± 0	0.143 ± 0.013	256,683 ± 965	0.081 ± 0.018	0 ± 0	0.159 ± 0.023
×1.5	624 ± 0	0.183 ± 0.029	0 ± 0	0.143 ± 0.013	192,512 ± 0	0.081 ± 0.018	0 ± 0	0.131 ± 0.010
×2.0	336 ± 0	0.097 ± 0.019	0 ± 0	0.143 ± 0.013	124,245 ± 2,554	0.081 ± 0.018	0 ± 0	0.115 ± 0.016
×3.0	0 ± 0	0.081 ± 0.018	0 ± 0	0.143 ± 0.013	0 ± 0	0.081 ± 0.018	0 ± 0	0.152 ± 0.023

- **Generous budgets remove the advantage.** FP16 for all 1,536 tokens needs 393 KB, which is 2.3× the baseline budget. At ×3.0 (504 KB) the FP16-only planner fits with zero overflow and the lowest error (0.081), and the capacity-aware UAMM-1 keeps every token at FP16 with error 0.081 and 0.068 ms, which beats UAMM-2 at that budget (error 0.152). The large gain of UAMM-2 over UAMM-1 in the main experiment depends on a budget that is 2.3× smaller than the data.
- **The original UAMM-1 deletes even when memory is ample.** At ×3.0 it still drops 261 tokens, because a block may only go to its target tier or a slower one. This is a defect of the threshold rule, not of capacity.
- **Tighter budgets favour the planner.** At ×0.75 the all-4-bit policy overflows by 24 KB, whereas the planner fits (error 0.180) by using 2-bit for cold blocks. At ×0.5 neither fits (overflow 68 KB against 33 KB). Above ×1.0 the 4-bit policy fits and is simpler and faster.

6.4.5.1 Which codecs should be enabled? (E5b)

Table 6.19: Codec-set study at 1,536 tokens (3 seeds).

Enabled codecs	Resident / FP16	Rel. L2 error	Fetch ms / token	Overflow (B)
FP16 only	1.000 ± 0.000	0.081 ± 0.018	1.040 ± 0.007	256,683 ± 965
FP16 + INT8	0.606 ± 0.000	0.083 ± 0.018	0.460 ± 0.014	104,704 ± 0
+ KIVI-4	0.372 ± 0.002	0.116 ± 0.013	0.115 ± 0.003	0 ± 0
+ KIVI-2 (all)	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	0 ± 0
FP16 + KIVI-4 only	0.364 ± 0.000	0.132 ± 0.013	0.112 ± 0.002	0 ± 0
FP16 + KIVI-2 only	0.346 ± 0.015	0.308 ± 0.018	0.475 ± 0.012	0 ± 0

- **Feasibility needs 4-bit.** FP16 alone (overflow 257 KB) and FP16+INT8 (overflow 105 KB) cannot fit the budget; a 4-bit codec is the first that does.
- **KIVI-4 alone is competitive.** FP16 + KIVI-4 reaches 0.364 resident at error 0.132 and fetch 0.112 ms, close to the full set on speed with lower error than the full set (0.159); the full set saves a further 15 % memory.
- **KIVI-2 alone is a poor choice.** With only FP16 and KIVI-2 available the planner reaches 0.346 resident, but error doubles (0.308) and fetch time is 4× worse (0.475 ms, about 6,200 flash reads). The reason is structural: the tier rules force warm and cold storage to hold a 2-bit block, and without intermediate precision the planner cannot place blocks that matter more at reasonable cost.
- **Entropy coding gave no benefit here.** Results with and without zlib variants are identical, which means the planner never preferred a `z` variant. On the synthetic data, zlib on already quantised random-like values saves almost nothing (Table of codec sizes in Section 5.3). Real KV tensors have more redundancy and the result may differ; this is an open question rather than a conclusion.

6.4.6 Codec micro-benchmarks (E7)

Table 6.20: Key quantisation axis: relative squared error of keys only (200 random 16 × 64 blocks per row).

Bits	Outlier channels	Per-channel keys	Per-token keys	Per-token / per-channel
2-bit	without	1.07e-01	2.02e-01	1.9×
2-bit	with	1.07e-01	5.81e-01	5.4×
4-bit	without	4.22e-03	8.04e-03	1.9×
4-bit	with	4.18e-03	2.17e-02	5.2×
8-bit	without	1.47e-05	2.79e-05	1.9×
8-bit	with	1.47e-05	7.52e-05	5.1×

Quantising keys per channel is always better, by a factor of 1.9 without outlier channels and 5.2–5.4× when four of the 64 channels are six times larger, which agrees with the design choice reported for KIVI. Error rises by roughly 25× from 4-bit to 2-bit for the same block (0.0052 → 0.130 in the codec table), which is why 2-bit is treated as the tier of last resort, although with the default settings the planner still assigns it to about half of the blocks (Section 6.4.1).

6.4.7 Final placement map

The final state has 96 blocks. Tier T0 holds only FP16 blocks (protected sink and recent window). T1 holds 28 blocks (mainly 4-bit), T2 holds 12 and the flash tier T3 holds 53, all at 4 or 2 bits. No block in T0 or T1 is FP16 beyond the protected set: at this budget the planner has chosen to spend bytes on keeping everything rather than on keeping any unprotected block uncompressed.

6.4.8 Summary of the simulation findings

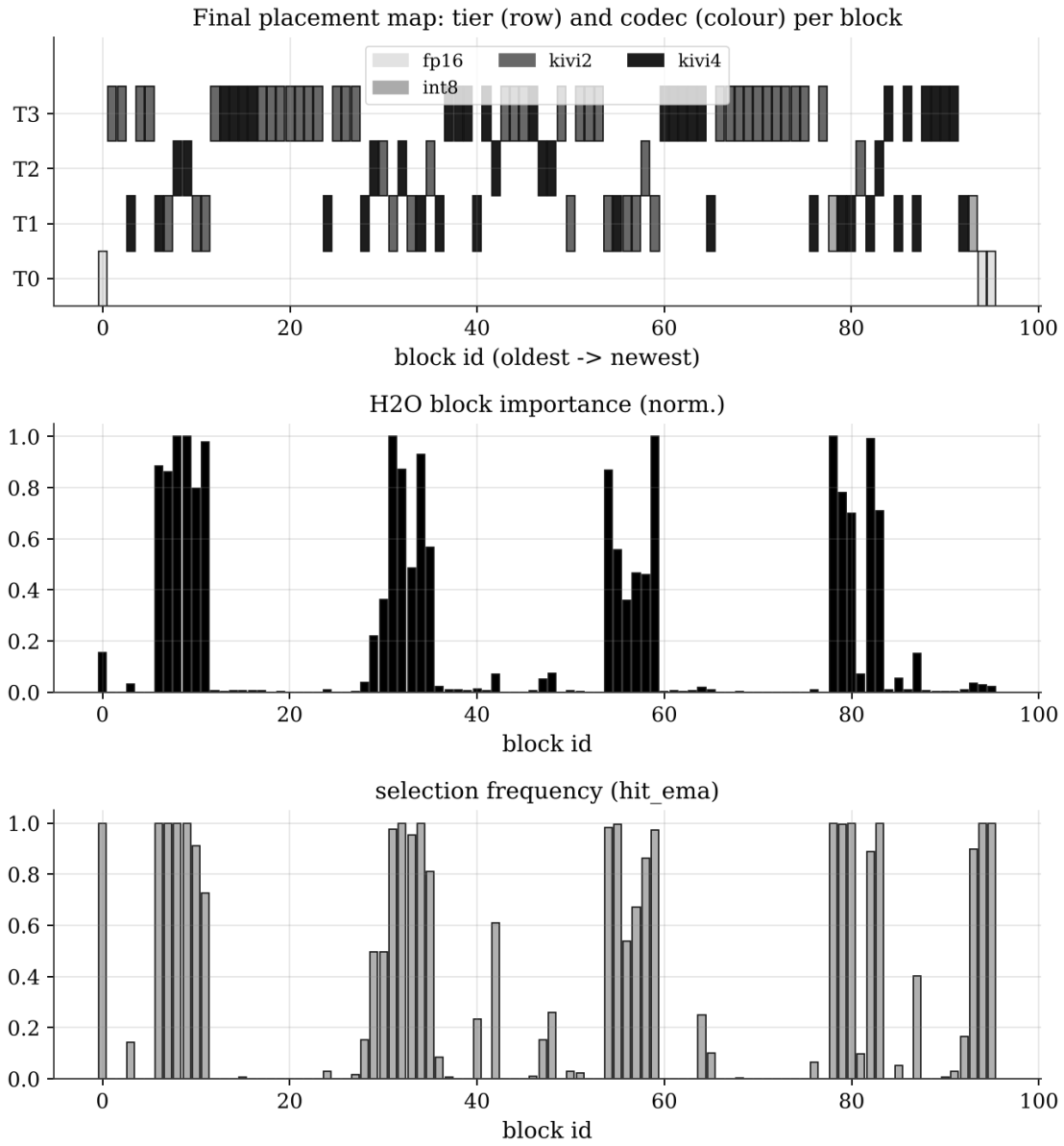


Figure 6.8: State of the cache at step 1,536: tier (row) and codec (colour) of every block, with block importance and selection frequency. Older, low-importance blocks occupy flash at 2- or 4-bit; recent blocks are in T0 at FP16.

Table 6.21: Key findings from the simulations.

Finding	**Evidence**
A joint (tier, codec) choice lets a tight budget hold every token without deleting	E1: 0 dropped tokens against 973; but a one-codec 4-bit baseline does this too (E1, E10)
The planner buys accuracy and robustness to tight budgets, not speed	E10: error 0.116–0.131 against 0.143 at equal memory, fetch 0.11 against 0.068 ms; E5: all-4-bit overflows at $\times 0.75$, planner does not
The size of the advantage over UAMM-1 depends on how tight the budget is	E5: at $\times 3.0$ FP16 alone fits with error 0.081
Counting the key summaries changes which variants fit	E9: INT8 + KIVI-4 and all-4-bit overflow; all-codec planner fits (0.435 resident)
Quality is bounded and tunable	E4: λ and decay move error between 0.11 and 0.58 along a memory axis
Selective attention delivers most of the saving	E3: 24 of ~96 blocks gives 94.5 % mass recall
Hysteresis removes most churn	E4: 1,871 $\rightarrow$ 630 migrations with the default margin
Replanning cadence matters for flash traffic	E4: 599 $\rightarrow$ 3,326 flash reads from interval 2 $\rightarrow$ 64
Budget exhaustion is graceful, not catastrophic	E2: gradual error growth; overflow reported, not hidden

Finding	Evidence
Some behaviour is non-obvious and needs a floor or a policy	E5: non-monotonic error at 3× memory; E2: compression when memory is ample

6.5 Hardware Results: RTL Simulation, Waveforms and Verification

This chapter presents the results for the three Verilog modules: how they were verified, the waveforms recorded during the testbench runs, the lint and mutation results and the synthesis statistics. Every figure showing signals was rendered directly from the value-change-dump (VCD) files written by the Verilator simulations (`docs/wave/trace_*.vcd`).

How to read the waveform figures

The figures are screenshots of the GTKWave waveform viewer (GTK 3 build, run in a virtual X display). Each opens one of the VCD dumps written by Verilator from the testbenches in `hw/tb` (rerun with `+trace`), with the signals, radix, zoom range and marker placed by a Tcl script kept in `docs/gtkwave/`. Buses are shown in hexadecimal or decimal as indicated in the signal values; the vertical line is the marker.

Scope of the hardware

The sweep scorer implements the weighted score of the original UAMM-1 (importance, recency and frequency with the weights 154/64/38 and two thresholds). It does not implement the planner's utility cost, so the hardware and the UAMM-2 software policy are not yet the same function; Scope 13 in Chapter 7 describes the change.
The codec module quantises 8-bit group values with a reciprocal supplied by firmware. The software codecs start from FP16, so a front stage that produces the 8-bit values is still missing.
All checks are simulations in Verilator against Python golden models. There is no place-and-route result, no timing and no power figure.

6.5.1 Verification method

1. **Golden models.** `uamm2/hwmodel.py` re-implements each module's integer arithmetic line for line.
2. **Vector generation.** The same file writes stimulus and expected-output hex files (`hw/vectors/*.hex`), including directed boundary cases and random cases.
3. **Self-checking testbenches.** Each testbench reads the vectors, drives the module, compares every output with the expectation and prints `PASS` or `FAIL` with a count.
4. **Verilator 5.** `make -C hw test` builds and runs all three benches with timing support; `+trace` additionally dumps a VCD.
5. **Lint.** `verilator -lint-only -Wall` on every module.
6. **Mutation testing.** Twelve single-fault variants of the RTL are generated automatically; the regression must fail for every one.

7. **Logic synthesis check.** Yosys `synth` with `check -assert` confirms the designs elaborate to generic gates with no inferred latches.

6.5.2 Sweep scorer

The scorer test applies four metadata datasets of 64 blocks, each swept at four pressure levels (0, 242, 243, 255), which gives 16 sweeps; a further sweep exercises restart behaviour (17 in total). Dataset 0 is a set of directed boundary cases: for each of the scores 0, 1, 88, 89, 90, 173, 174, 175, 254 and 255 a metadata word is generated whose weighted score equals that value, once unprotected and once protected, followed by corner cases (all zeros, all 255, single maxima) and random fill. The other three datasets are random with a 20 % chance of the `protected` bit.

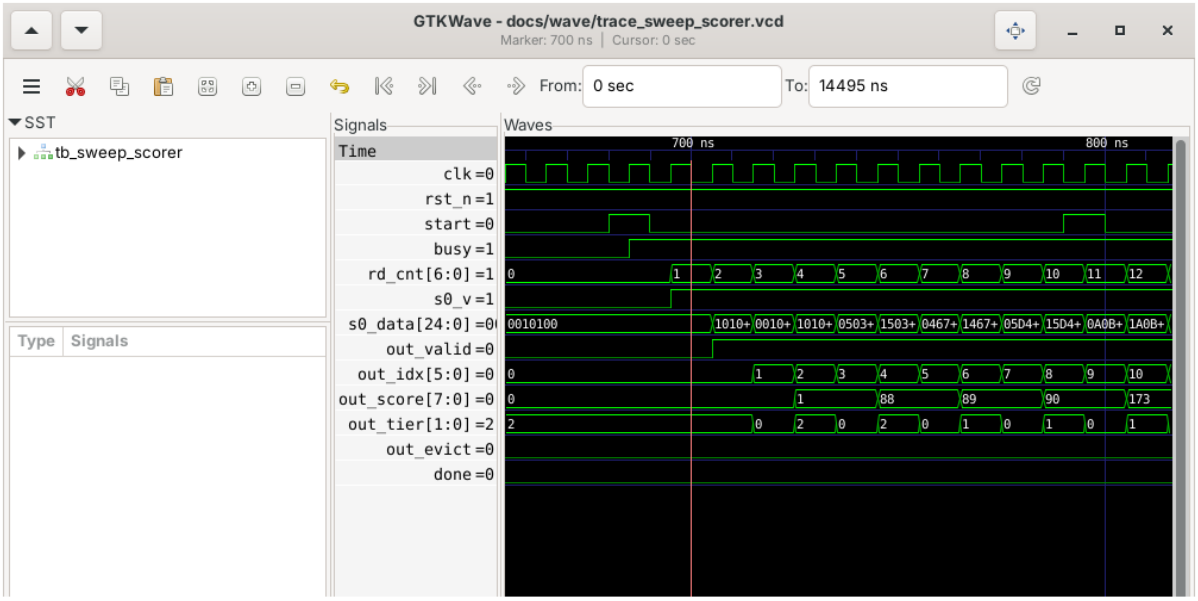


Figure 6.9: Sweep start. start is asserted for one clock; busy rises on the next edge; the read counter then increments once per cycle. s0_data shows the metadata words read from the SRAM; the results appear two cycles later as out_valid with the block index, score and tier.

The first results are the directed boundary cases, which makes the figure self-explanatory: block 0 has score 0 and tier 2 (cold) because it is unprotected; block 1 has the same metadata with `protected` set and is forced to tier 0; block 2 has score 1; blocks 4 and 5 have score 88 (just below the warm threshold of 89, cold when unprotected); blocks 6 and 7 have score 89 (exactly at the warm threshold: warm when unprotected, hot when protected); blocks 8 and 9 have score 90; block 10 has score 173 (one below the hot threshold: warm). The alternation of tier values 2, 0, 2, 0, 2, 0, 1, 0, 1, 0, 1 is the pair-wise unprotected/protected pattern.

This is the behaviour the UAMM-1 RTL got wrong (it ignored `protected`) and the main reason for the boundary vectors: at pressure 242 no block is flagged for eviction; at 243 exactly the unprotected blocks whose score is below 89 are flagged.

The latency is as designed: for NBLK = 64 a sweep produces its last result NBLK + 3 cycles after `start`, and it sustains one block per cycle. At a 100 MHz clock a 64-block sweep takes under 0.7 µs; for 4,096 blocks it would take under 41 µs, so the sweep is cheap

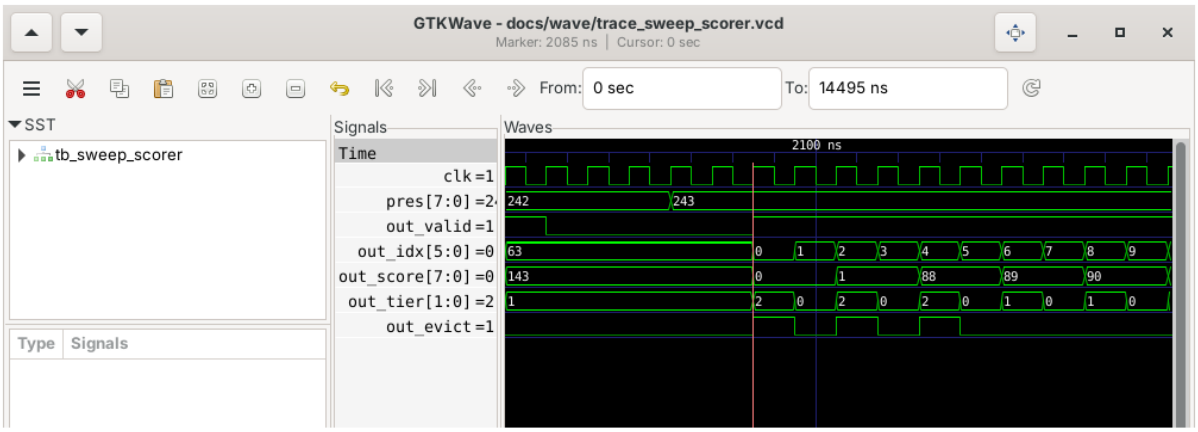


Figure 6.10: Eviction under pressure. The pressure latched for the sweep changes from 242 to 243; in the sweep at 243, out_evict is raised for blocks 0, 2 and 4 (unprotected, score below 89) and not for the protected blocks or for blocks 6 and 7 (score 89).

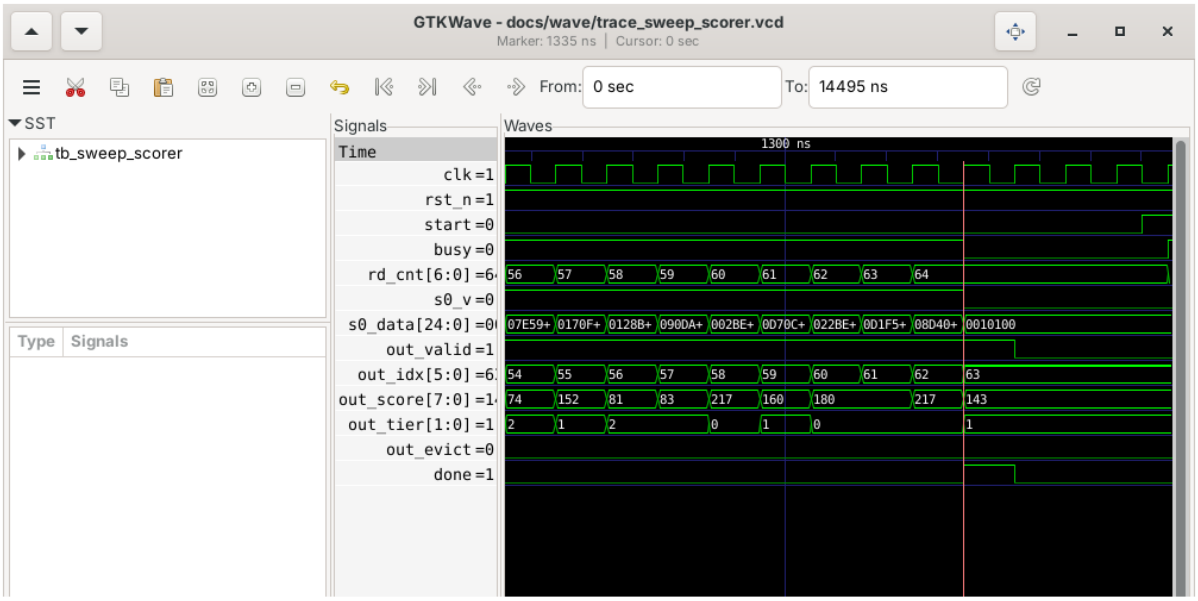


Figure 6.11: End of a sweep. The read counter reaches 64 and busy falls; the last result (block 63) is emitted one cycle later together with the done pulse. A new start can follow immediately.

compared with the planning interval of several decode steps. (These are cycle counts from the simulation multiplied by an assumed clock, not timing-closure results.)

Table 6.22: Sweep scorer results.

Check	Count	Result
Sweeps (4 datasets × 4 pressures + restart)	17	PASS
Total per-block comparisons (score, tier, evict, index, valid)	1,088	PASS
`start` while busy ignored	directed	PASS
Reset in the middle of a sweep	directed	PASS

6.5.3 Codec pipeline

The quantiser and dequantiser testbench feeds 400 vectors of 16 lanes. The vectors are produced in four modes in rotation: full-range random, narrow range around a random

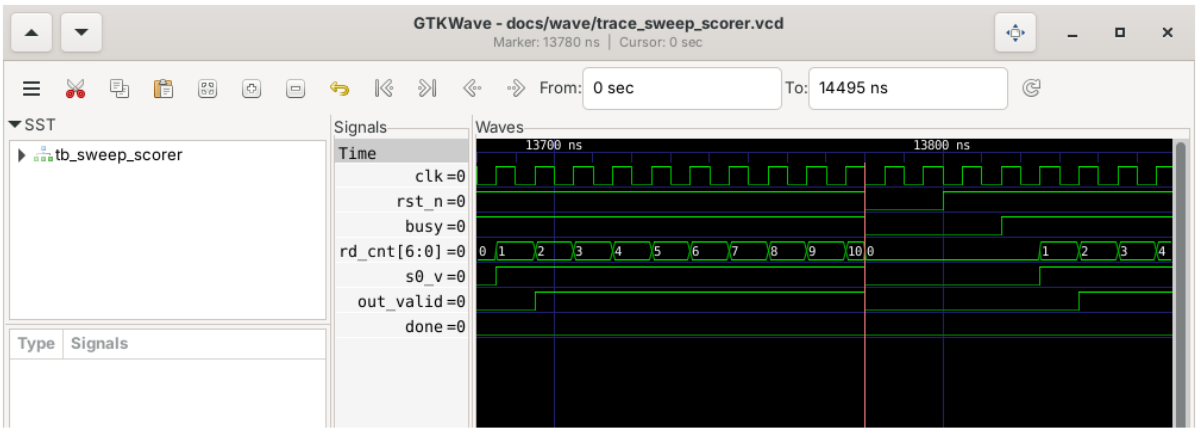


Figure 6.12: Asynchronous reset in the middle of a sweep. rst_n is pulled low; busy, s0_v and out_valid drop immediately, the read counter returns to zero and a later start begins a clean sweep.

centre, constant group (range zero) and extremes (0, 1, 254, 255). Further variations set the zero point inside the data or deliberately **below** the true minimum (stale statistics), which exercises the clamp at code 15. The testbench checks both the 16 packed 4-bit codes and the 16 dequantised bytes.

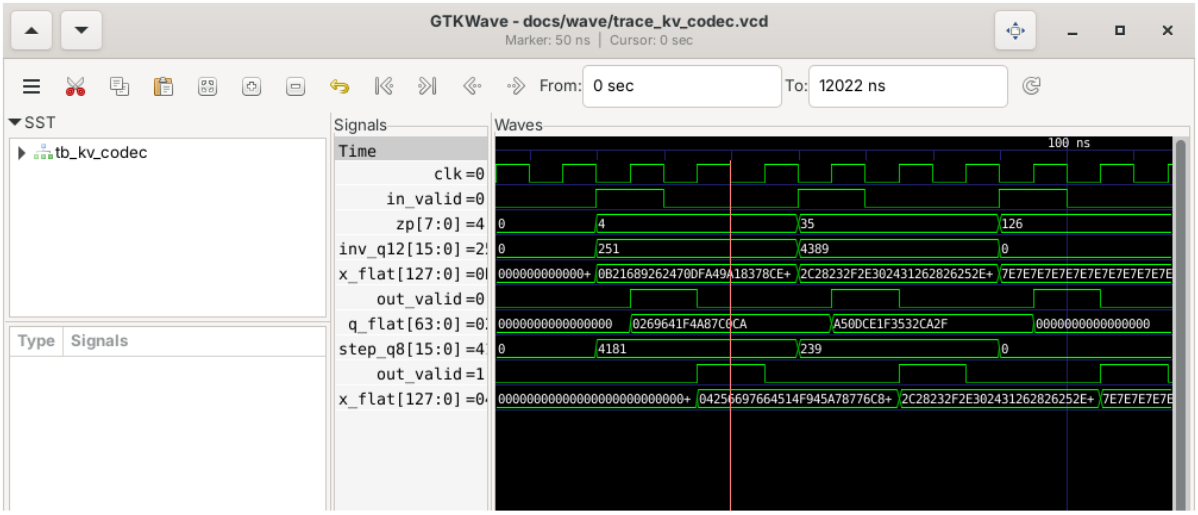


Figure 6.13: Two vectors and a constant group through uamm_kv_quant4 and uamm_kv_dequant4. x_flat is the 128-bit input; q_flat is the packed result one cycle after in_valid; the dequantiser is fed the code vector and step and produces x' one cycle later.

The third vector in the figure is the constant group (`inv_q12 = 0`, `step_q8 = 0`): all lanes are `7e`, every code is zero and the dequantiser reproduces `7e` exactly, as the model predicts. The first two vectors have non-trivial ranges (parameters 251 and 4,389 for inv_q12), show the one-cycle latency of each stage, and match the expected nibble words.

Table 6.23: Codec results.

Check	Count	Result
Codec vectors (quantiser and dequantiser outputs)	400	PASS

6.5.4 Block table

The block-table testbench performs 6,000 random operations (writes, lookups, and the occasional same-cycle read-and-write to one address) and compares every lookup with a behavioural reference table. It also checks that the valid bits are cleared by reset and that a read in the same cycle as a write returns the old entry.

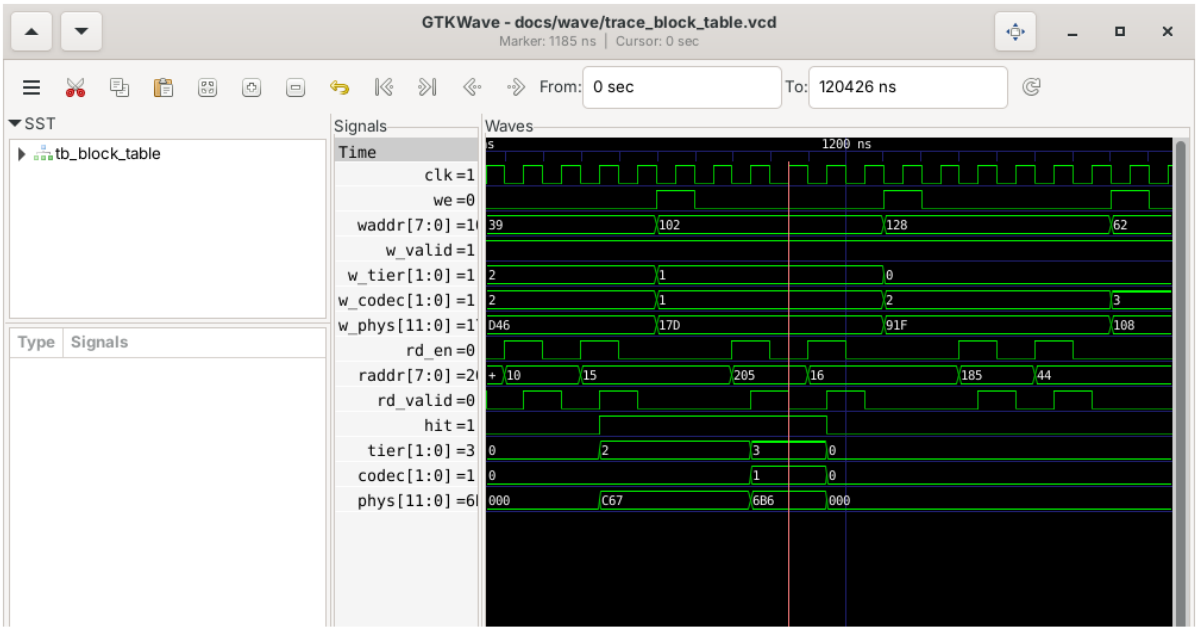


Figure 6.14: Random updates and lookups. we pulses write {tier, codec, phys} to the address on waddr; rd_en pulses start a lookup whose result (hit, tier, codec, phys) appears one cycle later through rd_valid. A lookup of a never-written address returns hit = 0; a hit returns the stored fields.

Table 6.24: Block-table results.

Check	Count	Result
Writes	3,049	PASS
Reads	2,971	PASS
Reads that hit	2,313	PASS

6.5.5 Regression, lint and mutation results

```
$ make -C hw test

/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS "-std=c++20 -fcoroutines" --top-module
TB_SWEEP_SCORER PASS checks=1088
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS "-std=c++20 -fcoroutines" --top-module
TB_KV_CODEC PASS vectors=400
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS "-std=c++20 -fcoroutines" --top-module
TB_BLOCK_TABLE PASS writes=3049 reads=2971 hits=2313
```

Figure 6.15: Output of make -C hw test: all three testbenches print PASS with their check counts.

Mutation testing measures the strength of the testbenches rather than the design: a testbench that cannot detect an injected fault is not protecting anything. Twelve mutants

```
$ make -C hw lint   (verilator --lint-only -Wall)

/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_sweep_scorer rtl/uamm_sweep_scor
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.035 MB sources in 2 modules, into 0.021 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.013 s (elab=0.001, cvt=0.008, bld=0.000); cpu 0.009 s on 1 threads; allocated 20.539 MB
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_kv_quant4 rtl/uamm_kv_codec.v
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.034 MB sources in 3 modules, into 0.045 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.027 s (elab=0.001, cvt=0.011, bld=0.000); cpu 0.013 s on 1 threads; allocated 20.680 MB
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_kv_dequant4 rtl/uamm_kv_codec.v
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.034 MB sources in 3 modules, into 0.037 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.008 s (elab=0.001, cvt=0.004, bld=0.000); cpu 0.008 s on 1 threads; allocated 20.609 MB
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_block_table rtl/uamm_block_table
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.033 MB sources in 2 modules, into 0.012 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.006 s (elab=0.001, cvt=0.002, bld=0.000); cpu 0.006 s on 1 threads; allocated 20.418 MB
lint exit=0
```

Figure 6.16: Output of make -C hw lint (verilator –lint-only -Wall on every module). Exit status 0, no warnings.

were generated, each a single small change to the RTL source, and the full regression was run on each.

```
$ RTL mutation testing (injected bugs must make a testbench FAIL)

mutation                          result
W_IMP 154 -> 153                  KILLED (testbench FAIL)
W_FRQ 38 -> 40                    KILLED (testbench FAIL)
TH_HOT 174 -> 175                 KILLED (testbench FAIL)
TH_WARM 89 -> 88                  KILLED (testbench FAIL)
EVICT_P 243 -> 244                KILLED (testbench FAIL)
tier ignores protected            KILLED (testbench FAIL)
evict ignores protected           KILLED (testbench FAIL)
quant: drop +2048 rounding        KILLED (testbench FAIL)
quant: remove clamp at 15         KILLED (testbench FAIL)
dequant: drop +128 rounding       KILLED (testbench FAIL)
dequant: remove saturation at 255 KILLED (testbench FAIL)
valid bit write disabled          KILLED (testbench FAIL)

12/12 mutants killed
```

Figure 6.17: Mutation results: 12 of 12 injected faults are detected.

The mutants cover weights, thresholds, rounding, clamping, saturation, the protected-block override and the valid-bit write. During development one mutant (removing the clamp at code 15) initially **survived** because no vector exercised an out-of-range code; the stale-zero-point vectors described in Section 6.5.3 were added to close that gap, and the mutant is now detected. This is the usual value of mutation testing: it found a hole in the tests that a pass count alone would not have shown.

6.5.6 Synthesis statistics

Table 6.25: Yosys `synth -flatten` generic-cell statistics (technology-independent). The raw reports are in the repository under `docs/captures`.

Module	Generic cells	Flip-flops	Combinational	Muxes	XOR/XNOR
Sweep scorer (64 blocks)	4,426	1,667	2,759	1,246	119
Quantiser (16 lanes)	11,292	65	11,227	4	3,252
Dequantiser (16 lanes)	5,241	129	5,112	0	1,364
Block table (256 entries)	12,796	4,370	8,426	3,155	8

- The **block table and the scorer metadata** appear as thousands of flip-flops because the memories are written as generic arrays and Yosys was run without a memory-mapping step. On an FPGA or ASIC flow they would be mapped to block RAM or SRAM macros. The flip-flop counts are therefore an upper bound that reflects storage, not control logic.
- The **quantiser is the largest logic block** (about 11,300 cells), dominated by the sixteen parallel 8 × 16-bit multipliers. A shift-based reciprocal (power-of-two ranges) or time-multiplexing fewer lanes would shrink it; this is noted as a design trade-off in Chapter 7.
- These are technology-independent counts. **No place-and-route, timing or power analysis has been done.**

6.6 The Ara Vector Kernel: Results

The vector-processor part of the system is a C kernel (`ara/kv_kernels.c`) that computes attention scores directly on 4-bit packed keys. It exists in two variants that share data layout and algorithm: a scalar path that executes on any host, and an RVV 1.0 path written with the standard vector intrinsics, compiled for a RISC-V vector target.

6.6.1 Algorithm in detail

For a block of T tokens and d channels with per-channel scale s_c and zero z_c, the key of token t is reconstructed as k_{t,c} = Q_{t,c} · s_c + z_c. The attention logit is therefore:

$$q \cdot k_t = \Sigma_c\ q_c\ (Q_\{t,c\}\ s_c + z_c) = \Sigma_c\ (q_c\ s_c)\ Q_\{t,c\} + \Sigma_c\ q_c\ z_c = \Sigma_c\ \backslash ensuremath\{\backslash tilde\{q\}\}_c\ Q_\{t,c\} + b$$

The pre-scaled query \ensuremath{\tilde{q}}_c = q_c · s_c and the bias b = Σ q_c z_c are computed once per block (`kv_qk_prepare`), and \ensuremath{\tilde{q}} is de-interleaved into even-channel and odd-channel halves so that each packed byte (low nibble = even channel, high nibble = odd channel) multiplies two contiguous vectors. The per-token loop then does only integer-to-float conversion and fused multiply–add:

Table 6.26: Mapping from the scalar algorithm to the RVV instructions.

Step per token	Scalar C	RVV instruction (observed in the compiled output)
Load packed bytes	`row[j]`	`vle8.v` (byte load with `vsetvli`)
Extract low / high nibble	`& 0x0F`, `» 4`	`vand.vx`, `vsrl.vi`
Widen to 32 bits	implicit	`vzext.vf4`
Convert to float	cast	`vfcvt.f.xu.v`
Load pre-scaled query halves	`qs_even[j]`, `qs_odd[j]`	`vle32.v`
Accumulate	`acc += ...`	`vfmacc.vv` with tail-undisturbed policy
Reduce across lanes	sequential sum	`vfredusum.vs`
Add bias	`+ bias`	scalar add after `vfmv.f.s`

The tail-undisturbed (`_tu`) form of the multiply–add matters for correctness: when the last strip of a row has fewer than a full vector of elements, the accumulator lanes beyond the active length must keep their previous values until the final reduction. Using the default tail-agnostic form could let those lanes be overwritten with arbitrary values.

6.6.2 Numerical agreement (host)

```
$ scalar C kernel vs NumPy dequantize-then-dot; RVV compile check


case  tokens  dim  max|C-kernel - numpy dequant.dot|
   0      16   64  5.72e-06
   1      32   64  4.77e-06
   2      16  128  2.00e-05
   3      64   64  6.68e-06
RVV compile check (clang riscv64 rv64gcv): OK -> kv_kernels_rvv.o 2856 bytes (object only, NOT executed)
```

Figure 6.18: Scalar C kernel versus NumPy dequantise-then-dot on random blocks, and the RVV compile check.

On four block shapes the maximum absolute difference between the C kernel and the straightforward NumPy computation (dequantise the whole block, then take the dot product) is between 4.8×10^{-6} and 2.0×10^{-5}, which is floating-point rounding for sums of 64–128 products of magnitude about one. The test suite also checks the pre-scaling algebra and the nibble order.

6.6.3 RISC-V Vector compile check

The same source was compiled with clang for `riscv64-unknown-elf` with `-march=rv64gcv -O2` (object file only). The inner loop of the compiled code is shown below; it contains the expected vector instructions listed in the mapping table above.

This demonstrates that the intrinsics lower to the intended instruction sequence for a vector target. **It does not demonstrate that the code runs correctly or fast on Ara:** no Ara, Spike or QEMU execution has been performed (Chapter 7).

6.6.4 Why a fused kernel: bandwidth arithmetic

An unfused implementation would first dequantise the key block into a floating-point buffer and then compute dot products. If that buffer is staged in memory, each 4-bit key element costs 0.5 bytes to read, 4 bytes to write and 4 bytes to read back (8.5 bytes of traffic) versus 0.5 bytes in the fused kernel, a factor of 17 in traffic for the key stream. If the dequantised tile is kept entirely in registers the memory traffic is the same as the fused kernel's but the kernel then executes the extra dequantisation instructions (a multiply and add per element) before each multiply–add. The fused form removes both. These are analytical counts from the algorithm, not measurements.

What would make this a measured result

Run the kernel on Spike for functional verification, then on the Ara RTL simulation or an FPGA build to obtain cycles per token for block sizes of 16–64 tokens and head dimensions of 64–128; compare with a dequantise-then-dot baseline written with the same intrinsics, on the same data.

```
$ clang --target=riscv64 -march=rv64gcv -O2 -S ara/kv_kernels.c   (kv_qk_dot_kivi4_rvv)


kv_qk_dot_kivi4_rvv:                    # @kv_qk_dot_kivi4_rvv
# %bb.0:
 vsetvli a6, zero, e32, m1, ta, ma
 blez a3, .LBB2_6
# %bb.1:
 li a6, 0
 srliw a7, a4, 31
 add a7, a7, a4
 sraiw a7, a7, 1
 vmv.v.i v8, 0
 li t0, 2
 vmv.s.x v9, zero
 j .LBB2_3
.LBB2_2:                                #   in Loop: Header=BB2_3 Depth=1
 vsetvli t1, zero, e32, m1, ta, ma
 vfredusum.vs v10, v10, v9
 vfmv.f.s fa5, v10
 fadd.s fa5, fa5, fa0
 slli t1, a6, 2
 add t1, t1, a5
 addi a6, a6, 1
 fsw fa5, 0(t1)
 beq a6, a3, .LBB2_6
.LBB2_3:                                # =>This Loop Header: Depth=1
                                        #     Child Loop BB2_5 Depth 2
 vmv1r.v v10, v8
 blt a4, t0, .LBB2_2
# %bb.4:                                #   in Loop: Header=BB2_3 Depth=1
 li t1, 0
 mul t2, a6, a7
 add t2, t2, a2
 vmv1r.v v10, v8
.LBB2_5:                                #   Parent Loop BB2_3 Depth=1
                                        # =>  This Inner Loop Header: Depth=2
 subw t3, a7, t1
 vsetvli t3, t3, e8, mf4, ta, ma
 add t4, t2, t1
 vle8.v v11, (t4)
 vand.vi v12, v11, 15
 vsrl.vi v11, v11, 4
 vsetvli zero, zero, e32, m1, ta, ma
 vzext.vf4 v13, v12
 vfcvt.f.xu.v v12, v13
 vzext.vf4 v13, v11
 vfcvt.f.xu.v v11, v13
 slli t4, t1, 2
 add t5, a0, t4
 vle32.v v13, (t5)
 add t4, t4, a1
 vle32.v v14, (t4)
 vsetvli zero, zero, e32, m1, tu, ma
 vfmacc.vv v10, v12, v13
 addw t1, t1, t3
 vfmacc.vv v10, v11, v14
 blt t1, a7, .LBB2_5
 j .LBB2_2
.LBB2_6:
 ret
```

Figure 6.19: RVV assembly for the 4-bit key kernel (excerpt of the compiler output).

6.7 Scaling the Ratios to Larger Models: Arithmetic Only

Section 6.4 reports results on a small, deliberately tight configuration (168 KB for 1,536 tokens of a 64-channel cache). This chapter extrapolates those *ratios* to the scale of an actual edge deployment, using arithmetic only. These are projections from measured ratios and stated assumptions, not new measurements.

6.7.1 Why the compression ratios transfer to other geometries

For a block of t tokens and d channels with value group g, the bytes of a b-bit block relative to FP16 are b/16 + 1/t + 1/g (scales and zeros are two FP16 numbers each, per channel for keys and per group for values). The expression does not depend on d. For t = 16 and g = 32 it gives 0.594 (INT8), 0.344 (4-bit) and 0.219 (2-bit), identical to the

measured values in Section 5.3. Larger blocks reduce the metadata share (t = 32 gives 0.3125 for 4-bit) but, as Section 6.4.4.3 shows, enlarge the unit of placement.

6.7.2 What fits on a phone-class device

Table 6.27: Llama-3-8B-like geometry (131,072 B per token in FP16). The mixed rows use the resident ratios measured in E1 and assume the same mix at this scale.

Storage	Bytes vs FP16	Max tokens in 1 GiB	KV size at 32K tokens
FP16	1.000	8,192	4.00 GiB
INT8	0.594	13,791	2.38 GiB
Mixed int8/4-bit (E1: 0.371)	0.371	22,081	1.48 GiB
Mixed incl. 2-bit (E1: 0.310)	0.310	26,426	1.24 GiB
All 4-bit	0.344	23,814	1.38 GiB
All 2-bit	0.219	37,406	0.88 GiB

In the model, a 1 GiB KV budget that holds about 8,200 tokens in FP16 holds about 26,000 tokens at the 0.310 mix without dropping any: a gain of 3.2×, in line with 1/0.310. A 32K-token context shrinks from 4 GiB to about 1.2 GiB.

6.7.3 Reads per token

In the 1,536-token run, after step 1,024 the engine read on average 35 % of the blocks per decode step (24 selected plus the protected set). If the same fraction applied at larger scale and the blocks read had the average encoding, bytes read per token would be roughly $0.35 \times 0.31 = 0.11$ of the dense FP16 read. The fraction would shrink further with longer contexts because the block budget is fixed while the cache grows, but recall would also fall (Section 6.4.3); the budget has to be tuned against the application's quality needs.

Table 6.28: Projected KV-read bound for the 8B-like geometry. For 8,192 tokens the measured read fraction is used; beyond that the fixed 24-block budget plus a 36-token protected set is assumed. Weight reads and compute are ignored, and the bound is not a prediction of end-to-end speed.

Context	Dense FP16 KV read / token	KV-only bound (tok/s, 12.8 GB/s)	Selective + mixed codecs read / token	KV-only bound (tok/s)
8,192	1.00 GiB	11.9	112 MiB	109.3
32,768	4.00 GiB	3.0	16 MiB	750.0
131,072	16.00 GiB	0.7	16 MiB	750.0

These bounds extrapolate from one 64-channel head on synthetic data and from a read fraction measured at 1,536 tokens. They illustrate the arithmetic; no model of this size was run and they are not predictions of speed. At 32K tokens the dense FP16 read bound is about 3 tokens per second on 12.8 GB/s memory; with selective reads and mixed codecs the KV-only bound is far higher, which would make the weights, not the cache, the limit again. This is the sense in which the design aims to restore the bottleneck to where it belongs. Whether recall at 24 blocks is sufficient at that length for a real model is exactly the question that real-model validation (Chapter 7, gap 1) must answer.

6.7.4 What would convince a sceptic

Table 6.29: Evidence available and evidence needed for each claim.

Claim	Evidence in this report	Evidence still needed
The joint policy beats placement alone in a tight budget	E1, E2, E5 on the synthetic trace (a one-codec 4-bit baseline matches its feasibility, E1 and E10)	Same ablation on real traces, with task accuracy
Hysteresis and cadence control churn	E4	Energy measured on a device
Hardware blocks implement the specified arithmetic	Bit-exact RTL regression, mutation tests, lint	Place-and-route, timing, power, integration with a bus
The kernel is correct	Host agreement with NumPy; RVV compiles	Execution on Spike/Ara
The design speeds up an edge device	Modelled fetch time (in the model the planner is slower than a one-codec 4-bit baseline)	Measured tokens/s on target hardware

6.8 Conclusions we can draw

- **Choosing precision together with placement fixes the capacity problem.** With a budget smaller than the FP16 data, UAMM-1 had to delete 63 % of the tokens. UAMM-2 kept all of them at a measured error.
- **The codecs do most of that work.** A single 4-bit codec for every block also fits. The planner is not what makes the data fit.
- **The planner buys accuracy and robustness.** At the same size it has lower error than the 4-bit baseline, with 2-bit blocks it gets smaller, and it still fits when the budget is tightened.
- **Reading less matters as much as storing less.** Reading 24 of about 96 blocks keeps 94.5 % of the attention mass.
- **Stability needs explicit rules.** Hysteresis, a minimum residency and a replanning interval reduced migrations from 1,871 to 630.
- **Small hardware blocks can be verified thoroughly.** The three RTL modules are bit-exact against their models, and 12 of 12 injected bugs were caught. The scorer implements UAMM-1's score, not the planner's cost.

6.9 Did we do anything new?

Almost every ingredient of UAMM-2 comes from a published method, and we have said so wherever it appears. We do not claim a new algorithm for any one of them. What we think is new, or at least not something we found in the papers we read, is the following combination and the way it was checked.

- Treating a block's placement as a (tier, codec) pair and choosing it with one cost that adds read time, decode time, a measured quantisation penalty weighted by heat, and a capacity price, under per-tier byte limits.
- A precision ratchet and an opt-in drop, so that every loss in quality is explicit and capacity overflow is reported as a number and never hidden.
- Using Quest-style min/max bounds to choose the blocks to read in a system whose blocks are stored at different precisions in different tiers.

- A small hardware path (scorer, 4-bit quantiser and dequantiser without a divider, block table) checked against bit-exact models and by mutation testing, together with a fused vector kernel that works on packed 4-bit keys.
- A measurement habit that we consider part of the contribution: a fair one-codec baseline, a capacity-aware version of UAMM-1, charging the metadata to the budget, and a budget sweep that shows where the advantage disappears.

What we do not claim: a speed-up on real hardware, results on real language models, or any result from running on Ara. Those require measurements we have not made.

7. Future Scope

UAMM-2 is a first complete version, not a finished system. While building it and reading the results we kept running into places where the design could be taken further. This chapter collects them as research scopes. For each one we describe what UAMM-2 does today, what we noticed, how we think it could be improved and how we would go about building and testing it. None of this is implemented or tested yet. These are directions we think are worth following, and the sketches show where a change would go in the code.

Status of this chapter

Everything here is future work. No scope in this chapter has been implemented or measured. Where we quote a number, it is a result from the simulations in Section 6.4 that motivated the idea.

7.1 Scopes on precision

7.1.1 Scope 1: Different bit widths for keys and for values

Where UAMM-2 is today. Keys and values of a block use the same bit width, whatever codec the planner picks.

What we noticed. The two enter attention in different ways. An error in a key changes a logit, and the softmax then scales that token's weight by about e to the power of the change, so a small key error can reorder tokens whose logits are close. An error in a value only shifts the output linearly. In our workload the 2-bit codec has a squared error of 0.130 per block. That looks tolerable for values but risky for keys.

How we could take it forward. We may separate the two widths and let the planner choose a pair such as K4/V2, so that keys keep more bits than values. The planner would price the two error terms separately, since the two do not hurt attention equally.

How we would build it. The codec name would carry two widths, `encode` would take `bits_k` and `bits_v`, and calibration would measure a logit error for keys (the largest $|q \cdot (k - \ensuremath{\hat{k}})|$ over sampled queries) next to the value error. We would test it by comparing K4/V2 with 3-bit for both at the same memory.

```
# codecs.py (sketch)
def encode(k, v, bits_k, bits_v, v_group):
    qk, sk, zk = _asym_quant(k, bits_k, axis=0)          # per-channel keys
    qv, sv, zv = _asym_quant(v_grouped, bits_v, axis=2)   # per-token values
    return pack(qk, bits_k) + pack(qv, bits_v) + meta

# planner cost with separate quality terms
cost = p*read_ms + lam_k*heat*err_logit_k[c] + lam_v*heat*err_v[c] + congestion
```

7.1.2 Scope 2: Knowing when a compressed block is safe to use

Where UAMM-2 is today. The planner uses an average error measured when the block is sealed. It does not know whether, for the current query, a compressed block could change which tokens rank highest.

What we noticed. With a per-channel scale s_c, the dequantisation error in channel c is at most s_c/2, so for a query q the logit error of any token in the block is bounded by ½ Σ|q_c| · s_c. That is one dot product with the scale vector, and the selection step already does something very similar with the min/max summaries.

How we could take it forward. At selection time we could compute this bound for every candidate block. If it is large compared with the logit gap to the next block in the ranking, the block is risky and we would read it from a higher-precision copy or keep it out of the approximate path.

How we would build it. The per-channel key scales are already in each payload. We would keep them as a small resident vector, add an `err_bound(q, block)` function next to `select_blocks`, and measure how often it flags a block whose error actually changed the output.

```
def err_bound(q, scale_k):          # per-channel key scales of the block
    return 0.5 * float(np.abs(q) @ scale_k)

risky = [b for b in chosen if err_bound(q, b.scale_k) > tau]
```

7.1.3 Scope 3: Better quantisation ranges: outlier tokens and clipping

Where UAMM-2 is today. Keys are scaled per channel, which deals with channels that are large for every token. The range of a block is its exact minimum and maximum.

What we noticed. A single token with an unusually large key stretches the range of every channel it touches, and the other 15 tokens then use only a few levels. Heavy-hitter and sink tokens are the ones most likely to be large and the ones we most want to keep accurate. Our block-size sweep points the same way: 8-token blocks had the lowest error (0.110) and 64-token blocks the highest (0.341).

How we could take it forward. Two ideas fit together. One is a small FP16 side buffer for the largest one or two tokens of a block, left out of the range calculation. The other is to take the range from the 2nd and 98th percentile of each channel and let the side buffer hold what is clipped. Calibration already encodes each block once, so trying a second clipping level costs one more encode.

How we would build it. In `_asym_quant` we would find the rows with the largest key norm, replace them by the channel median before computing min and max, and append (index, key, value) records to the payload. The overhead is 256 bytes per block for one outlier at 64 channels. We would test on data with injected outlier tokens.

```
idx = np.argsort(-np.linalg.norm(k, axis=1))[:m]
k_clip = k.copy(); k_clip[idx] = np.median(k, axis=0)
q, s, z = _asym_quant(k_clip, bits, axis=0)
payload += pack_sidecar(idx, k[idx], v[idx])   # exact rows
```

7.1.4 Scope 4: Keeping track of error across repeated re-encoding

Where UAMM-2 is today. Each block carries an error table measured against its FP16 original when it is sealed. Re-encoding decodes the current payload and encodes it again.

What we noticed. A block that goes from INT8 to 4-bit to 2-bit ends up with more error than the 2-bit entry of its table says, so the planner underestimates the cost of its own earlier decisions. If the errors are independent, the total is roughly $1 - (1 - e_1)(1 -$

e_2).

How we could take it forward. We could keep an accumulated error per block, use it in the quality term and in the ratchet check, and limit the number of lossy steps. For cold blocks a copy of the FP16 original could stay in flash so that re-encoding always starts from it.

How we would build it. We would add an `err_acc` field to the block, update it in `reencode`, and have the planner use the larger of the table value and the accumulated value. The test is to compare the predicted error of long-lived blocks with the error measured after the fact.

```
b.err_acc = 1 - (1 - b.err_acc) * (1 - b.err_table[codec])      # in reencode()
```

7.1.5 Scope 5: Choosing precision by sensitivity, and keeping a quality floor

Where UAMM-2 is today. The planner lowers precision according to how hot a block is, and the cost function trades accuracy for speed everywhere, even when memory is plentiful.

What we noticed. Two blocks with the same heat can differ in how much their error moves the output, because it also depends on the size of their values and on how sharp the logits are. We also saw that spare memory is not always put to good use. At 512 tokens the cache fits in FP16 with error 0.0013 under UAMM-1, but UAMM-2 still compresses it and the error becomes 0.016.

How we could take it forward. We may replace heat by a sensitivity score (mean attention weight times a measure of how much the block's error can change the output) and only demote a block when its tier is under pressure. A per-request error budget would let the user say how much total error is acceptable, with the planner refusing a demotion that would break it.

How we would build it. `h2o.py` already provides the mean attention weight, and sealing could store the norm of the values. We would add a fill threshold to `Config`, a `pressure(tier)` function to `tiers.py` and a running error sum to the planner. We would check that error no longer rises when the budget is increased.

```
if used[t] / cap[t] < cfg.min_fill_to_demote and codec_rank(c) > b.rank:
    continue                                  # no pressure: keep precision
if err_sum + heat * new_err > cfg.error_budget:
    continue
```

7.1.6 Scope 6: Storing keys before positional encoding

Where UAMM-2 is today. The engine stores whatever key vector it is given. In a real model that vector has usually been through rotary position embedding (RoPE).

What we noticed. The rotation changes the per-channel statistics with position and mixes large channels into their partners, which weakens per-channel quantisation.

How we could take it forward. We could store keys before the rotation and apply it on the query side instead, because (R_p k) · q equals k · (R_p^{T} q). For a block of 16 tokens that is 16 rotated copies of the query, which is small next to reading the keys.

How we would build it. The kernel call would take cosine and sine tables for the

positions of the block. We would test it on a real model's keys, which we do not have yet.

7.2 Scopes on placement and policy

7.2.1 Scope 7: Prefetching blocks before they are needed

Where UAMM-2 is today. A block is fetched when attention asks for it. If the query returns to an old topic, the first reads wait for flash.

What we noticed. Replanning every 64 steps gave 3,326 flash reads and 0.272 ms per token, against 599 reads and 0.090 ms with replanning every 2 steps. Replanning often costs migrations (768 against 345), so there is a trade-off we cannot fix by only changing the interval.

How we could take it forward. Selection already ranks every block. The blocks ranked just below the ones we read are the most likely to be read next, so we could promote those that sit in flash into DRAM while the current attention step is running.

How we would build it. A `prefetch(q, candidates)` function in `attention.py` would return block ids. The simulator would move them through the runtime adapter with a cost charged to a background channel. We would measure flash reads and time per token against the numbers above.

```
ranked = np.argsort(-bound)
read     = ranked[:B]
prefetch = [b for b in ranked[B:B+P] if tier[b] == FLASH]   # promote ahead of need
```

7.2.2 Scope 8: Layers, heads and grouped-query attention

Where UAMM-2 is today. The engine keeps one flattened vector per token, which is one head of one layer in effect.

What we noticed. A model of the 8B class has 32 layers with 8 KV heads each, which is 256 separate streams with different attention patterns. One global budget of 24 blocks would starve heads that look broadly and waste reads on heads that look sharply.

How we could take it forward. We would add the layer and head to the identity of a block and give each stream its own read budget in proportion to the entropy of its attention, as PyramidKV and FastGen do in their own ways.

How we would build it. The block would gain `layer` and `head` fields, the importance tracker would keep one score array per stream, and the block table in the RTL would need a few more address bits. This is the largest change on our list, since it touches almost every module.

7.2.3 Scope 9: A better optimiser than the greedy planner

Where UAMM-2 is today. The planner visits blocks from hottest to coldest and each takes its cheapest feasible slot.

What we noticed. In the budget sweep the error was 0.115 at twice the budget but 0.152 at three times, which means the heuristic is not always close to a good assignment.

How we could take it forward. We may write the choice as a multiple-choice knapsack, solve it offline on recorded traces to see how far the greedy answer is from the best one, and, if the gap matters, use a price per tier that is adjusted until the capacities are met. That maps onto the congestion term the planner already has, and FlexGen formulates placement in a similar way.

How we would build it. The offline solver only needs the cost table of each block. The online version is a short dual-update loop.

```
for it in range(20):
    choice = {b: argmin_tc(cost(b,t,c) + price[t]*bytes(b,c)) for b in blocks}
    for t in tiers:
        price[t] = max(0, price[t] + step * (used(choice,t) - cap[t]))
```

7.2.4 Scope 10: Counting and shrinking the selection summaries

Where UAMM-2 is today. Each block keeps minimum and maximum key vectors in FP16 so that attention can choose blocks. The main experiments do not count them against the memory budget.

What we noticed. They take 256 bytes per block at 64 channels, which is 14 % of our 168 KB budget. When we charged them in the key-summary experiment, some variants stopped fitting.

How we could take it forward. We could store the summaries in 8 bits (rounding the minimum down and the maximum up so the bound stays valid), or keep one summary for each group of eight channels and accept a looser bound.

How we would build it. The summary arrays would become `int8`. We would measure how much the selection recall changes, since a looser bound makes the selection less sharp.

7.2.5 Scope 11: When the budget runs out, and sharing between sequences

Where UAMM-2 is today. If even 2-bit blocks do not fit, the planner reports the overflow and stops. Each sequence also owns all of its blocks.

What we noticed. At 3,072 tokens the all-codec system overflows by 28.5 KB. A system prompt of 800 tokens shared by four sessions takes four times the memory of one copy.

How we could take it forward. As a last resort we could drop the least important tokens inside the least important blocks, at token level and with an explicit counter. For shared prompts we could count references to blocks and copy a block only when someone writes to it, the way PagedAttention does.

How we would build it. A `compact(block, keep_mask)` function would re-encode a block with fewer tokens and `allow_drop` would become never, block or token. Sharing needs a reference count in the block and a table from the hash of the prefix to a block.

7.3 Scopes on systems and hardware

7.3.1 Scope 12: Moving blocks safely while attention is running, and flash wear

Where UAMM-2 is today. In the simulator a block moves instantly. Every demotion into flash is also treated like any other move.

What we noticed. On a real system a block may be re-encoded while a step is reading it, which would mix two formats in one result. Flash also has limited write endurance, and the planner can still move a block back and forth between DRAM and flash.

How we could take it forward. We could give every block-table entry a version number, write the new payload to a second buffer and then switch the entry in one update. A step that sees the version change would read again. For flash we could count writes, add their cost to the move cost and ask for a larger improvement before a block is moved into flash.

How we would build it. The block-table RTL already returns the old entry when a read and a write collide, so it is a natural place for the version number. The wear counters live in `tiers.py`.

7.3.2 Scope 13: Bringing the hardware closer to the planner

Where UAMM-2 is today. The Verilog scorer computes the original UAMM-1 weighted score, not the planner's utility cost, and the codec works on 8-bit data while the software codecs start from FP16. Nothing has been placed and routed.

What we noticed. The quantiser alone is about 11,300 generic gates, mostly sixteen 8 $\times$ 16-bit multipliers.

How we could take it forward. We would add a front stage that turns FP16 into a group-normalised 8-bit value using the group minimum and maximum from firmware. We would write the utility cost in fixed point with small lookup tables for the read-time and error terms. We could also save area by rounding the range up to a power of two, so the reciprocal multiplier becomes a shift, or by using fewer lanes.

How we would build it. The new scorer would be checked the same way as the current one: a bit-exact model, boundary vectors and a mutation test. Place and route would be the first step towards real area and timing numbers.

7.4 Order of work

If we were to carry on, we would start with the scopes that need no new data and finish with those that need a real model or real hardware.

Table 7.1: Suggested order of work.

Step	Goal	Scopes
1	Precision control: separate key and value widths, error bound, outliers and clipping, error tracking, sensitivity	1, 2, 3, 4, 5, 6
2	Policy: prefetch, layers and heads, a better optimiser, counting summaries	7, 8, 9, 10
3	Limits and sharing: token-level fallback, copy-on-write	11

Step	Goal	Scopes
4	Systems and hardware: safe migration, flash wear, utility scorer in RTL	12, 13

8. Project Timeline

The work ran from 10 August 2026 to the middle of November 2026. The first draft was submitted on 24 August, and everything after that is the work reported here. Dates after the first draft are approximate periods, since several activities overlapped.

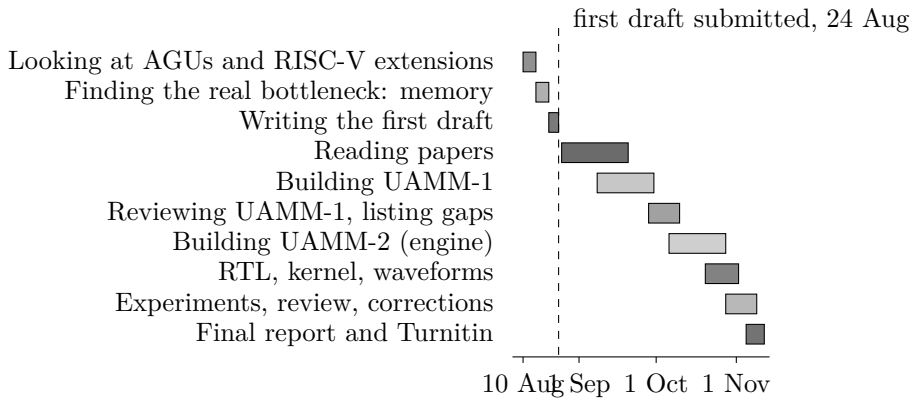


Figure 8.1: Project timeline. Bars show the approximate period of each activity.

8.1 From 10 to 24 August: finding the problem

Table 8.1: The first two weeks.

Period	What we did
10–12 Aug	Started from the processor. We wanted to see whether address-generation units in a RISC-V core could be made to help language-model inference, and read about the RISC-V vector extension and about vector processors such as Ara.
13–15 Aug	Estimated where the time goes for one decoded token. The arithmetic units were not the limit; address computation, table walks, unpacking and movement from slow memory were.
16–19 Aug	Narrowed the topic to the KV cache, because it grows with every token, is read at every step and competes with the weights for memory. Studied PagedAttention, H2O and KIVI at a high level.
20–23 Aug	Wrote the problem analysis: why constrained RISC-V platforms are memory-bound, what the cache costs and what a memory manager would have to do. Discussed it with Prof. DuttaRoy.
24 Aug	**Submission of the first draft.**

8.2 From 24 August to November: building and checking

Table 8.2: From the first draft to the final report.

Period	What we did
25 Aug – mid Sep	Read the papers more carefully and wrote down, for each one, what it solves and what it leaves open for an edge device. Built the taxonomy and the list of gaps.
Early Sep – end Sep	Built UAMM-1: paged blocks, H2O importance, three levels with thresholds, a simulator, and a first Verilog scorer.

Period	**What we did**
End Sep – 10 Oct	Reviewed UAMM-1. Found that it could not make data smaller and that parts of it were unreliable (silent deletion, a saturating importance value, free migrations, a divider, thin tests). Decided on a redesign.
Early – late Oct	Built the UAMM-2 engine: real tensors, codecs, per-block error, utility planner, query-aware attention and the simulator. Added the tests.
Late Oct – early Nov	Wrote the three Verilog modules, ran them in Verilator, looked at the waveforms in GTKWave, ran Yosys and a mutation test. Wrote the vector kernel and compiled it.
Late Oct – 9 Nov	Ran the experiments, then re-ran everything against the claims in our own write-up. Found a weak baseline, uncounted metadata and a few wrong numbers, and corrected them.
9–12 Nov	Rewrote the report in its present structure, ran the similarity check and prepared the repository.

9. Repository, Git Setup and Summary

9.1 What is in the repository

All of the code, the experiment results, the waveform dumps and the scripts that regenerate every figure and table in this report are in one public repository: **https://github.com/IshanviPrasad29/uamm2-kv-cache-manager**.

Table 9.1: Repository layout.

Folder	Contents
uamm2/	Python engine: cache, codecs, importance, planner, attention, simulator, command line
uamm1_original/	Listings of the original UAMM-1 Python and Verilog
hw/	Verilog modules, testbenches and Makefile
ara/	C kernel with scalar and RISC-V vector paths
experiments/	Experiment scripts E1 to E10, mutation test, figure scripts and raw results (JSON)
docs/	Waveform dumps, GTKWave views, synthesis reports, figures and terminal captures
tests/	59 pytest tests
report/	This report as a PDF

9.2 Setting it up with Git

These commands were written for Ubuntu. The first block gets the code onto your machine, the second installs what the tests need, and the third runs the checks.

Clone, install and run

```
# 1. get the code
git clone https://github.com/IshanviPrasad29/uamm2-kv-cache-manager.git
cd uamm2-kv-cache-manager

# 2. install the Python tools (Verilator is only needed for the hardware tests)
pip install numpy pytest
sudo apt install verilator g++ make

# 3. run the checks
python3 -m pytest -q tests                # 59 tests
make -C hw test                           # Verilog regression
python3 -m uamm2 ablate --tokens 1536     # the ablation table
python3 experiments/run_experiments.py    # all experiments, results in experiments/results/
```

9.3 Summary

Running a language model on an edge processor is limited by memory. The KV cache grows with each token, is read again at every step and competes with the weights for a few gigabytes. Published work answers parts of the problem: paging, eviction, query-aware selection, quantisation and offloading.

We built UAMM-1 to combine paging and importance-based placement and found that it could not make data smaller, so it deleted tokens when the budget was tight, and that parts of its implementation were unreliable. UAMM-2 stores each block as a (tier, codec)

pair, picks the pair with a priced utility, lets precision only go down and reads only the blocks a query needs. We added three verified Verilog blocks and a vector kernel for the inner loops.

On a synthetic workload with assumed memory costs, UAMM-2 keeps all 1,536 tokens in a budget where UAMM-1 deleted 63 % of them, retaining 0.94 of the attention mass instead of 0.76. A one-codec 4-bit baseline also fits and is faster in our model, so the planner's value is lower error at the same size, a smaller size with 2-bit blocks and feasibility in tighter budgets. Charging the key summaries to the budget raises the resident size to 0.435 of FP16. The Verilog blocks are bit-exact against their models and every injected fault was caught.

What remains is to replace the assumptions with measurements: real model activations, real memory costs, the vector kernel on Ara or a simulator, and integration with a deployed runtime. The future-scope chapter lists these steps and the ideas we have for them.

References

[1] Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[2] Zhang, Z. et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. NeurIPS 2023. arXiv:2306.14048.

[3] Liu, Z. et al. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. NeurIPS 2023. arXiv:2305.17118.

[4] Xiao, G. et al. Efficient Streaming Language Models with Attention Sinks. ICLR 2024. arXiv:2309.17453.

[5] Li, Y. et al. SnapKV: LLM Knows What You are Looking for Before Generation. 2024. arXiv:2404.14469.

[6] Ge, S. et al. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs (FastGen). 2023. arXiv:2310.01801.

[7] Cai, Z. et al. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. 2024. arXiv:2406.02069.

[8] Tang, J. et al. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. ICML 2024. arXiv:2406.10774.

[9] Lee, W. et al. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. OSDI 2024. arXiv:2406.19707.

[10] Liu, Z. et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. ICML 2024. arXiv:2402.02750.

[11] Hooper, C. et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. NeurIPS 2024. arXiv:2401.18079.

[12] Chang, C.-C. et al. Palu: Compressing KV-Cache with Low-Rank Projection. 2024. arXiv:2407.21118.

[13] Sheng, Y. et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. ICML 2023. arXiv:2303.06865.

[14] Alizadeh, K. et al. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. 2023. arXiv:2312.11514.

[15] Shazeer, N. Fast Transformer Decoding: One Write-Head is All You Need. 2019. arXiv:1911.02150.

[16] Ainslie, J. et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. 2023. arXiv:2305.13245.

[17] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv:2405.04434.

[18] Dao, T. et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. arXiv:2205.14135.

[19] Xiao, G. et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023. arXiv:2211.10438.

[20] Cavalcante, M. et al. Ara: A 1-GHz+ Scalable and Energy-Efficient RISC-V Vector Processor with Multiprecision Floating-Point Support in 22-nm FD-SOI. IEEE TVLSI, 2020.

[21] Perotti, M. et al. Ara2: Exploring Single- and Multi-Core Vector Processing with an Efficient RVV 1.0 Compliant Open-Source Processor. 2023. arXiv:2311.07493.

[22] MLC team. MLC LLM: universal LLM deployment engine with ML compilation. `https://github.com/mlc-ai/mlc-llm`.

[23] Chen, T. et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. OSDI 2018. arXiv:1802.04799.

[24] EvicPress (arXiv:2512.14946); Rehg, I., KV-Compress (arXiv:2410.00161); PagedEviction (arXiv:2509.04377); DiffKV (arXiv:2412.03131); TensorRT-LLM KV-cache compression framework; CXL-SpecKV; HiKV (arXiv:2607.22389); Titanus (arXiv:2505.17787); Kelle (arXiv:2510.16040). Recent preprints; details to be re-verified against the latest versions before formal citation.

[25] RISC-V International. RISC-V Vector Extension, version 1.0.

[26] Verilator, Yosys and GTKWave open-source tools used for simulation, synthesis checking and waveform viewing.

[27] Sarangi, S. R. Basic Computer Architecture (version 2.0). White Falcon Publishing, 2021. `https://www.cse.iitd.ac.in/~srsarangi/archbooksoft.html`

A. Plagiarism Report

The report was checked with Turnitin (Integrity Overview) for class ELL305. The check was run on the 108-page version submitted on 11 November 2026 at 23:27; the only later changes are on the cover page and in this appendix. The first five pages of the Turnitin output follow this page, unmodified.

Table A.1: Summary of the Turnitin similarity check.

Item	Value
Service	Turnitin (Integrity Overview), class ELL305 2601 Computer Architecture
Submission	UAMM2_Report.pdf, 108 pages, 39,683 words, 188,155 characters, submitted 11 November 2026, 23:27 IST
Submission ID	trn:oid:::1:3708550872
Overall similarity	2 %
Filtered from the report	Bibliography and quoted text
Match groups	51 not cited or quoted (1 %); 5 missing quotations (0 %); 0 missing citation; 0 cited and quoted
Top sources	1 % internet sources, 0 % publications, 0 % submitted works (student papers); no single source above 1 %

Ishanvi Prasad

Term Paper

Term Paper

ELL305 2601 Computer Architecture

India Institute of Technology Delhi

Document Details

Submission ID
trn:oid:::1:3708550872

Submission Date
Nov 11, 2026, 11:27 PM GMT+5:30

Download Date
Nov 11, 2026, 11:32 PM GMT+5:30

File Name
UAMM2_Report.pdf

File Size
2.9 MB

108 Pages

39,683 Words

188,155 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

51 Not Cited or Quoted 1%
Matches with neither in-text citation nor quotation marks

5 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
0% Submitted works (Student Papers)

Match Groups

- **51** Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- **5** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		<1%
2	Internet	
github.com		<1%
3	Publication	
Mannon Ochilov, Muhammadjon Musaev, Shukrullo Kakharov, Dilorom Nabieva, ...		<1%
4	Publication	
Rajarshi Chowdhury. "I/o for LLM inference: a survey of storage and memory bott...		<1%
5	Student papers	
Hong Kong University of Science and Technology		<1%
6	Student papers	
Manipal University		<1%
7	Internet	
aicassindra.com		<1%
8	Student papers	
Higher Education Commission Pakistan		<1%
9	Publication	
"Advanced Parallel Processing Technologies", Springer Science and Business Med...		<1%
10	Student papers	
Aalto Yliopisto		<1%

11	Internet	
championsschool.com		<1%
12	Student papers	
British University in Egypt		<1%
13	Internet	
gowda.ai		<1%
14	Internet	
briansu.co		<1%
15	Internet	
haochengxi.github.io		<1%
16	Internet	
ses.library.usyd.edu.au		<1%
17	Publication	
Fatemeh Sarhaddi, Ngoc Thi Nguyen, Agustin Zuniga, Musfira Khan, Pan Hui, Sas...		<1%
18	Internet	
hyper-accel.github.io		<1%
19	Internet	
people.eecs.berkeley.edu		<1%
20	Internet	
www.local-llm.net		<1%
21	Internet	
www.pvsm.ru		<1%
22	Publication	
"Euro-Par 2026: Parallel Processing", Springer Science and Business Media LLC, 20...		<1%
23	Publication	
Nimas Ayu Untariyati, Kusworo Adi, Aris Puji Widodo, M. Teduh Uliniansyah. "Ont...		<1%
24	Internet	
dokumen.pub		<1%

25 Internet

lcd-www.colorado.edu <1%

26 Internet

learn.freetools.me <1%

27 Internet

mikenguyen13.github.io <1%

28 Internet

tw.ic-on-line.cn <1%

B. Readability Indices

We measured the running prose of the main chapters (Chapters 1 to 9: paragraphs, bullet points and boxes; tables, captions, equations, code and appendices excluded). The indices are computed by our own script with the usual published formulas. Syllables are counted with a rule-based heuristic, so the values are approximate. The method follows the description at analyzemywriting.com/readability_indices.html.

Table B.1: Counts used for the readability indices.

Quantity	Value
Words	19,117
Sentences	1,098
Words per sentence	17.4
Syllables per word	1.50
Words of three or more syllables	11.7 %

Table B.2: Readability indices of the main text.

Index	Value	What it means
Flesch Reading Ease	62.1	0 to 100; higher is easier. 30 to 50 is difficult, college level.
Flesch-Kincaid Grade Level	8.9	US school grade needed to follow the text.
Gunning Fog	11.6	Years of schooling needed; 12 is the level of a senior high-school student.
SMOG	11.3	Grade level from the count of long words per sentence.
Coleman-Liau	10.0	Grade level from letters per word and sentences per 100 words.
Automated Readability Index	9.3	Grade level from characters per word and words per sentence.

A Flesch Reading Ease of about 62 and grade levels between 9 and 12 put the text at the level of plain, careful technical English. The long words are mostly terms (quantisation, hysteresis, heterogeneous) that we could not replace.

C. Raw Experiment Tables

Mean ± standard deviation over seeds. Columns: resident / FP16, dropped tokens, mean relative L2 error, 95th-percentile error, attention-mass recall, modelled fetch ms per token, flash reads, migrations, re-encodes, overflow bytes.

Table C.1: E1 ablation (5 seeds, 1,536 tokens) (columns 1 to 5)

setting	res/FP16	drop	err	err p95	recall
v1_threshold_fp16	0.3667 ± 0.01019	972.8 ± 15.68	0.4789 ± 0.03459	1.288 ± 0.1304	0.7581 ± 0.01638
v1_capacity_aware_fp16	0.4062 ± 0	912 ± 0	0.4056 ± 0.03049	1.142 ± 0.08034	0.7917 ± 0.01163
static_kivi4_no_planner	0.3643 ± 0	0 ± 0	0.1519 ± 0.01504	0.3679 ± 0.06149	0.9389 ± 0.0117
static_kivi2_no_planner	0.2432 ± 0	0 ± 0	0.5716 ± 0.01873	1.022 ± 0.06567	0.9389 ± 0.0117
v2_planner_fp16	1 ± 0	0 ± 0	0.09209 ± 0.01912	0.3312 ± 0.07139	0.9389 ± 0.0117
v2_planner_int8_kivi4	0.3711 ± 0.002651	0 ± 0	0.1224 ± 0.01366	0.3614 ± 0.06096	0.9389 ± 0.0117
v2_planner_all_codecs	0.3101 ± 0.001326	0 ± 0	0.1662 ± 0.02025	0.5709 ± 0.1013	0.9389 ± 0.0117
v2_all_codecs_with_drop	0.3101 ± 0.001326	0 ± 0	0.1662 ± 0.02025	0.5709 ± 0.1013	0.9389 ± 0.0117

Table C.2: E1 ablation (5 seeds, 1,536 tokens) (columns 6 to 10)

setting	fetch ms	flash	migr	reenc	overflow B
v1_threshold_fp16	0.9229 ± 0.02585	1.261e+04 ± 374	744.8 ± 35.78	0 ± 0	0 ± 0
v1_capacity_aware_fp16	0.6181 ± 0.008936	8205 ± 129.2	883 ± 60.1	0 ± 0	0 ± 0
static_kivi4_no_planner	0.06814 ± 0.002529	270 ± 37.5	694.2 ± 38.49	93 ± 0	0 ± 0
static_kivi2_no_planner	0.0493 ± 4e-06	0 ± 0	309.8 ± 24.6	93 ± 0	0 ± 0
v2_planner_fp16	1.041 ± 0.005997	1.436e+04 ± 86.17	628.6 ± 18.78	0 ± 0	2.58e+05 ± 1832
v2_planner_int8_kivi4	0.1156 ± 0.002161	969.6 ± 32.11	698.6 ± 25.26	178.8 ± 1.6	0 ± 0
v2_planner_all_codecs	0.1117 ± 0.002028	915 ± 30.09	644.4 ± 21.27	225.6 ± 1.625	0 ± 0
v2_all_codecs_with_drop	0.1117 ± 0.002028	915 ± 30.09	644.4 ± 21.27	225.6 ± 1.625	0 ± 0

Table C.3: E4 quality weight lambda (columns 1 to 5)

setting	res/FP16	drop	err	err p95	recall
0.0	0.2432 ± 0	0 ± 0	0.5772 ± 0.01988	1.043 ± 0.07619	0.9454 ± 0.01103
0.25	0.264 ± 0.001838	0 ± 0	0.2893 ± 0.01638	0.7877 ± 0.06544	0.9454 ± 0.01103
0.5	0.2835 ± 0.002123	0 ± 0	0.2003 ± 0.02808	0.6013 ± 0.1195	0.9454 ± 0.01103
1.0	0.2948 ± 0.0006128	0 ± 0	0.1815 ± 0.03109	0.5715 ± 0.1199	0.9454 ± 0.01103
2.0	0.3096 ± 0.001061	0 ± 0	0.1592 ± 0.02339	0.5361 ± 0.1069	0.9454 ± 0.01103
4.0	0.3152 ± 0.002671	0 ± 0	0.1523 ± 0.02454	0.5262 ± 0.1153	0.9454 ± 0.01103
8.0	0.3252 ± 0.003184	0 ± 0	0.1382 ± 0.01213	0.4728 ± 0.07563	0.9454 ± 0.01103
16.0	0.3434 ± 0.003677	0 ± 0	0.1341 ± 0.01333	0.4557 ± 0.06814	0.9454 ± 0.01103
32.0	0.3656 ± 0.006457	0 ± 0	0.127 ± 0.008716	0.4326 ± 0.05711	0.9454 ± 0.01103

Table C.4: E4 quality weight lambda (columns 6 to 10)

setting	fetch ms	flash	migr	reenc	overflow B
0.0	0.09789 ± 0.002523	723.7 ± 37.56	572.7 ± 6.944	93 ± 0	0 ± 0
0.25	0.101 ± 0.001069	763 ± 16.08	610.3 ± 18.35	175.7 ± 1.7	0 ± 0

setting	fetch ms	flash	migr	reenc	overflow B
0.5	0.1062 ± 0.002955	836.3 ± 43.76	631 ± 10.98	188.3 ± 2.494	0 ± 0
1.0	0.1079 ± 0.003259	859.7 ± 48.4	639.3 ± 16.94	213.7 ± 3.3	0 ± 0
2.0	0.1112 ± 0.00247	907.7 ± 36.75	629.7 ± 14.52	226.3 ± 1.7	0 ± 0
4.0	0.1973 ± 0.03053	2189 ± 455.1	567 ± 5.715	226.3 ± 2.867	0 ± 0
8.0	0.2453 ± 0.003006	2912 ± 44.85	567.3 ± 24.28	222.7 ± 1.247	0 ± 0
16.0	0.237 ± 0.004147	2792 ± 60.42	613.7 ± 20.29	214.7 ± 1.886	0 ± 0
32.0	0.2535 ± 0.006791	3036 ± 101.6	726 ± 52.71	205 ± 3.559	0 ± 0

Table C.5: E5 capacity factor, combined system (columns 1 to 5)

setting	res/FP16	drop	err	err p95	recall
0.5	0.2878 ± 0.008107	0 ± 0	0.2204 ± 0.0104	0.6296 ± 0.03874	0.9454 ± 0.01103
0.75	0.2935 ± 0.002671	0 ± 0	0.1801 ± 0.02752	0.5765 ± 0.1167	0.9454 ± 0.01103
1.0	0.3096 ± 0.001061	0 ± 0	0.1592 ± 0.02339	0.5361 ± 0.1069	0.9454 ± 0.01103
1.5	0.3226 ± 0.004246	0 ± 0	0.1306 ± 0.009824	0.4271 ± 0.06095	0.9454 ± 0.01103
2.0	0.33 ± 0.001621	0 ± 0	0.115 ± 0.01631	0.3613 ± 0.06399	0.9454 ± 0.01103
3.0	0.3239 ± 0.002808	0 ± 0	0.1518 ± 0.02296	0.5418 ± 0.09728	0.9454 ± 0.01103

Table C.6: E5 capacity factor, combined system (columns 6 to 10)

setting	fetch ms	flash	migr	reenc	overflow B
0.5	0.3011 ± 0.01844	3729 ± 272.7	863.7 ± 35.46	200.7 ± 5.437	3.311e+04 ± 3205
0.75	0.1415 ± 0.005091	1360 ± 76.25	804.7 ± 40.14	229.7 ± 1.7	0 ± 0
1.0	0.1112 ± 0.00247	907.7 ± 36.75	629.7 ± 14.52	226.3 ± 1.7	0 ± 0
1.5	0.09922 ± 0.002136	728 ± 30.69	594.3 ± 9.672	221.3 ± 3.3	0 ± 0
2.0	0.08823 ± 0.002131	566.3 ± 29.51	548.3 ± 15.54	217 ± 0.8165	0 ± 0
3.0	0.04853 ± 0.0003229	3.333 ± 4.028	162 ± 6.532	225.7 ± 1.247	0 ± 0

D. Core Source Listings

The complete project is supplied as a separate archive; the listings below are the parts the report describes most closely. The remaining files are in the repository.

hw/rtl/uamm_sweep_scorer.v

```
// UAMM-2 sweep scorer: hardware "heat classifier" for the block table.
//
// Replaces the single-shot uamm_controller of UAMM-1:
//   * no divider: weights are integers summing to 256 (154/64/38 ~= 60%/25%/15%), score = sum >> 8
//   * block metadata lives in a small SRAM; one sweep streams every block through a 2-stage pipeline
//     (1 block/cycle) so thousands of blocks are scored per planner interval
//   * ‘protected‘ (sink / recent window) is honoured in hardware, matching the Python planner
//
// Metadata word (25 bits): {protected, frequency[7:0], recency[7:0], importance[7:0]}
// Output tier code: 2'b00 hot (T0/T1), 2'b01 warm (T2), 2'b10 cold (T3).
// Synthesizable Verilog-2001.
module uamm_sweep_scorer #(
    parameter NBLK = 64,
    parameter AW   = 6            // AW = log2(NBLK)
) (
    input  wire          clk,
    input  wire          rst_n,
    // metadata write port
    input  wire          we,
    input  wire [AW-1:0] waddr,
    input  wire [24:0]   wdata,
    // sweep control
    input  wire          start,       // ignored while busy
    input  wire [7:0]    pressure,    // memory pressure, latched at start
    output reg           busy,
    output reg           done,        // pulses with the last out_valid of a sweep
    // result stream
    output reg           out_valid,
    output reg  [AW-1:0] out_idx,
    output reg  [7:0]    out_score,
    output reg  [1:0]    out_tier,
    output reg           out_evict
);
    localparam [7:0] W_IMP   = 8'd154;
    localparam [7:0] W_REC   = 8'd64;
    localparam [7:0] W_FRQ   = 8'd38;
    localparam [7:0] TH_HOT  = 8'd174;
    localparam [7:0] TH_WARM = 8'd89;
    localparam [7:0] EVICT_P = 8'd243;

    reg [24:0] mem [0:NBLK-1];
    always @(posedge clk) if (we) mem[waddr] <= wdata;

    reg [AW:0]    rd_cnt;
    reg [7:0]     pres;
    reg           s0_v;
    reg [AW-1:0]  s0_idx;
    reg [24:0]    s0_data;

    // synchronous SRAM read, address = issue counter
    always @(posedge clk) s0_data <= mem[rd_cnt[AW-1:0]];

    wire [7:0]  imp   = s0_data[7:0];
    wire [7:0]  rec   = s0_data[15:8];
    wire [7:0]  frq   = s0_data[23:16];
    wire        prot  = s0_data[24];
    /* verilator lint_off UNUSEDSIGNAL */
    wire [15:0] wsum  = (imp * W_IMP) + (rec * W_REC) + (frq * W_FRQ);  // <= 255*256
    /* verilator lint_on UNUSEDSIGNAL */
    wire [7:0]  score = wsum[15:8];   // wsum[7:0] intentionally dropped (>> 8)
    wire [1:0]  tier  = prot ? 2'b00 :
                        (score >= TH_HOT)  ? 2'b00 :
                        (score >= TH_WARM) ? 2'b01 : 2'b10;
    wire        evict = (!prot) && (pres >= EVICT_P) && (score < TH_WARM);
    localparam [31:0] LAST_IDX = NBLK - 1;
```

```
    wire        last  = s0_v && ({{(32-AW){1'b0}}, s0_idx} == LAST_IDX);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            busy <= 1'b0;  done <= 1'b0;  rd_cnt <= {(AW+1){1'b0}};  pres <= 8'd0;
            s0_v <= 1'b0;  s0_idx <= {AW{1'b0}};
            out_valid <= 1'b0;  out_idx <= {AW{1'b0}};  out_score <= 8'd0;
            out_tier <= 2'b10;  out_evict <= 1'b0;
        end else begin
            // ---- stage 1: score the block read last cycle
            out_valid <= s0_v;
            done      <= last;
            if (s0_v) begin
                out_idx   <= s0_idx;
                out_score <= score;
                out_tier  <= tier;
                out_evict <= evict;
            end
            // ---- stage 0: issue reads
            if (start && !busy) begin
                busy <= 1'b1;  rd_cnt <= {(AW+1){1'b0}};  pres <= pressure;  s0_v <= 1'b0;
            end else if (busy && (rd_cnt < NBLK)) begin
                s0_v   <= 1'b1;
                s0_idx <= rd_cnt[AW-1:0];
                rd_cnt <= rd_cnt + 1'b1;
            end else begin
                s0_v <= 1'b0;
            end
            if (last) busy <= 1'b0;
        end
    end
endmodule
```

E. UAMM-1 Source Code (Original Prototype)

This appendix reproduces, unchanged, the prototype that Chapter 4 analyses: the Verilog controller and its testbench (the Python modules are in the repository). The concatenated listings are also stored in the archive under `uamm1_original/`. They are the authors' own earlier code and are included so that the comparison of UAMM-1 and UAMM-2 can be checked against the actual implementation.

The Python modules of UAMM-1 are not reproduced here because of their length; they are in `uamm1_original/uamm1_python_listing.txt` in the repository.

D.1 UAMM-1 Verilog controller and testbench

```
‘timescale 1ns/1ps
module tb_uamm_controller;
    reg clk = 0;
    always #5 clk = ~clk;
    reg rst_n = 0, valid = 0;
    reg [7:0] importance, recency, frequency, memory_pressure;
    wire out_valid, evict;
    wire [7:0] score;
    wire [1:0] tier;

    uamm_controller dut(.clk(clk), .rst_n(rst_n), .valid(valid),
        .importance(importance), .recency(recency), .frequency(frequency),
        .memory_pressure(memory_pressure), .out_valid(out_valid),
        .score(score), .tier(tier), .evict(evict));

    task apply;
        input [7:0] h, r, f, p;
        input [1:0] expected_tier;
        input expected_evict;
        begin
            @(negedge clk);
            valid=1; importance=h; recency=r; frequency=f; memory_pressure=p;
            @(posedge clk); #1;
            if (!out_valid || tier !== expected_tier || evict !== expected_evict) begin
                $display("FAIL h=%0d r=%0d f=%0d p=%0d score=%0d tier=%b evict=%b",
                         h,r,f,p,score,tier,evict);
                $fatal(1);
            end
        end
    endtask

    initial begin
        importance=0; recency=0; frequency=0; memory_pressure=0;
        repeat(2) @(posedge clk);
        rst_n=1;
        apply(8’d255,8’d255,8’d255,8’d0,2’b00,1’b0);
        apply(8’d150,8’d100,8’d80,8’d0,2’b01,1’b0);
        apply(8’d10,8’d10,8’d10,8’d255,2’b10,1’b1);
        $display("PASS: controller decisions");
        $finish;
    end
endmodule

// UAMM policy decision engine - synthesizable Verilog-2001.
// Inputs are normalized unsigned 8-bit values (0..255).
module uamm_controller (
    input wire        clk,
    input wire        rst_n,
    input wire        valid,
    input wire [7:0] importance,
    input wire [7:0] recency,
    input wire [7:0] frequency,
    input wire [7:0] memory_pressure,
    output reg        out_valid,
    output reg [7:0] score,
    output reg [1:0] tier,        // 00 GPU, 01 CPU, 10 NVMe
```

```
    output reg        evict
);
    // Weighted score: 60% importance + 25% recency + 15% frequency.
    // Sum of weights is 100; divide by 100 to retain 8-bit scale.
    wire [15:0] weighted_sum =
        (importance * 8'd60) +
        (recency    * 8'd25) +
        (frequency * 8'd15);
    wire [7:0] computed_score = weighted_sum / 16'd100;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            out_valid <= 1'b0;
            score       <= 8'd0;
            tier        <= 2'b10;
            evict       <= 1'b0;
        end else begin
            out_valid <= valid;
            if (valid) begin
                 score <= computed_score;
                 if (computed_score >= 8'd174)
                      tier <= 2'b00; // GPU
                 else if (computed_score >= 8'd89)
                      tier <= 2'b01; // CPU
                 else
                      tier <= 2'b10; // NVMe

                  evict <= (memory_pressure >= 8'd243) &&
                           (computed_score < 8'd89);
              end else begin
                  evict <= 1'b0;
              end
        end
    end
endmodule
```

List of Tables

1.1 Order-of-magnitude comparison between a data-centre accelerator and an edge SoC. Ranges are representative, not measurements of one device. . . 1

1.2 KV-cache size for four illustrative model geometries (FP16, one sequence). Geometries are representative of model families, not of any specific checkpoint. 3

1.3 Single-technique fixes and why each leaves part of the problem open. . . 5

2.1 Chapters of Sarangi's Basic Computer Architecture [27] and how we used them. 7

2.2 Levers used by the existing work. 8

2.3 Joint and system-level work reviewed, and how UAMM-2 relates to each. 14

2.4 Sources, inferences and effects on the design. 18

4.1 Gaps at the level of ideas. 23

4.2 Implementation defects found on review and the response to each. 24

4.3 The files of UAMM-1. 25

5.1 Tiers used in the simulations. Capacities are deliberately small so that the budget is tight at a few thousand tokens; latencies and bandwidths are assumed values (Section 6.2). 29

5.2 Measured size and reconstruction error per codec for a 16×64 block of K and V (mean over test blocks; synthetic data, see Section 6.2). The z variants reduce size slightly on flash-like data; for 4-bit and 2-bit they add a few bytes of overhead on random-like blocks. 32

5.3 Python modules. 37

5.4 Hardware files. 39

5.5 Test files. 40

5.6 Design decisions and how each one was reached. 42

5.7 Design problems and what we did about them. 43

6.1 What the results rest on. 45

6.2 Status of each element of the evaluation. 45

6.3 Metrics reported in the experiments. 46

6.4 Default configuration of the experiments. 47

6.5 Experiments performed. 48

6.6 UAMM-1 against UAMM-2 and a one-codec baseline (1,536 tokens, 168 KB, mean $\pm$ std over 5 seeds). 48

6.7 Ablation and baselines at 1,536 tokens (mean ± std over 5 seeds). The first column of sizes does not count the key min/max summaries; the second does. 50

6.8 Baselines that place blocks without a cost model (hottest block first into the fastest tier with room) against the planner over its quality weight λ (1,536 tokens, 3 seeds). 51

6.9 Effect of charging the per-block key min/max summaries to the 168 KB budget (1,536 tokens, 5 seeds). 53

6.10 Context scaling with the 168 KB budget fixed (mean of 3 seeds). UAMM-2 columns use all codecs unless stated. (columns 1 to 5) 53

6.11 Context scaling with the 168 KB budget fixed (mean of 3 seeds). UAMM-2 columns use all codecs unless stated. (columns 6 to 9) 53

6.12 Effect of the attention block budget at 1,536 tokens (mean ± std, 3 seeds). 54

6.13 Effect of the quality weight λ at 1,536 tokens (3 seeds). 54

6.14 Hysteresis margin (3 seeds). 56

6.15 H2O decay (3 seeds). 57

6.16 Block size (3 seeds; block budget scaled to keep ≈384 non-protected tokens fetched). 57

6.17 Memory-budget sweep at 1,536 tokens (mean of 3 seeds). 58

6.18 Baselines across memory budgets at 1,536 tokens (3 seeds). 59

6.19 Codec-set study at 1,536 tokens (3 seeds). 59

6.20 Key quantisation axis: relative squared error of keys only (200 random 16 × 64 blocks per row). 60

6.21 Key findings from the simulations. 61

6.22 Sweep scorer results. 64

6.23 Codec results. 65

6.24 Block-table results. 66

6.25 Yosys `synth -flatten` generic-cell statistics (technology-independent). The raw reports are in the repository under `docs/captures`. 67

6.26 Mapping from the scalar algorithm to the RVV instructions. 68

6.27 Llama-3-8B-like geometry (131,072 B per token in FP16). The mixed rows use the resident ratios measured in E1 and assume the same mix at this scale. 71

6.28 Projected KV-read bound for the 8B-like geometry. For 8,192 tokens the measured read fraction is used; beyond that the fixed 24-block budget plus a 36-token protected set is assumed. Weight reads and compute are ignored, and the bound is not a prediction of end-to-end speed. 71

6.29 Evidence available and evidence needed for each claim. 72

7.1 Suggested order of work. 79

8.1 The first two weeks. 81

8.2 From the first draft to the final report. 81

9.1 Repository layout. 83

A.1 Summary of the Turnitin similarity check. 87

B.1 Counts used for the readability indices. 93
B.2 Readability indices of the main text. 93

C.1 E1 ablation (5 seeds, 1,536 tokens) (columns 1 to 5) 94
C.2 E1 ablation (5 seeds, 1,536 tokens) (columns 6 to 10) 94
C.3 E4 quality weight lambda (columns 1 to 5) 94
C.4 E4 quality weight lambda (columns 6 to 10) 94
C.5 E5 capacity factor, combined system (columns 1 to 5) 95
C.6 E5 capacity factor, combined system (columns 6 to 10) 95

List of Figures

1.1 KV-cache size grows linearly with context. Dashed lines show plausible free-DRAM budgets on edge devices; every geometry except the smallest crosses one of them within the context lengths that applications now request. 3

1.2 Upper bound on decode speed from KV-cache reads alone (12.8 GB/s DRAM). The curves fall as 1/context: this is the bandwidth wall that appears even when the cache fits in memory. 4

2.1 Capability coverage of the related work. No single prior system covers placement, precision, query-aware selection, importance scoring, a hardware decision engine and a vector kernel together. 17

3.1 Mind map of the project, step by step. 20

5.1 UAMM-2 system architecture: the decode loop, the paged cache with its tiers and codecs, the importance and planning loop, and the hardware and vector-kernel attachments. 29

5.2 Which codecs are permitted in which tier. Flash only accepts entropy-coded variants; the warm DRAM tier only accepts 4- and 2-bit codecs. 31

5.3 Planner flow: candidate enumeration, cost evaluation, greedy capacity-aware selection and the hysteresis check. 34

5.4 One decode step: bound ranking, fetch of selected blocks, attention, H2O update and (every few steps) a planning pass. 35

5.5 Sweep scorer micro-architecture: metadata SRAM, weighted sum, threshold comparators, protected override and eviction logic. 36

6.1 UAMM-1 against UAMM-2 on the same trace and budget. Labels: 4-bit only = every block in 4-bit with no planner; INT8+4 = UAMM-2 with INT8 and 4-bit codecs; all = UAMM-2 with every codec. 49

6.2 Error against resident size (left) and against modelled fetch time (right) for the no-planner baselines and for the planner over its λ sweep. 52

6.3 Context scaling at a fixed budget. UAMM-1's error and drops grow quickly beyond 768 tokens; UAMM-2 degrades gradually and keeps every token until the budget is exceeded near 3,072 tokens. 54

6.4 Block budget sweep. Left: recall of the min/max-bound selector versus the oracle choice of the B heaviest blocks averaged over sampled steps (indicative; the system also reads the protected blocks). Middle and right: error and fetch cost. 55

6.5 Effect of λ on error, memory, fetch latency and flash reads. 56

6.6 Left: churn versus hysteresis margin. Middle: minimum residency. Right: planning interval. 56

6.7 Budget sweep. UAMM-1 needs roughly three times the baseline budget to bring error below 0.09 and still drops 261 tokens; UAMM-2 reaches comparable error without drops at much smaller budgets. 58

6.8 State of the cache at step 1,536: tier (row) and codec (colour) of every block, with block importance and selection frequency. Older, low-importance blocks occupy flash at 2- or 4-bit; recent blocks are in T0 at FP16. 61

6.9 Sweep start. start is asserted for one clock; busy rises on the next edge; the read counter then increments once per cycle. s0_data shows the metadata words read from the SRAM; the results appear two cycles later as out_ valid with the block index, score and tier. 63

6.10 Eviction under pressure. The pressure latched for the sweep changes from 242 to 243; in the sweep at 243, out_evict is raised for blocks 0, 2 and 4 (unprotected, score below 89) and not for the protected blocks or for blocks 6 and 7 (score 89). 64

6.11 End of a sweep. The read counter reaches 64 and busy falls; the last result (block 63) is emitted one cycle later together with the done pulse. A new start can follow immediately. 64

6.12 Asynchronous reset in the middle of a sweep. rst_n is pulled low; busy, s0_v and out_valid drop immediately, the read counter returns to zero and a later start begins a clean sweep. 65

6.13 Two vectors and a constant group through uamm_kv_quant4 and uamm_ kv_dequant4. x_flat is the 128-bit input; q_flat is the packed result one cycle after in_valid; the dequantiser is fed the code vector and step and produces x' one cycle later. 65

6.14 Random updates and lookups. we pulses write {tier, codec, phys} to the address on waddr; rd_en pulses start a lookup whose result (hit, tier, codec, phys) appears one cycle later through rd_valid. A lookup of a never-written address returns hit = 0; a hit returns the stored fields. 66

6.15 Output of make -C hw test: all three testbenches print PASS with their check counts. 66

6.16 Output of make -C hw lint (verilator –lint-only -Wall on every module). Exit status 0, no warnings. 67

6.17 Mutation results: 12 of 12 injected faults are detected. 67

6.18 Scalar C kernel versus NumPy dequantise-then-dot on random blocks, and the RVV compile check. 69

6.19 RVV assembly for the 4-bit key kernel (excerpt of the compiler output). . 70

8.1 Project timeline. Bars show the approximate period of each activity. . . . 81

Glossary

Ara Open-source RISC-V vector processor.. 102

Attention sink The first tokens, which attract attention whatever their content.. 102

Attention-mass recall Share of the dense attention probability that falls on the tokens that were read.. 102

Block Fixed group of 16 consecutive tokens; the unit of placement.. 102

Codec Storage format of a block: FP16, INT8, 4-bit or 2-bit, optionally entropy-coded. Our own term.. 102

H2O Heavy-Hitter Oracle: keeps tokens with the largest accumulated attention.. 102

Heavy hitter A token that collects a large share of attention.. 102

Hysteresis Rule that blocks a move unless it is clearly better than staying.. 102

KIVI Quantisation scheme that scales keys per channel and values per token.. 102

KV cache Stored keys and values of all previous tokens, per layer and head.. 102

Planner The routine that picks a (tier, codec) pair for each block by comparing costs.. 102

Precision ratchet Rule that a block's precision may only go down.. 102

Quality weight The factor lambda that scales the quantisation-error term in the planner cost.. 102

Quest Query-aware selection of KV blocks using per-block min/max key bounds.. 102

RTL Register-transfer level hardware description, here written in Verilog.. 102

RVV RISC-V Vector extension.. 102

Tier A memory level: T0 SRAM, T1 and T2 DRAM, T3 flash.. 102

UAMM Unified Adaptive Memory Manager, the name of our design (UAMM-1 and UAMM-2).. 102

VCD Value change dump: the waveform file a simulator writes, opened in GTKWave.. 102

Index

Ara, 15, 16, 18, 35, 36, 43, 69, 73, 84
attention sink, 26

baseline, 23, 43, 44, 48, 50–53, 59, 69, 72, 73, 84
block table, 9, 16, 22–24, 68, 73, 77
budget, 3, 5, 10, 11, 19, 24, 41–44, 47, 48, 50, 51, 53, 54, 57–60, 71–73, 76–78, 83, 84

codec, 6, 9, 13–15, 28–33, 37, 38, 40, 43, 44, 47, 50–52, 60, 62, 72, 74, 79, 83

EvicPress, 18, 28

flash, 4, 5, 7, 12, 14, 19, 21, 22, 24, 29, 30, 33, 42, 50–52, 55, 57, 59, 60, 76, 77, 79, 94
FP16, 3, 6, 7, 24, 28, 30, 31, 41, 43, 44, 47, 48, 50, 51, 53, 59, 60, 62, 70–72, 75, 76, 78, 79, 84, 94

GTKWave, 62

H2O, 9–11, 17, 22, 23, 25, 26, 28, 32
hysteresis, 38, 56, 93

InfiniGen, 12, 17
INT8, 7, 41, 51–54, 60, 70, 75

KIVI, 13, 17, 28, 31, 41, 51–54, 60
KV cache, 2–5, 7–9, 12–14, 16, 21, 22, 24, 29, 83

migration, 6, 33
mutation testing, 19, 67, 73

PagedAttention, 9, 22, 28, 41, 78
planner, 14, 16, 28–30, 32–34, 38, 41–44, 47, 48, 51–57, 59, 60, 62, 72, 74–79, 84
precision ratchet, 72
prefill, 11

quantisation, 7, 10, 13, 14, 21, 22, 24, 28, 31, 35, 40, 46, 51, 72, 76, 83, 93
Quest, 12, 17, 28, 42, 72

recall, 24, 47, 50, 54, 71, 78, 94
residency, 51, 52, 54–57, 72
RISC-V, 1, 6, 15, 16, 18, 21, 36, 42, 68

Scissorhands, 10, 17
selective attention, 28
SnapKV, 11, 17
StreamingLLM, 10, 17, 28
sweep scorer, 57, 62
synthetic trace, 40, 48, 50

tier, 5, 6, 9–12, 14, 17, 21, 28–30, 32, 33, 35, 38, 41, 44, 47, 50–53, 57, 59, 60, 63, 72, 76, 78, 83

UAMM-1, 1, 7, 20–28, 35, 38, 40, 41, 43, 44, 50, 51, 53, 54, 58, 59, 62, 63, 72, 73, 76, 79, 83, 84, 98
UAMM-2, 1, 7, 10, 15, 16, 18, 20, 23, 26, 28, 38, 41, 47, 50, 53, 54, 58, 59, 62, 72, 74, 76, 83, 84, 98

Verilator, 25, 34, 62
vLLM, 9

Yosys, 63, 68